

Verification Continuum™

Verdi® Performance Analyzer User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	7
Synopsys Statement on Inclusivity and Diversity	7

1. Performance Analyzer Overview 9

2. Verdi Performance Analyzer GUI	11
Invoking Verdi Performance Analyzer	11
Verdi Performance Analyzer Window Overview	12
Hierarchy Tree Pane	12
Performance Report Pane	21
Details Pane	28

3. Using Verdi Performance Analyzer	30
Usage Summary	31
Shortcut Keys	31
Loading Metrics From Multiple Directories	31
Sorting and Filtering	32
Search Field for Hier. Tree, Metrics, and Results Tabs	35
Customizing Range and Aggregate Functions	36
Selecting Data for x and y Axis	36
Splitting the Data for the x Axis Into Ranges	38
Aggregate Functions for Range	39
Applying Chart Configurations	40
Defining Groups for Metrics	42
Adding Metrics to Groups	43
Enabling and Disabling Groups	43
Displaying Data Type and Unit for Metric Results Columns	44
Setting the Original Data Type and Data Unit	45
Setting the Display Unit	47
Synchronization Between Performance Analyzer and Transaction and Protocol Analyzer	50

Selection Synchronization	51
Synchronization From Performance Analyzer to Transaction and Protocol Analyzer	52
Synchronization From Transaction and Protocol Analyzer to Performance Analyzer	52
Synchronization Between Performance Analyzer and nWave Windows	53
Cross-Instance Relationship	55
Support for Instance-Based Constraints	56
Support for Multiple Protocol Report Generation in Single Window	57
Support for Merge/Split of Metrics Data in a Single Graph	59
Support for Summary Report	62
Use Model	62
New Tcl Commands (Summary Report)	63
Support for Show/Hide Column and Column Ordering in Detail Tree	64
Use Model	64
New Tcl Commands (column ordering)	66
New Tcl Commands (show and hide columns)	66
Support for Precision for Double Values	68
New Tcl Commands	68
Support for Column Value Sum in the Details Tab	69
New Tcl Commands	69
Support for Exporting of csv Report and HTML Report Generation for Selected Metric	70
New Tcl Commands	71
Improving Debug Messages for SQL Flow	71
Resolving Missing Attributes	72
Use Model	72
4. Batch Mode Operation	74
Batch Mode Use Model	74
Batch Mode Tool	75
Parameters	76
Protocol Extensions	77
Batch Mode Processing Flow	77
Batch Mode Commands	78

5. Aggregate Performance Analysis	83
Prerequisites	83
Generating Aggregate Performance Analysis Report	84
verdi_perf Utility	86
Viewing Aggregate Performance Analysis Report in GUI	88
Files Tab for Multiple FSDB Files Database	90
Hierarchy Tree Tab for Multiple FSDB Files	90
Metrics Tab for Multiple FSDB Files	91
Results Tab for Multiple FSDB Files	91
Performance Report Pane for Multiple FSDB Files	91
Details Tab for Multiple FSDB Files	92
Examples to Demonstrate Merged Results for Multiple FSDB Files	92

6. Custom Performance Analyzer Metrics	98
Elements of a Performance Metric	99
Performance Metric Definitions	99
Performance Analyzer SQL Database	100
Examples of a Performance Metric Definition	103
Instance and Multi-Instance Performance Metrics	112
Primary and Dependent Performance Metrics	115
Examples of Dependent Performance Metric Definition	117
Multi-Instance Performance Metrics	120
Summary of Performance Metric Definition Types	121
Performance Metric Column Data Types	122
Performance Metric Visualization	124
Types of Performance Metric Visualization	125
Controlling Metric Visualization	126
Default Metric Visualization	127
Simple Chart Types	128
Aggregating Chart Data into Ranges	132
Viewing, Modifying, and Creating Performance Metrics	134
VIP PreDefined Performance Metrics	134
Directory Structure and Files	135
Merging Custom Metrics With PreDefined Metrics	138
Summary	139

7. Performance Analysis Runtime APIs	140
Introduction	140
API Descriptions	141
new (constructor)	141
disable_all_metrics	142
enable_all_metrics	142
disable_metric	142
enable_metric	143
set_metric_constraints	143
set_metric_min_constraint	143
set_metric_max_constraint	144
clear_metric_constraints	144
clear_metric_min_constraint	145
clear_metric_max_constraint	145
analyze_performance	145
get_metric_data	146
has_violation	147
has_min_violation	147
has_max_violation	148
Source code and Examples	148

Preface

The Verdi Performance Analyzer User Guide provides graphical performance, and statistical and transaction analysis of protocol related data.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also

Contents

includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Performance Analyzer Overview

The *Verdi Performance Analyzer* is an interactive graphical application that provides graphical performance, and statistical and transaction analysis of protocol related data. It provides a convenient way to measure the performance of protocols using protocol-related transaction information that has been recorded in an FSDB (for example, from Synopsys Verification IP, Custom Verification IP, or from emulation).

Performance Analyzer is based on *Protocol Analyzer*. It works as a feature of *Protocol Analyzer* and provides protocol-oriented analysis, compliance checking, report generation, and report comparison. In addition to accelerating the investigation of protocol behavior, *Performance Analyzer* also defines several extendable interfaces and commands that can be used to integrate *Performance Analyzer* to the regression system or other VIP tools.

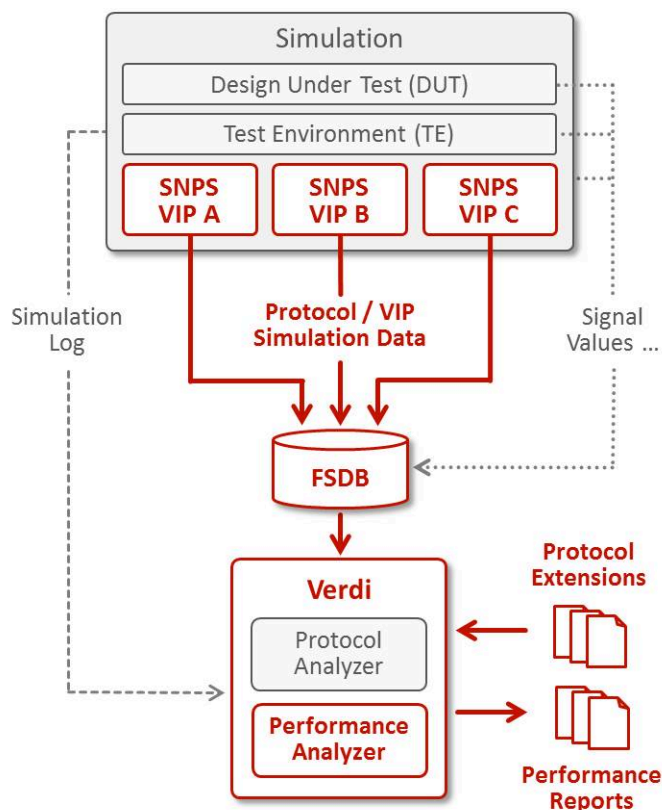
Performance Analyzer is a protocol-aware platform that supports different protocols with different versions. It also provides extendable interfaces to support new protocols. By default, *Performance Analyzer* provides common performance metrics and report layouts. VIP developers and end users can also define their own performance metrics and report layouts for different protocols.

The following are the key features of *Performance Analyzer*:

- **Protocol-aware platform:** *Performance Analyzer* is a general protocol-aware platform that supports different protocols with different versions. It provides extendable interfaces to support new protocols.
- **Performance metric customization:** You can customize the performance metrics for different protocols. In addition to the metric definition, *Performance Analyzer* also provides convenient ways for you to manage and configure performance metrics.
- **Performance report customization:** *Performance Analyzer* uses WYSIWYG visual designer and layout tools to create performance reports. You can easily add metrics and UI controls using the drag-and-drop feature, change various properties of visual component using a property editor, and apply compliance check rules.
- **Graphical view:** *Performance Analyzer* provides different types of charts and controls, such as bar chart, pie chart, and line chart to display the performance data. Related charts and controls can be grouped into the section control.

- **Multi format report:** Performance reports can be exported in different formats, such as PDF, HTML, and Excel file. You can also export the simulation performance data to a CVS file, so that you can perform data analysis using Microsoft Excel.
- **Batch mode capability:** Most functions in *Performance Analyzer* can be run in the non-GUI mode. You can call functions by typing the commands in the *Console* view. Full batch mode capability can also be used in regression tests.

Figure 1 *Performance Analyzer Use Model*



Performance metric calculations are performed using the same transaction data that is recorded during simulation for viewing in Verdi *Transaction and Protocol Analyzer*. Thus, in order to calculate performance metrics for one or more VIP instances in the design, FSDB dumping must be enabled for those instances using the same process that is used to enable viewing and analysis of protocol-related data in Protocol Analyzer. Pre-defined performance metrics are provided for select Synopsys VIP titles (AXI/ACE, AHB, DDR, USB, Ethernet, and so on). Refer to the individual VIP documentation for details about Verdi *Performance Analyzer* support.

2

Verdi Performance Analyzer GUI

This chapter discusses about how to invoke *Verdi Performance Analyzer* and provides details on the *Performance Analyzer* window layout and commands in the following sections:

- [Invoking Verdi Performance Analyzer](#)
- [Verdi Performance Analyzer Window Overview](#)

Invoking Verdi Performance Analyzer

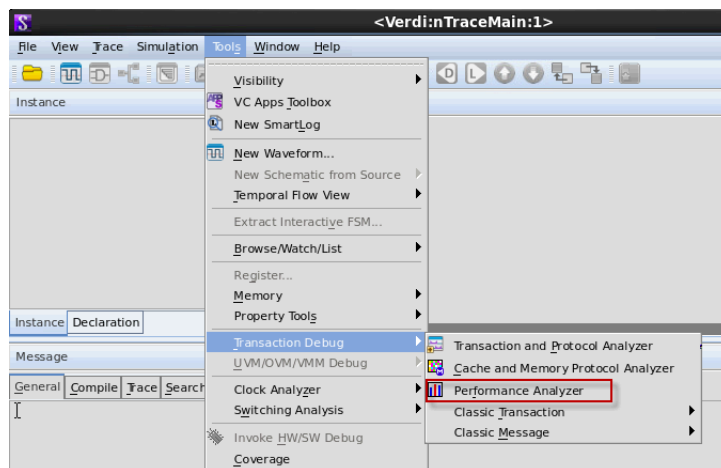
The *Performance Analyzer* application in the Verdi platform is used to measure the performance of protocols and provides performance, statistical, and transaction analysis of protocols.

Note:

To use *Verdi Performance Analyzer*, make sure that the `VerdiPerformanceAnalyzer` license feature is available in the license server.

Verdi Performance Analyzer is invoked with the **Tools -> Transaction Debug -> Performance Analyzer** command in the *nTrace* window as illustrated in the following figure:

Figure 2 Invoking Performance Analyzer From nTrace



Verdi Performance Analyzer Window Overview

The default layout of the *Performance Analyzer* window is illustrated in the [Performance Analyzer Default Layout](#). The *Performance Analyzer* window is divided into the following panes:

- The left region contains the *Hierarchy Tree* pane that displays the instance hierarchy, protocol metrics, and performance results.
- The middle region contains the *Performance Report* pane that displays the performance report.
- The right region contains the *Details* pane that displays the details associated with the selected metric result.

Figure 3 Performance Analyzer Default Layout



If there is a primary FSDB file, any newly-opened *Performance Analyzer* window automatically opens the primary FSDB file; otherwise, the newly-opened *Performance Analyzer* pane is empty. You can then open an FSDB file with the **File -> Open** command or the **Open FSDB File** icon.

The selection is synchronized between *Performance Report* pane, the *Metrics* tab, and the *Results* tab. The *Details* tab shows metric results and constraint violations for the selected metric.

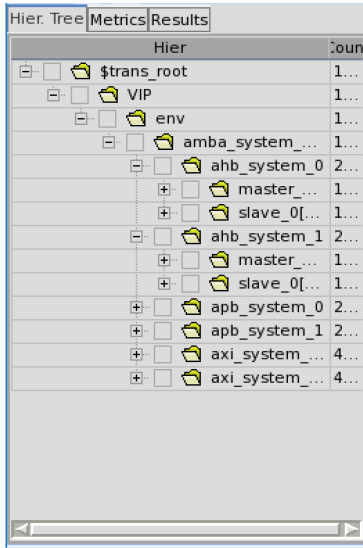
Hierarchy Tree Pane

The *Hierarchy Tree* pane contains [Hier. Tree Tab](#), [Metrics Tab](#), and [Results Tab](#).

Hier. Tree Tab

The **Hierarchy Tree** tab is used to display all VIP instances in the design. The hierarchy is extracted from the FSDB file and is the same hierarchy that is displayed for the *Protocol Analyzer*. However, only the instances are checkable. The hierarchy view is used to select the instances for which performance metrics are to be calculated.

Figure 4 The Hierarchy Tree Tab



The following right-click commands are available in the *Hierarchy Tree* tab:

- **Expand All:** Expands all items.
- **Collapse All:** Collapses all items.
- **Show in Result View:** Shows the selected instance in the result view.
- **Show Report:** Evaluates the enabled performance metrics for the selected instances and displays the Performance Report.
- **Show Report in New Window:** Evaluates the enabled performance metrics for the selected instances and displays the Performance Report in a new *Performance Analyzer* pane.

Only one Performance Report is displayed in the new *Performance Analyzer* pane. You can open the Performance Report in a new window using the **Show Report in New Window** command.

If instances from multiple protocols are selected for performance evaluation, only the instances from the first protocol are used and a warning message appears to enable you to view performance reports for other instances in the new *Performance Analyzer* pane.

Metrics Tab

The *Metrics* tab displays metrics and its associated constraints for all protocols in the design. If there are multiple instances of the same protocol in the design, there is still only one root node for that protocol. This tab allows you to enable/disable metrics, define/edit custom metrics and constraints, or add/ change/remove metric constraints.

Figure 5 The Metrics Tab

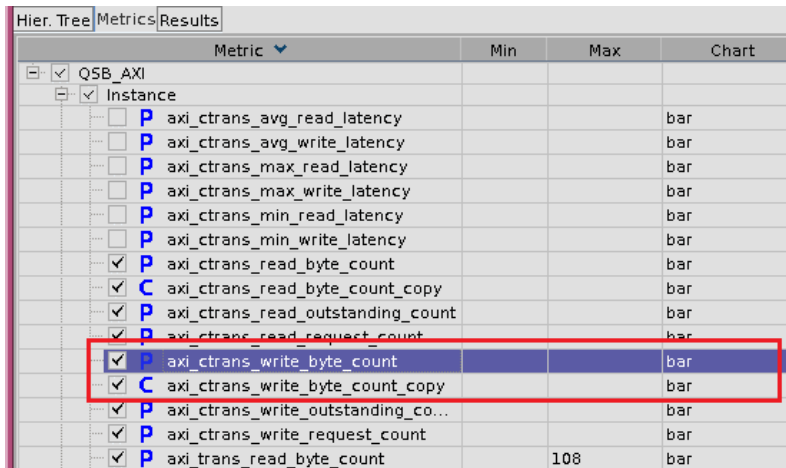
Hier. Tree Metrics Results				
Metric		Min	Max	Chart
QSB_AXI				
Instance				
☐	P axi_ctrans_avg_read_latency			bar
☐	P axi_ctrans_avg_write_latency			bar
☐	P axi_ctrans_max_read_latency			bar
☐	P axi_ctrans_max_write_latency			bar
☐	P axi_ctrans_min_read_latency			bar
☐	P axi_ctrans_min_write_latency			bar
☒	P axi_ctrans_read_byte_count			bar
☒	P axi_ctrans_read_outstanding_count			bar
☒	P axi_ctrans_read_request_count			bar
☒	P axi_ctrans_write_byte_count			bar
☒	P axi_ctrans_write_outstanding_co...			bar
☒	P axi_ctrans_write_request_count			bar
☒	P axi_trans_read_byte_count		108	bar
☒	P axi_trans_read_latency			bar
☒	P axi_trans_write_byte_count			bar
☒	P axi_trans_write_latency			bar
Multiple Instances				
☒	P axi_cinst_read_byte_count			bar
☒	P axi_cinst_read_request_count			bar
☒	P axi_cinst_read_request_percentage			pie
☒	P axi_cinst_write_byte_count			bar
☒	P axi_cinst_write_request_count			bar
☒	P axi_cinst_write_request_percenta...			pie

For each protocol, the tree of performance metrics has two sub-nodes: one node for the instance performance metrics and another node for the multi-instance performance metrics. The corresponding metrics appear as leaf-level tree nodes within each of these sub-nodes.

Performance metrics can be enabled or disabled by clicking the check boxes in the hierarchy tree. The check box for the instance folder and the protocol folder is used to enable or disable all metrics in the folder.

Predefined metrics are differentiated from custom metrics by the icon of the metric name. You cannot delete or edit predefined metrics.

Figure 6 Enable Performance Metrics by Clicking Check Box



The following right-click commands are available in the *Metrics* tab:

- **Expand All:** Expands all items.
- **Collapse All:** Collapses all items.
- **Add Metric:** Displays a dialog box for adding a custom metric.
- **View Metric:** Shows the details of the selected metric in the **Metric** dialog box.
- **Edit Metric:** Opens the *Add/Edit Metric* form in which you can edit a custom metric. Only custom metrics can be edited. Predefined metrics cannot be edited, but they can be copied with the **Copy Metric** command.
- **Copy Metric:** Displays a dialog box for copying the selected metric. All fields remain the same with the selected metric except the metric name.
- **Delete Metric:** Deletes the selected metric.
- **Set Chart:** Displays a dialog box to set the chart option for the selected metric.
- **Set Column Datatype:** Sets the original data type and data unit for a selected column of the metric.
- **Save Configuration:** Saves the current metric/constraint configuration into a Tcl file.
- **Load Configuration:** Loads a metric/constraint configuration from a Tcl file.
- **Reset Constraint:** Resets the constraints for the protocol of the selected metric. You are prompted with a dialog box to confirm the operation.
- **Reset Configuration:** Resets both metrics and constraints for the protocol of the selected metric. You are prompted with a dialog box to confirm the operation.

If the metric definition contains any syntax error, a warning message appears when you attempt to create or modify the metric.

Any runtime errors associated with the metric definition, such as an incorrect table name or column name, can only be detected when the metric is evaluated. This class of errors are annotated in the *Results* tab.

Custom performance metrics can be defined with **New Metric** and **Edit Metric** right-click commands. If one of these commands are invoked, the *Add/Edit Metric* form is opened as illustrated in the following figure where you can specify or modify the performance metric definition:

Figure 7 *The Add/Edit Metric Form*

Add/Edit Metric

Name:

Protocol:

Description:

Type:

Dependent Metric:

Definition:

In the *Metrics* tab, double-click **Min** and **Max** columns of the metric to set minimum and maximum constraint for that metric.

Figure 8 *Set the Minimum/Maximum Constraint for a Metric*

Metric	Min	Max	Chart
AMBA_SVT AXI_SVT			
Cross Instance			
✓ P ace_cinst_read_bus_bandwidth			bar
✓ P ace_cinst_read_bus_bandwidth_percentage	566		pie
✓ P ace_cinst_read_byte_count			bar
✓ P ace_cinst_read_byte_count_percentage		60	pie
✓ C ace_cinst_read_byte_count_percentage_c...			bar
✓ P ace_cinst_read_request_count		780	bar
✓ P ace_cinst_read_request_percentage			pie
✓ C ace_cinst_read_request_percentage_copy			bar

The **Set Chart** command is used to set the metric display chart option as illustrated in the following figure:

Figure 9 Set Chart Type Form

The screenshot shows the 'Set Chart Type' dialog box. At the top, the 'Metric' is 'ace_trans_avg_read_latency' and the 'Protocol' is 'AMBA_SVT AXI_SVT'. Below this is a 'Data' section with 'x axis' set to 'avg_latency' and 'y axis' empty. A 'Customize Range' section is checked, containing 'Start', 'End', and 'Step' input fields, a 'function' dropdown set to 'avg', and an unchecked checkbox 'Show data items out of range'. At the bottom, the 'Chart Type' is 'Bar Chart'. Buttons for 'Default', 'OK', and 'Cancel' are at the very bottom.

The following settings are used for each chart type:

- **Chart Type**
 - **Bar chart:** Shows the bar chart using the first column as the y axis and the second column as the x axis.
 - **Line chart:** Shows the line chart using the first column as the y axis and the second column as the x axis.
 - **Pie chart:** Shows the pie chart using the first column as the percentage of slice and the second column as the slice tag (name).
 - **Step chart:** Shows the count changes on the x axis in the form of steps. This chart uses the data from x and y axis to compute and form a range.

- **Data**

Use the same column as both x axis and y axis in the Data field to specify the frequency.

- **Customize Range**

Customize the range, such as specify start, end, step, and function in the **Customize Range** field.

Results Tab

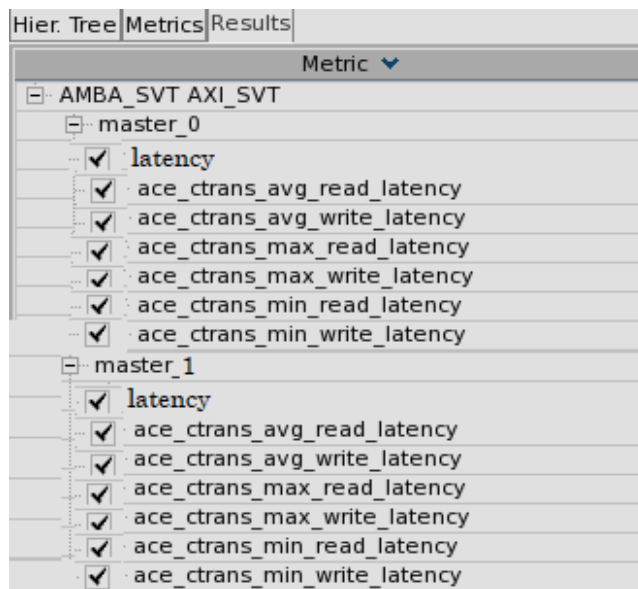
The *Results* tab is a tree view that displays performance analysis results. The root folder for the tree is the protocol name. This folder contains a sub-item that contains the results of evaluating all multi-instance metrics and separate sub-items for each instance that was evaluated.

The *Results* tab also shows the grouping information of the metrics. Groups are created only in the *Metrics* tab. After you evaluate the report, the same grouping is applied to the *Results* tab.

In addition, check boxes are present in the *Results* tab, which enable you to select the metrics that you want to view. By default, all the metrics selected in the *Metrics* tab are selected inside the *Results* tab; you need to de-select the metrics for the instances for which you do not want to view the reports.

The following figure illustrates the *Results* tab:

Figure 10 The Results Tab



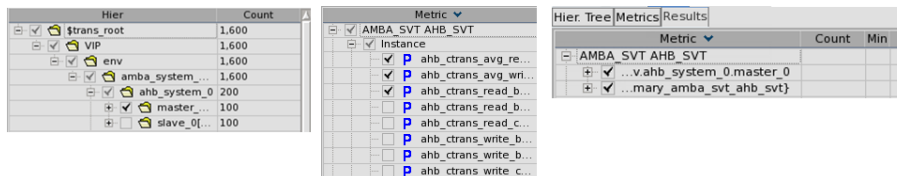
The next level in the tree hierarchy displays the individual metrics (instance or multi-instance) that are evaluated. Constraint violations are indicated with an alternate background color for the violating metrics.

The *Count* column shows the number of results generated for the metric. If a runtime error occurs when evaluating the metric, *NA* is shown and the tip tag provides details of the error.

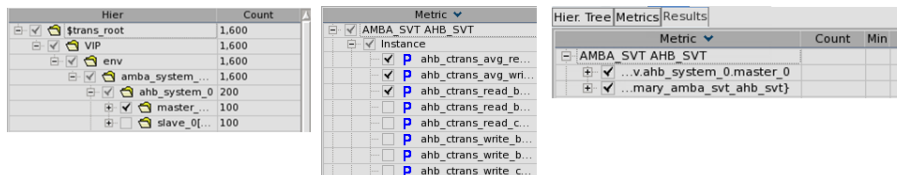
Any metric that has constraint violations is highlighted in a different color, purple for maximum constraint violations, yellow for minimum constraint violations, and red for both. Selections in the *Results* tab are synchronized automatically with the corresponding item in the *Results* tab of the *Performance Report* pane. For example, if the *trans_write_latency* metric is selected for *instance2*, the *Results* tab of the *Performance Report* pane scrolls to the section for *instance2* and highlights the chart for the *trans_write_latency* metric automatically.

There are multiple scenarios to consider while displaying the information in the *Results* tab and the results inside the *Chart Section*. Following are the different use cases with examples:

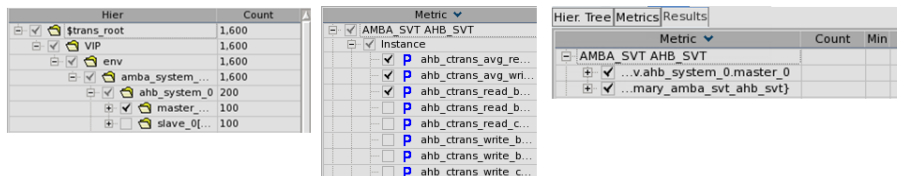
- When you open an FSDB file, the results are empty until you select any instance inside the *Hier. Tree*. After you select one instance from the *Hier. Tree*, the other panes are updated. The results are populated as shown in the below figure, but the *Performance Report* pane shows the results only after you click the **Show Report** action.



- When you de-select metrics in the *Results* tab, the *Performance Report* pane automatically updates to show the results only for Metric1 and Metric3. Results for Metric2 are not displayed.



- When you de-select/select metrics from the *Metrics* tab, the *Results* tab is automatically updated with the selection/de-selection as shown in the following figure:



Note that, the *Results* tab is updated with the  **Results** icon. After you click the **Show Report** icon with the updated metric configuration, the report is re-evaluated.

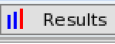
- When you select *Instance2* from *Hier.Tree*, the view is updated as shown in the following figure:

The figure shows three screenshots of the Verdi Performance Analyzer GUI. The first screenshot shows the **Hier** tab with a tree view where *Instance2* is selected. The second screenshot shows the **Metric** tab with a list of metrics where *Instance2* is selected. The third screenshot shows the **Results** tab with a table of results for *Instance2*.

Hier	Count
\$trans_root	1,600
VIP	1,600
env	1,600
amba_system...	1,600
ahb_system_0	200
master...	100
slave_0[...]	100

Metric	Count
ahb_ctrans_avg_re...	
ahb_ctrans_avg_wri...	
ahb_ctrans_read_b...	
ahb_ctrans_read_b...	
ahb_ctrans_read_c...	
ahb_ctrans_write_b...	
ahb_ctrans_write_b...	

Metric	Count
env.amba_system.env.ahb_system_0.master_0	
env.amba_system.env.ahb_system_0.slave_0	
env.amba_system.env.ahb_system_1.master_0	
env.amba_system.env.ahb_system_1.slave_0	
ahb_ctrans_avg_read_latency	0

Note that, the *Results* tab is updated with the  icon. After you click the **Show Report** icon with the updated instance configuration, the report is re-evaluated.

- When you de-select an instance in the *Hier.Tree* tab, the *Results* tab is automatically updated with the new state. The following figure shows the new state of all the three tabs:

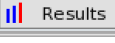
The figure shows three screenshots of the Verdi Performance Analyzer GUI. The first screenshot shows the **Hier** tab with a tree view where *Instance2* is de-selected. The second screenshot shows the **Metric** tab with a list of metrics where *Instance2* is de-selected. The third screenshot shows the **Results** tab with a table of results for *Instance2*.

Hier	Count
\$trans_root	1,600
VIP	1,600
env	1,600
amba_system...	1,600
ahb_system_0	200
master...	100
slave_0[...]	100

Metric	Count
ahb_ctrans_avg_re...	
ahb_ctrans_avg_wri...	
ahb_ctrans_read_b...	
ahb_ctrans_read_b...	
ahb_ctrans_read_c...	
ahb_ctrans_write_b...	
ahb_ctrans_write_b...	

Metric	Count	Min
env.amba_system_0.master_0		
env.amba_system_0.slave_0		
env.amba_system_1.master_0		
env.amba_system_1.slave_0		
ahb_ctrans_avg_read_latency		

The *Performance Report* pane is updated to show the results only for *Instance2* and all the selected metrics.

If there are any changes to instance or metric selection after the **Show Report** option is clicked, the *Results* tab is updated as , which indicates that the report is out-of-date. You must click the **Show Report** option again to view the updated report.

Note:

The *Results* tab shows instance grouping; however, the *Metrics* tab shows generic grouping. There is no option available to group metrics based on the instance; the generic metrics grouping is applied inside each instance.

The following right-click commands are available in the *Results* tab:

- Expand All:** Expands all items.
- Collapse All:** Collapses all items.
- Show in Hier. Tree:** Shows the selected instance in the *Hierarchy Tree* tab.
- Show in Metrics View:** Shows the selected metric in the *Metrics* tab.
- Show Metric:** It displays the metric definition as illustrated in the following figure:

Add/Edit Metric (on vginthw204)

Name:

Protocol:

Description:

Type:

Dependent Metric:

Definition:

```
SELECT latency AS min_latency, request_time AS request_time, response_time AS response_time, event AS event
FROM $inst_ace_trans_read_latency
ORDER BY latency ASC LIMIT 1;
```

- **Save Configuration:** Saves the current metric/constraint configuration into a Tcl file.
- **Load Configuration:** Loads a metric/constraint configuration from a Tcl file.
- **Reset Configuration:** Resets the configuration settings to default settings.
- **Clear Instance Constraints:** Clears the constraints for all the instances.

Note:

You can save or load configurations from the *Metrics* tab or the *Results* tab. If there are errors while saving or loading the configuration file, these errors are reported and the process is aborted.

Performance Report Pane

The *Performance Report* pane displays the performance report of the specified protocol.

After the performance evaluation is completed for the selected instances, a metric report is generated and displayed in the *Performance Report* pane.

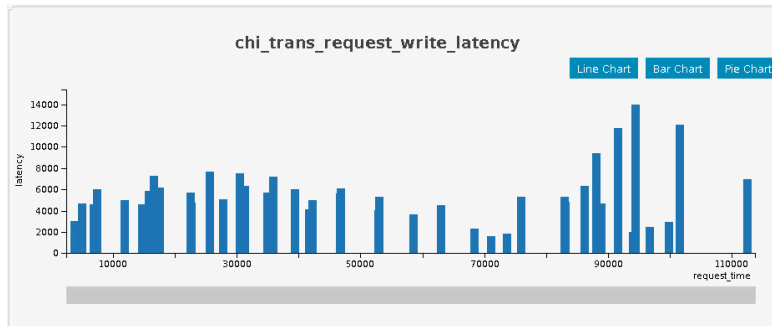
As previously explained, multi-instance metrics are evaluated only once on all input instances, whereas instance metrics are evaluated on every input instance. The performance report includes a separate section for each input instance and a summary section for all input instances. The instance section includes performance results for all instance performance metrics; the summary section only includes performance results for multi-instance performance metrics.

A performance metric can generate a result that has only one record or contains multiple records. The result can also have only one column or can include multiple columns.

Note:

If a performance result has multiple columns, the first column is taken as the value of the metric.

Figure 11 *Performance Report*



For a metric result with only one record, the result is shown as a label.

Figure 12 *Result as Label*

Average Read Latency : 1280

If there is a constraint violation for the metric, the result is shown in a different color.

Figure 13 *Result in Different Color in Label*

Maximum Write Latency : 6300

For a metric result with multiple records, the result is shown as either a bar chart or a line chart. If there are constraints associated with the metric, the constraint values are also displayed.

Use the middle button of mouse to scroll up/down to zoom in/out the chart, and drag left and right to move the chart left and right. The scroll bar below the chart is also used to move the chart left/right and zoom in and out.

For line/bar charts, some of the details may be lost if there are too many items in the visible range. Since some of the date items may overlap, zoom in the chart for several times makes all items in the desired sub-range visible.

Figure 14 Line Chart of the Performance Report

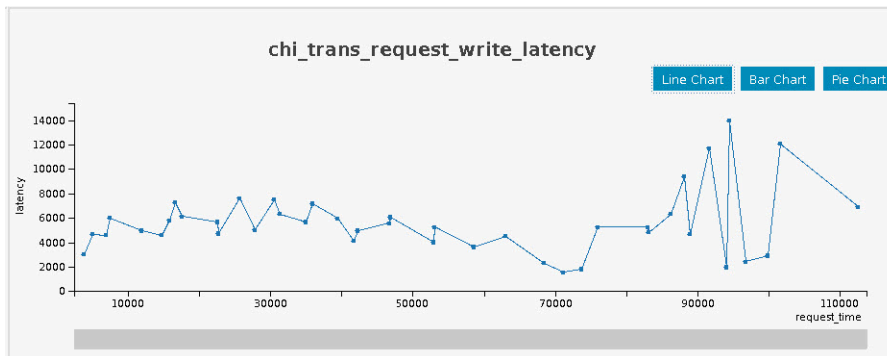
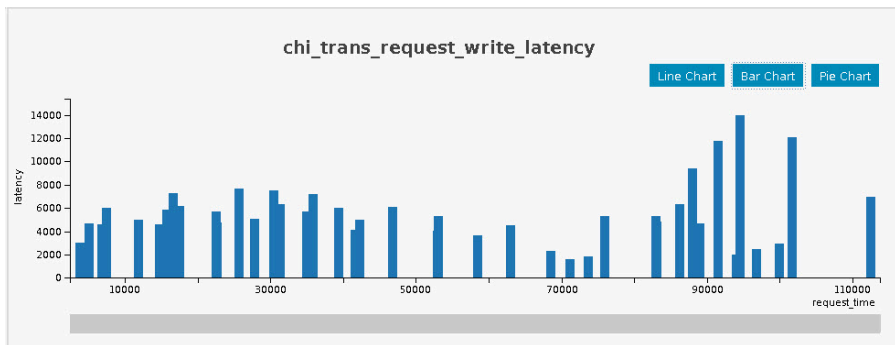


Figure 15 Bar Chart of the Performance Report

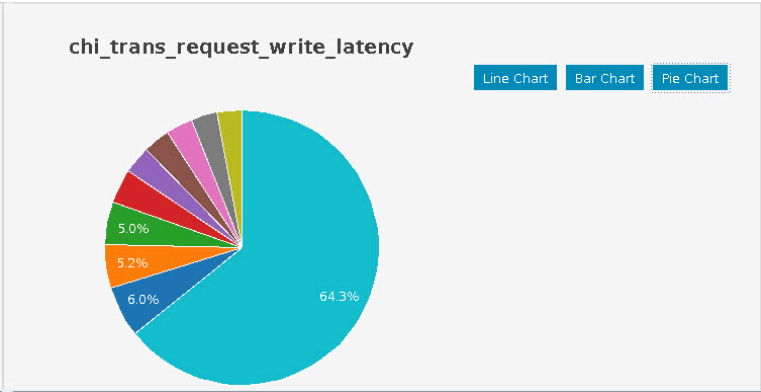


For pie charts, only the top ten items are shown in separate slices. Other items are grouped as one item with the name "others".

Note:

The selection of the pie chart does not synchronize to *Details* and *Results* pane. This is because the pie chart shows the ratio of various performance analysis results, and does not have one-to-one mapping with the results displayed in *Details* and *Results* pane.

Figure 16 *Pie Chart of the Performance Report*



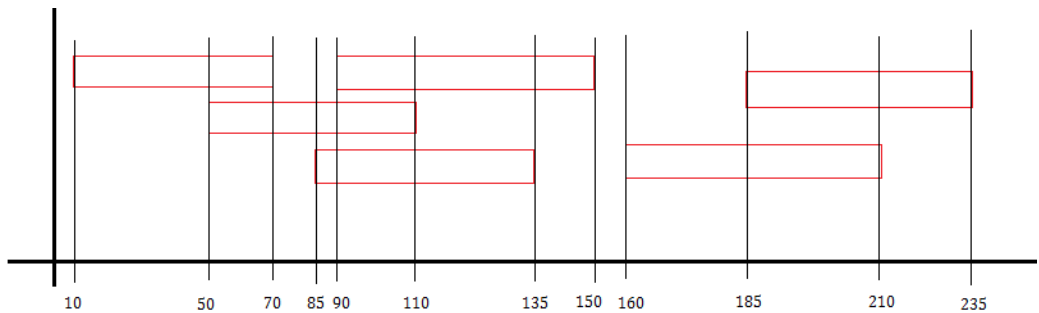
For step charts, count changes are shown on the x axis. This chart uses the data from x and y axis to compute and form a range.

Example

Consider the following metric results data:

Delay	Request_Time	Response_Time
60	10	70
60	50	110
50	85	135
60	90	150
50	160	210
50	185	235

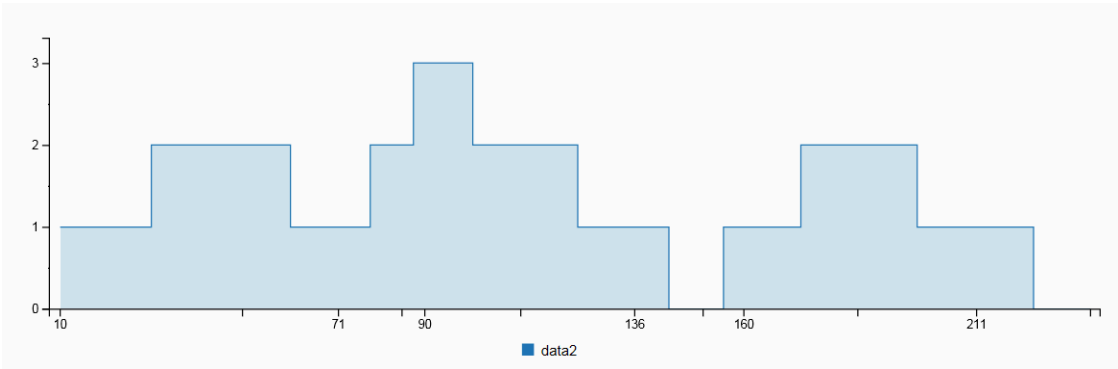
If *request_time* and *response_time* are selected as x and y axis data respectively, then the range formed by x and y axis data is as follows:



The final data for x and y axis is as follows:

x (time for count change)	10	50	71	85	90	111	136	151	160	185	211	236
x (count of transactions)	1	2	1	2	3	2	1	0	1	2	1	0

The step chart is as follows:

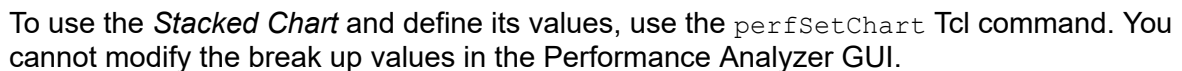


Points to Note for Step Chart

- The chart configuration is applied to the specified metrics. In other words, if a metric configuration is changed, then it is applied to all instances of the same metric. The chart configurations are saved when the session is saved and are restored when the session is loaded.
 - When the CSV format report is saved, the raw data is saved into the CSV file.
 - When the HTML format report is saved, the same chart setting is used that is present when the report is displayed.

- For Stacked Charts, you can view multiple y-axis values. Stacked Charts help you to view the break up in the total value, such as in the latency metrics, you can view the breakup of the latency value from the total latency.

Figure 17 Example of Stacked Chart



Syntax

The *Performance Report* pane includes a section for every input instance and a summary section for all input instances. The following figure shows a portion of an instance section:

The screenshot shows the Datadog Metrics Explorer interface. The left sidebar contains a list of metrics under the 'axi' namespace. The 'axi_trans_write_byte_count' metric is selected, and its details are shown in the main panel. The details panel displays a table with two columns: 'byte_count(Byte)' and 'write_time(us)'. The table shows data for the last 10 minutes, with values ranging from approximately 1,600,000 to 4,290,000 bytes and 160,000 to 2,630,000 microseconds.

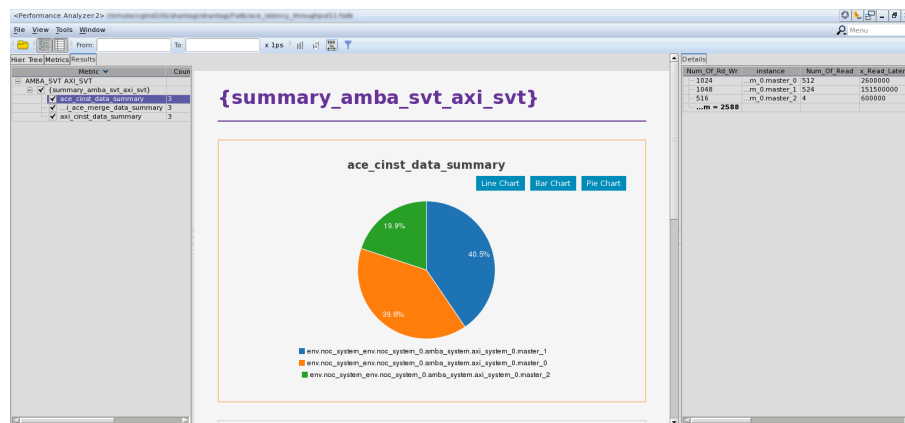
byte_count(Byte)	write_time(us)
1	1600000
4	2890000
3	4250000
4	5400000
2	6950000
2	7850000
3	9100000
2	10400000
2	11600000
1	12600000
3	13550000
2	14900000
1	15900000
4	16850000
4	17850000
2	18200000
4	20200000
3	21100000
3	22100000
2	23050000
4	24200000
4	25200000
2	26300000

Chapter 2: Verdi Performance Analyzer GUI

Verdi Performance Analyzer Window Overview

The following figure shows a portion of a summary section:

Figure 19 Performance Report - Summary Section



The following right-click commands are available in the *Performance Report* pane:

- **Save Report in HTML:** Saves the report in the HTML format.
- **Save Report in CSV:** Saves the report in the CSV format.

You can save the report in the HTML format that can be opened in any web browser supporting JavaScript and css3. You can also export the metric results into a CSV file that can be opened in Excel or OpenOffice as illustrated in the following figure:

Figure 20 Performance in the CSV Format

	A	B	C	D	E
1	file	instances	begin	end	
2	axi.fldb	master slave	1075	38279	
3					
4	instance				
5	master				
6					
7	trans_read_latency	request	response	min 1000	max 2000
8	1150	1075	2225	0	0
9	650	6125	6775	1	0
10	1550	8175	9725	0	0
11	4200	9775	13975	0	1
12					
13	ctrans_min_read_latency	min 1000			
14	650	1			
15					
16	ctrans_read_request	min 1000			
17	1250	0			
18					
19	instance				
20	slave				
21					
22	trans_read_latency	request	response	min 1000	max 2000
23	1150	1075	2225	0	0
24	1250	7975	9225	0	0
25	1550	8175	9725	0	0
26	4100	10825	14925	0	0
27					
28					
29	ctrans_min_read_latency	min 1000			
30	1150	0			
31					
32	ctrans_read_request	min 1000			
33	1100	0			
34					
35	instance				
36	master slave				
37					
38	cinst_reqd_request	min 2000			
39	2250	0			
40					

General Information

Section for "master" instance

Section for "slave" instance

Summary section

Note:
In the Performance Analyzer, the report section does not support global font.

Details Pane

For any metric result selected in the *Performance Report* pane, result data and any other data specified in the SQL metric expression is shown in the *Details* pane. Metric constraint violations are also annotated in this pane.

Details Tab

The *Details* tab displays the result data for any metric result selected in the *Performance Report* pane and includes all the data selected in the SQL metric expression as illustrated in the following figure:

Figure 21 The Details Tab

Details		
latency	request_time	response_time
450	1075	1525
3300	11075	14375
3650	33775	37425
500	50425	50925
3900	69375	73275
3400	81575	84975
850	96025	96875
3650	104425	108075
3550	118525	122075
3650	132275	135925
550	145625	146175
3700	170925	174625
3000	194125	197125
3350	205075	208425
3500	214925	218425
3950	225725	229675
2500	237125	239625
3950	269125	273075
3700	314275	317975
3950	336875	340825

The first column of the table displays the metric result. Any vector result (a result that has multiple records) is shown as a bar chart in the HTML report using the values from the first column as the data for the y-axis, and the values from the second column as the data for the x-axis.

If the table includes a column of transaction IDs, that information is used in the debugging process. For example, the transaction IDs are used to trace the ultimate cause for constraint violations or to synchronize selections with *Performance Analyzer*, *Transaction and Protocol Analyzer*, and *Transaction Table View* panes. In addition, the records that include transaction IDs are dragged and dropped to another pane to be viewed.

3

Using Verdi Performance Analyzer

This chapter discusses about the functionalities available in *Verdi Performance Analyzer* to help you to execute performance analysis in the following sections:

- [Usage Summary](#)
- [Shortcut Keys](#)
- [Loading Metrics From Multiple Directories](#)
- [Sorting and Filtering](#)
- [Search Field for Hier. Tree, Metrics, and Results Tabs](#)
- [Customizing Range and Aggregate Functions](#)
- [Defining Groups for Metrics](#)
- [Displaying Data Type and Unit for Metric Results Columns](#)
- [Synchronization Between Performance Analyzer and Transaction and Protocol Analyzer](#)
- [Synchronization Between Performance Analyzer and nWave Windows](#)
- [Cross-Instance Relationship](#)
- [Support for Instance-Based Constraints](#)
- [Support for Multiple Protocol Report Generation in Single Window](#)
- [Support for Merge/Split of Metrics Data in a Single Graph](#)
- [Support for Summary Report](#)
- [Support for Show/Hide Column and Column Ordering in Detail Tree](#)
- [Support for Precision for Double Values](#)
- [Support for Column Value Sum in the Details Tab](#)
- [Support for Exporting of csv Report and HTML Report Generation for Selected Metric](#)

- [Improving Debug Messages for SQL Flow](#)
- [Resolving Missing Attributes](#)

Usage Summary

To execute performance analysis using *Verdi Performance Analyzer*, perform the following steps:

1. Open the *Performance Analyzer* pane.
2. Open an FSDB file. Alternatively, use the primary FSDB file if it is loaded automatically.
3. Select instances from the *Hierarchy Tree* pane.
4. Set performance metrics and constraints (disable or enable metrics; set constraint values).
5. Perform the metric evaluation with **New Metric** and **Edit Metric** commands.
6. Open the performance report.
7. Check the performance report and debug any constraint violations.
8. Save the performance report in the HTML or the CSV format.
9. Open the HTML report in a web browser and review the report. Open the CSV report in Excel or OpenOffice, show charts for the data and review the report.

Shortcut Keys

The following table shows the available shortcut keys for the mentioned actions:

Action	Shortcut Key
File -> Save Report	Shift+S
File -> Close Window	Shift+C
File -> Reload	Shift+L

Loading Metrics From Multiple Directories

The Performance Analyzer allows you to load the metrics from both `PA_CUSTOM_VIP_PATH` and `DESIGNWARE_HOME` directories. You can define few metrics inside the `PA_CUSTOM_VIP_PATH` directory and remaining metrics in the `DESIGNWARE_HOME` directory.

Use Cases

- **Case1 - Only the `PA_CUSTOM_VIP_PATH` environment variable is set..**

In this case, there are no changes and the tool processes metric files from the `PA_CUSTOM_VIP_PATH` directory.

- **Case2 - Only the `DESIGNWARE_HOME` environment variable is set.**

In this case, there are no changes and the tool processes metric files from the `DESIGNWARE_HOME` directory.

- **Case:3 Both `PA_CUSTOM_VIP_PATH` and `DESIGNWARE_HOME` environment variables are set.**

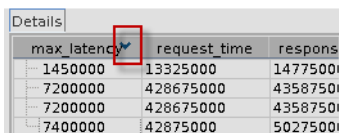
In this case, the tool firstly loads metric files from the `DESIGNWARE_HOME` directory. The tool then scans the `PA_CUSTOM_VIP_PATH` directory and overrides the metrics that exist in both paths.

Note:

The `PA_CUSTOM_VIP_PATH` environment variable allows you to specify multiple paths. However, the multiple paths must define metrics for different protocols; the specification of multiple protocols is not supported. If multiple occurrences of the same protocol are encountered, the first path in the list takes precedence.

Sorting and Filtering

You can sort metric result values present in the *Details* tab, the *Results* tab, and the *Metrics* tab in ascending or descending order. Click the head of a column to sort the metric result by that column in ascending or descending order.



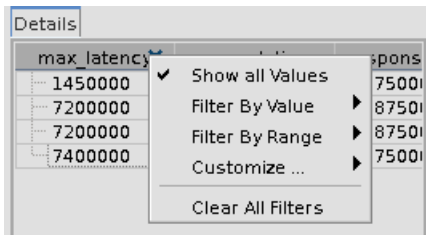
max_latency	request time	response
1450000	13325000	14775000
7200000	428675000	43587500
7200000	428675000	43587500
7400000	42875000	50275000

Note:

By default, sorting is not applied to the results displayed in the columns.

You can also filter metric result values present in the *Details* tab using the filtering mechanism that works on the specified metric result on the specified instance. For example, if the *Details* view displays results for metrics "M" for instance "I", the filtering mechanism is applied only to the current result (metrics "M" on instance "I"); it is not applied to the results of metric "M" on other instances.

To apply the filtering mechanism, right-click the head of a column in the *Details* tab, as illustrated in the following figure:



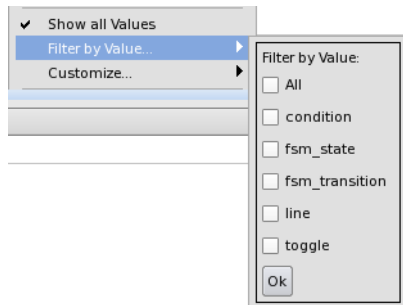
The following right-click options are available:

- **Show All Values**

Clears all the filters applied on a column.

- **Filter by Value**

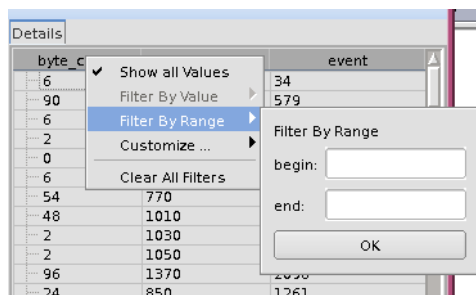
Applies filter based on the values present in a column. A list of values is displayed, which can be selected as per your preference.



Note that a maximum of ten values are displayed in the list menu for a column. If more than ten different values exist for a column, the **Filter by Value** option is disabled for that column.

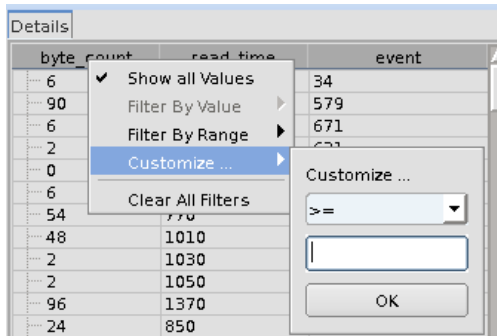
- **Filter By Range**

Applies a filter for a range of values. A range filter is applied only for numeric value columns.



- **Customize**

For a customized filter, you can specify the operator and the value of the filter, as illustrated in the following figure:



The following operators are supported for string and numeric values:

- >= greater and equal
- <= less and equal
- == equal
- != not equal
- ~= wildcard match, for example *abc*

You can apply filters on both the *Details* view and the *Performance Report* view. After the filter is applied, only the data items which pass the filter criteria are displayed in the *Details* view and the *Performance Report* view.

For single row metric results, all filters are disabled.

- **Clear all Filters**

Clears all the applied filters.

Note:

If you save a session for the current report, sorting, filtering, and color settings are saved in the session and are restored when you reload the session. However, if a new performance report is generated, all the filter settings are cleared for the new report.

Search Field for Hier. Tree, Metrics, and Results Tabs

To easily find data in *Performance Analyzer*, a search field is available in the *Hier. Tree* tab, the *Metrics* tab, and the *Results* tab. The search field is hidden by default, however, you can enable the search field using one of the following methods:

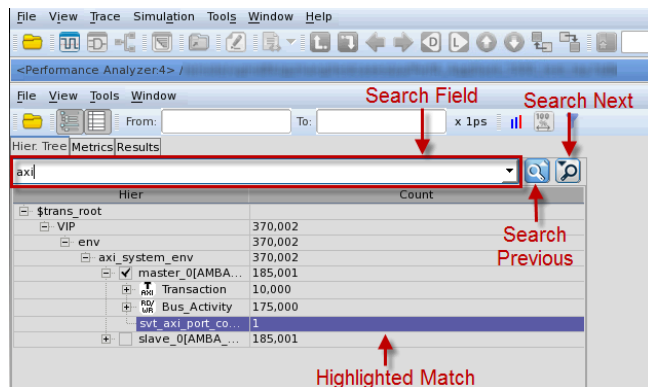
- Clicking the **Ctrl+F** keyboard key.

or

- Enabling the **Show Navigation Text Field** right-click option in *Hier. Tree/Metrics/Results* tab.

The search field is illustrated in the following figure:

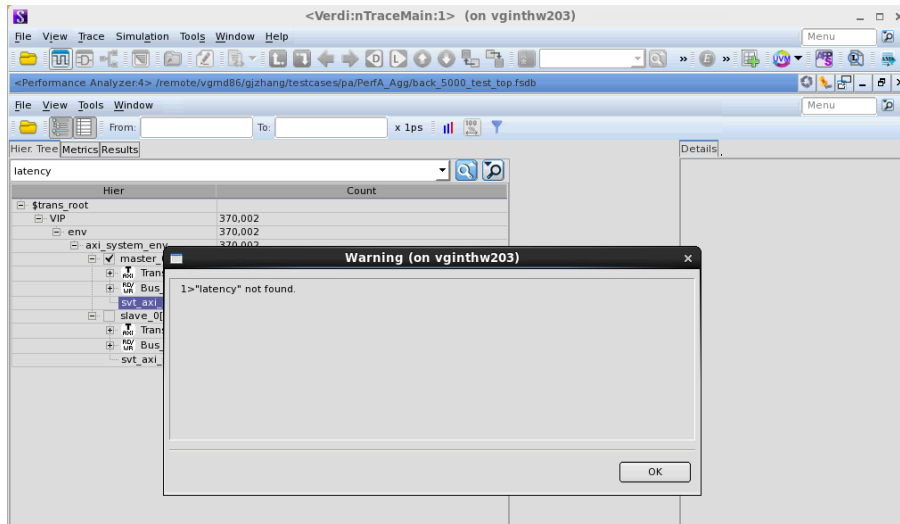
Figure 22 Search Field



The search field is an editable field that can show ten recently searched keywords. You can use the  icon to search previous row that contains the search keyword and the  icon to search the next row. The current matching row is highlighted in the *Hier. Tree* view and triggers the selection operation. You can enter the search keyword in the search field; the search engine scans the data row-by-row with all the columns' strings.

If the matched row is not expanded, the search engine automatically expands the hierarchy to show the row. If no matching results are found, a warning message is displayed as illustrated in the following figure:

Figure 23 Warning Message



Customizing Range and Aggregate Functions

Verdi Performance Analyzer allows you to perform the following:

- Select the data for x and y axis in the performance report chart.
- Split the data for the x axis into ranges and display only a statistical result item for every range in the report.
- Provide a variety of functions to get the y value for a range.
- Specify the type of chart for the performance report data.

Selecting Data for x and y Axis

You can select and specify the columns of the metric result to display as x and y axis in the performance report chart.

For the x axis:

- Any of the columns can be specified.
- The values can be numeric or string values.

For the y axis:

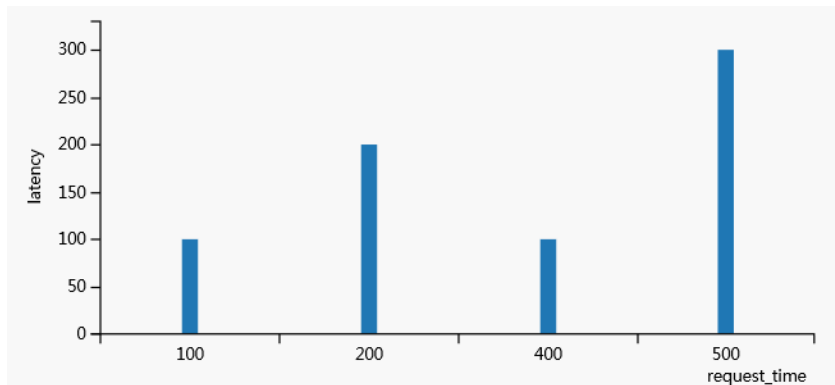
- Only columns with numeric values can be specified.

Example

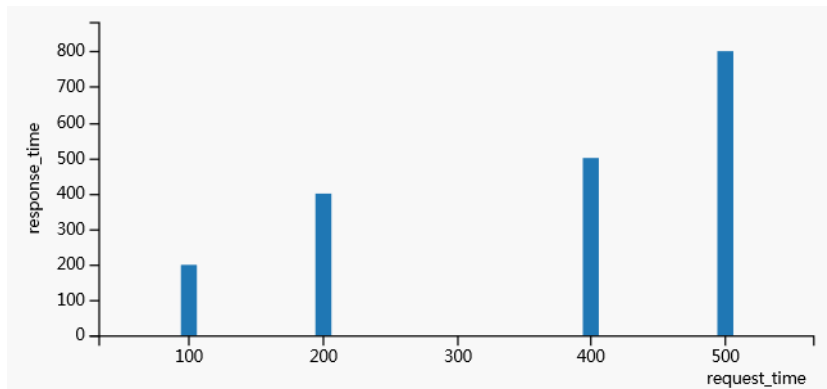
Consider the following metric results as an example:

Delay	Request_Time	Response_Time
100	100	200
200	200	400
100	400	500
300	500	800

If *request_time* is selected as the x axis and *delay* is selected as the y axis, then the bar chart is as follows:



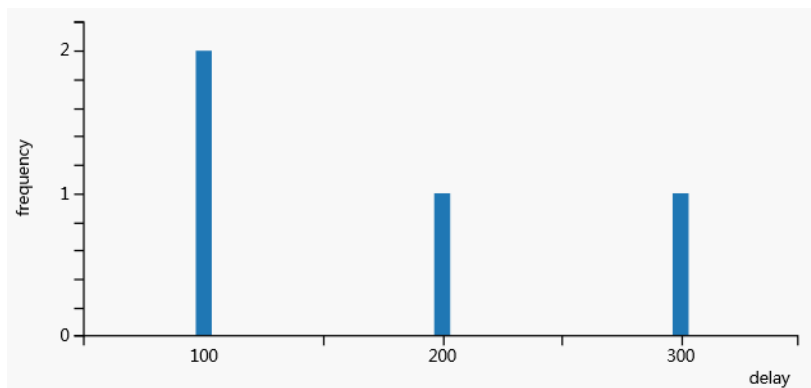
If *request_time* is selected as the x axis and *response_time* is selected as the y axis, then the bar chart is as follows:



Note:

The same column can also be used as both x and y axis. In this case, the frequency (that is, the count of appearance) of every distinct x value is the y value of that x value.

If *delay* is selected for both x and y axis, then the bar chart is as follows:



Splitting the Data for the x Axis Into Ranges

You can split the data for the x axis into fixed length ranges/intervals and display only a statistical result item for every range in the report.

Note:

This feature is supported only for numeric x values.

Use the following options to specify how the x axis must be split:

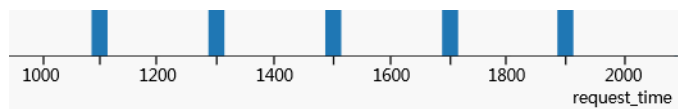
- The range width (required).
- The start of the range (optional; by default the minimal x value).

- The end of the range (optional; by default the maximal x value).
- Out-of-range data: ignore or take as one interval (optional; by default out-of-range data is ignored).

Example

If you specify 1000 as the start of the range, 2000 as the end of the range, and range width of 200, then the intervals are as follows:

Figure 24 Example



Aggregate Functions for Range

If the x axis is split into intervals, for every interval there may be multiple data items; however, only one data item is displayed for every interval. This value is a statistical result of all the data items in the interval that can be specified by the users.

The following aggregate functions are supported:

- `cnt`

It is used to show the count of data items in an interval.

- `max`

It is used to show the maximum value of the data items in an interval.

- `min`

It is used to show the minimum value of the data items in an interval.

- `sum`

It is used to show the sum of all the data items in an interval.

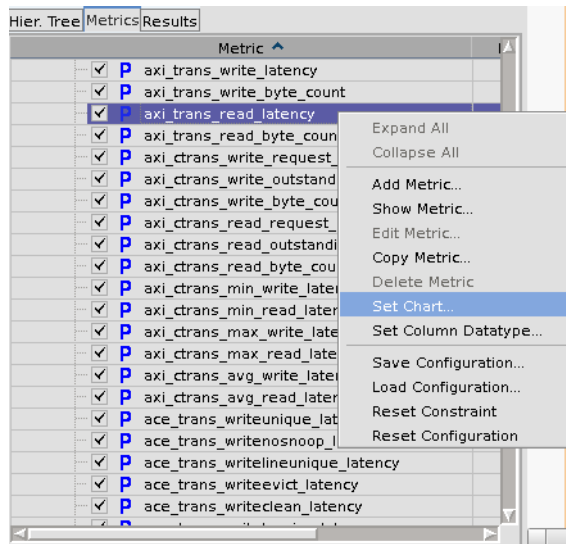
- `avg`

It is used to show the average value of all the data items in an interval.

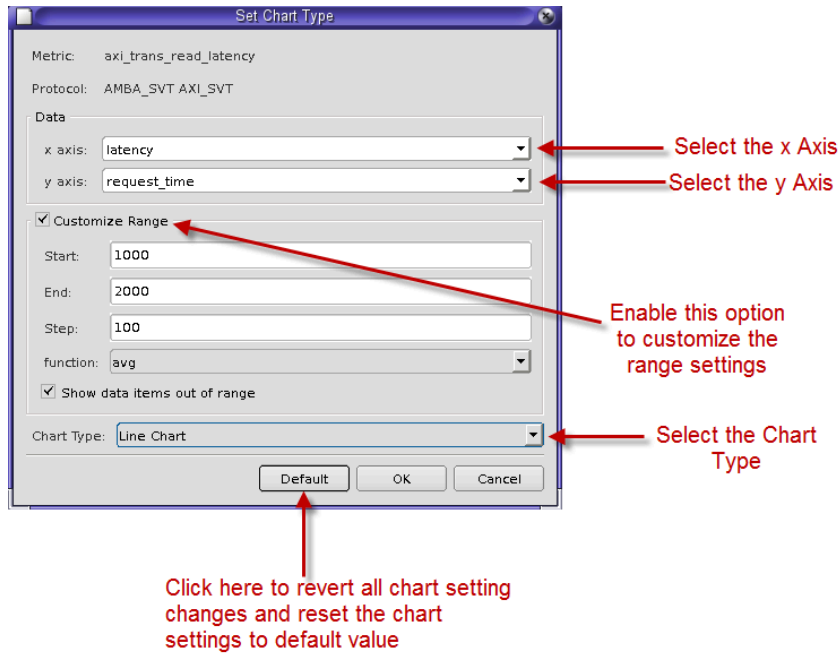
Applying Chart Configurations

To apply chart configurations using the Verdi GUI, perform the following steps:

1. In the *Metrics* tab, select an instance and click the **Set Chart** right-click option.



2. The *Set Chart Type* dialog box opens as illustrated in the following figure:



In this dialog box, perform the following:

- Specify x axis and y axis in the respective **Data** fields. By default, the first column is considered as y axis and the second column is considered as x axis.
 - Customize the range, such as specify start, end, step, and function in the **Customize Range** field. By default, this field is disabled.
 - Specify the chart type (Bar/Line/Pie/Step) in the **Chart Type** field. By default, Bar chart is selected.
3. Click the **Default** option to save the settings as default settings. Click the **OK** option to save the settings.

Tcl Command

To set the chart configuration for metrics, use the following Tcl command:

```
perfSetChart -protocol <protocol_name>
             -name <metric_name>
             -x <column_for_x_axis>
             -y <column_for_y_axis>
             -start <start_of_range>
             -end <end_of_range>
             -step <width_of_interval>
             -aggr <aggregate_function>
             -other <ignore_or_include_data_out_of_range>
             -type <chart_type>
```

Points to Note

- The chart configuration is applied to the specified metrics. In other words, if a metric configuration is changed, then it is applied to all instances of the same metric. The chart configurations are saved when the session is saved and are restored when the session is loaded.
 - When the CSV format report is saved, the raw data is saved into the CSV file.
 - When the HTML format report is saved, the same chart setting is used that is present when the report is displayed.
- Constraint violation highlighting is not supported for charts with split range.
- Constraints are always applied to the y axis. If you change the y axis column, the current constraint is removed.

Defining Groups for Metrics

The *Metrics* tab in the *Hierarchy Tree* pane of *Performance Analyzer* displays the metrics for all protocols in the design and allows you to enable and disable metrics. Each protocol node has two sub-nodes: *Instance* and *Multiple Instances*. The corresponding metrics appear as leaf-level tree nodes within each of these sub-nodes. However, as the number of metrics increases, it becomes difficult to enable or disable each metric individually.

Performance Analyzer provides you the capability to group together the metrics with specific meaning (with some rules), which allows you to enable or disable metrics based on groups.

Defining Groups

There are no explicit rules for metrics that can automatically separate metrics into several groups.

You need to add or define groups manually using the `perfGroup` Tcl command. The syntax of the `perfGroup` command is as follows:

```
perfGroup -protocol <protocol_name>
          -inst/-cinst
          -create <group_name>    //support hierarchical group
                                   name
```

To delete a group, use the following Tcl command:

```
perfGroup -protocol <protocol_name>
          -inst/-cinst
          -delete <group_name>    //support hierarchical group
                                   name
```

Points to Note

- A group only acts on metrics level; *Instance* and *Multiple Instances* sub-nodes have their own metric groups for each protocol.
- Once the groups are created, they cannot be modified.
- A group has a hierarchical structure. All the commands related to groups must use the full hierarchical name.
- The name of a group in each level only supports a-z, A-Z, and 0-9 characters. The dot '.' character is used to separate the group hierarchy.
- You can either create a simple group, such as `bandwidth`, or a hierarchical group, such as `latency.read`.

Adding Metrics to Groups

After the groups are created, you can adjust the hierarchy of the metrics by moving the metrics to related groups. You can move any metric (whether in another groups or not) to a specific group, or move the metric out of a group to related *Instance* and *Multiple Instances* sub-nodes.

To add a metric to a group, use the following Tcl command:

```
perfGroup -protocol protocol_name
          -inst/-cinst
          -group group_name
          -add metric_name
```

To remove a metric from a group, use the following tcl command:

```
perfGroup -protocol protocol_name
          -inst/-cinst
          -group group_name
          -remove metric_name
```

Enabling and Disabling Groups

You can enable/disable multiple metrics by enabling/disabling the group containing the specific metrics. You can use either of the following methods to enable/disable a group:

Using Tcl Commands

To enable a group, use the following command:

```
perfEnableMetric -protocol <protocol_name>
                 -inst/-cinst
                 -group <group_name>
                 -enable
```

Chapter 3: Using Verdi Performance Analyzer

Displaying Data Type and Unit for Metric Results Columns

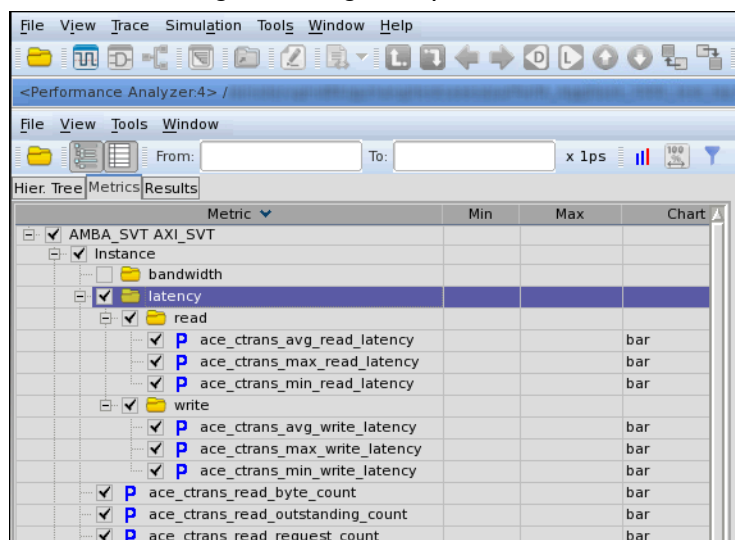
To disable a group, use the following command:

```
perfEnableMetric -protocol <protocol_name>
                 -inst/-cinst
                 -group <group_name>
                 -disable
```

Using the GUI Mode

The Verdi GUI mode supports enabling and disabling groups through the check-box displayed next to the group's folder in the *Metrics* tab of *Performance Analyzer*. To enable a group, select the check box next to the respective group folder and to disable a group, deselect the check box next to the group folder. This is illustrated in the following figure:

Figure 25 Enabling/Disabling Groups



If the group folder is empty, the enable operation cannot be performed.

Displaying Data Type and Unit for Metric Results Columns

The columns in metric results display data type and data unit.

The following types of data types are supported:

- **Time**

The *time* unit includes *fsdb* (time unit specified in the FSDB file), *fs*, *ps*, *ns*, *us*, *ms*, and *s*.

The default *time* unit is *fsdb*. The time unit is global for the *Performance Analyzer* view. All the columns of the *FSDB time* datatype share the unit in the same *Performance Analyzer* view.

- **Data**

The *data* unit includes *Byte*, *KB*, *MB*, *GB*, and *TB*. There is no default *data* unit and you must specify an original unit for data type. You can set the display unit for *data* type in the *Details* view.

- **Bandwidth**

The *bandwidth* data type is a compound type and its unit is `<data unit>/ <time unit>`. There is no default *bandwidth* unit and you must specify an original unit for the *bandwidth* type. You can set the display unit for the *bandwidth* type in the *Details* view.

- **User-Defined Type**

The user-defined data types do not support unit translation. You need to specify a unit string for user-defined types.

Setting the Original Data Type and Data Unit

You can set the original data type and data unit for a column before the performance report is generated by either using the `perfSetColumnType` Tcl command or using the GUI mode. This setting cannot be changed after the report generation.

Tcl Command

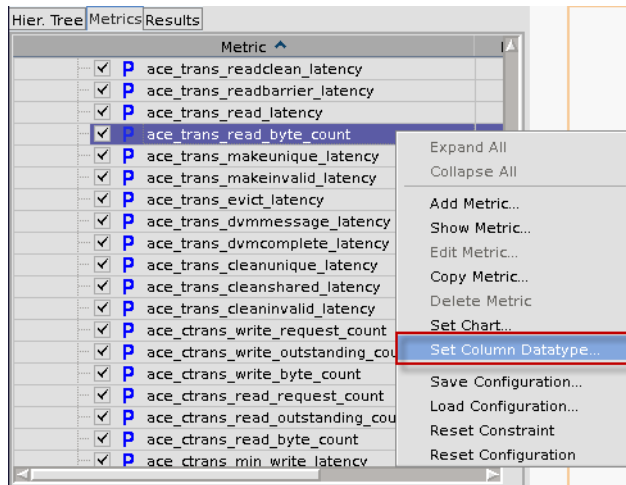
Use the following Tcl command to set the original data type and data unit for metric columns:

```
perfSetColumnType -protocol <protocol_name>
                  -name <metric_name>
                  -column <column_to_be_set>
                  -type <type_string> // column type
                  -unit <unit_string> // column unit
```

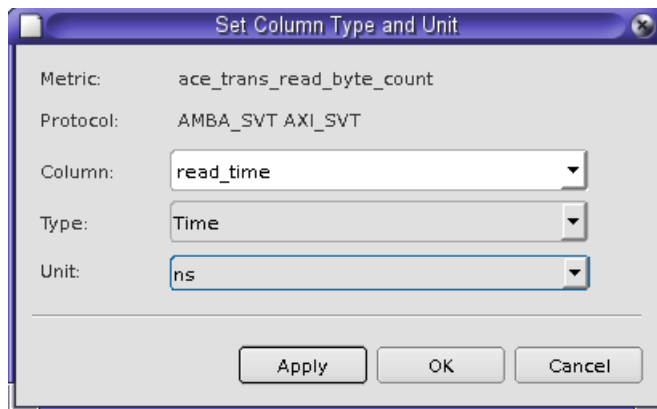
GUI Mode

To set the original data type and data unit for a column in the GUI mode, perform the following steps:

1. In the *Metrics* tab, select an instance and click the **Set Column Datatype** right-click option to open the *Set Column Type and Unit* dialog box.

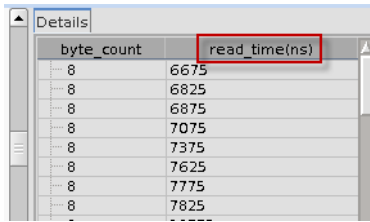


2. In this dialog box, select the column for which you want to specify data type and data unit in the *Column* field. Select Type and Unit in the respective fields and click **Apply**.



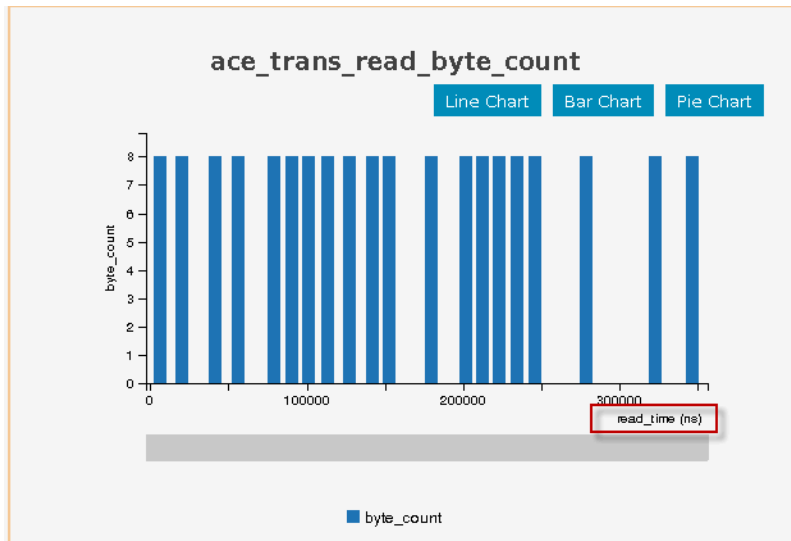
The specified data type and data unit for the column is displayed in the *Details* view and *Report* view after you regenerate the report.

In the *Details* view, the data unit is displayed in the header section of the list view, as illustrated in the following figure:



byte_count	read_time(ns)
8	6675
8	6825
8	6875
8	7075
8	7375
8	7625
8	7775
8	7825

In the *Reports* view, the data unit is displayed in the axis label as illustrated in the following figure:



Note:

Data type and unit settings are associated with a metric. In other words, if the data type is set for the column of a metric, that setting is applied to all instances of the same metric and column. If the original data type or unit is changed for a metric, the change takes effect when the next performance report is generated. Data type and unit configurations are saved during session saving and can be restored during session loading.

Setting the Display Unit

You can set the display unit for a performance report by either using the `perfColumnUnit` Tcl command or using the GUI mode. The display unit setting only changes the display unit of a column and not the datatype of the column. This setting is applied to the performance report immediately.

Tcl Command

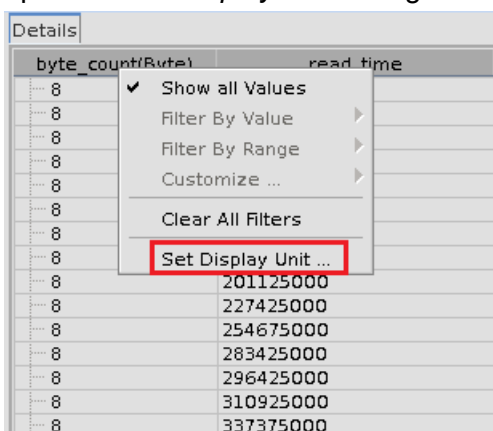
Use the following Tcl command to set the display unit for metric columns:

```
perfColumnUnit -protocol <protocol_name>
               -name <metric_name>
               -column <column_to_be_set>
               -type <type_string> // column type
               -unit <unit_string> // set the column display unit as the
given unit string
               -instance <instance_to_be_set> // optional, to apply the
setting to the specified instance, all instances are not specified
```

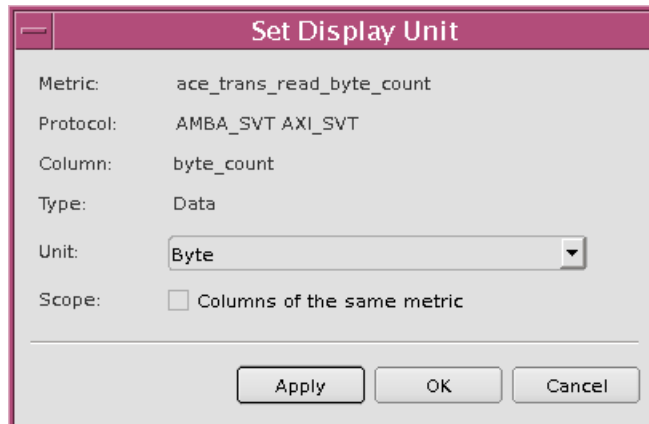
GUI Mode

To set the display unit for a column (*data* or *bandwidth* columns) in the GUI mode, perform the following steps:

1. Select a column in the *Details* view and click the right-click **Set Display Unit** option to open the *Set Display Unit* dialog box.



2. The *Set Display Unit* form opens. In this form, select the unit to be displayed for the column and click **Apply**.



The image shows a dialog box titled "Set Display Unit". It contains the following fields and controls:

- Metric: ace_trans_read_byte_count
- Protocol: AMBA_SVT AXI_SVT
- Column: byte_count
- Type: Data
- Unit: A dropdown menu showing "Byte".
- Scope: ☐ Columns of the same metric
- Buttons: Apply, OK, Cancel

If the display unit is changed for a column, the change takes effect immediately on the current performance report, however, the display unit setting is not retained when the report is generated again. Additionally, the filter on that column is removed.

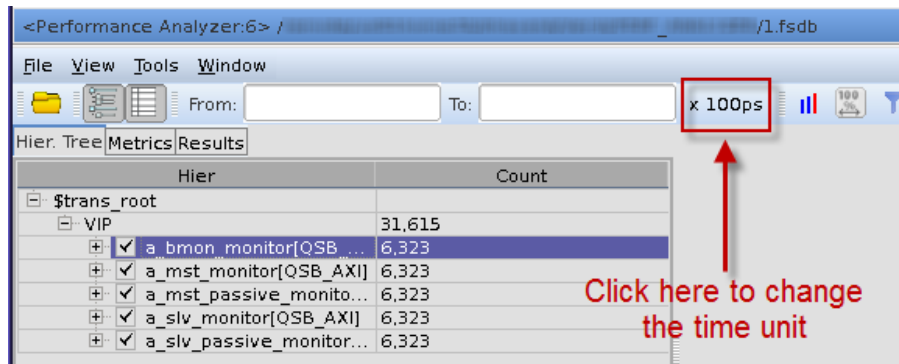
If the display unit is changed for the y axis column, the constraint value remains the same and is compared with the value in the original unit. In the chart, the constraint line is adjusted according to the new display unit.

Note:

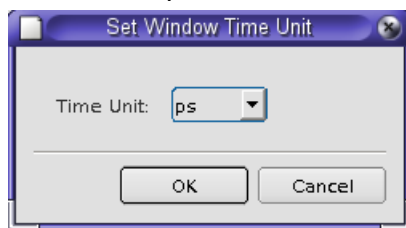
For both CSV and HTML format reports, the display unit setting is not used; the original type and unit is used in the saved report.

You can also change the *time* unit in the *Performance Analyzer* toolbar as follows:

1. Click the time unit appearing in the toolbar as illustrated in the following figure as illustrated in the following figure:



2. The *Set Window Time Unit* dialog box opens. In this dialog box, select the time unit from the drop-down list and click **OK**.



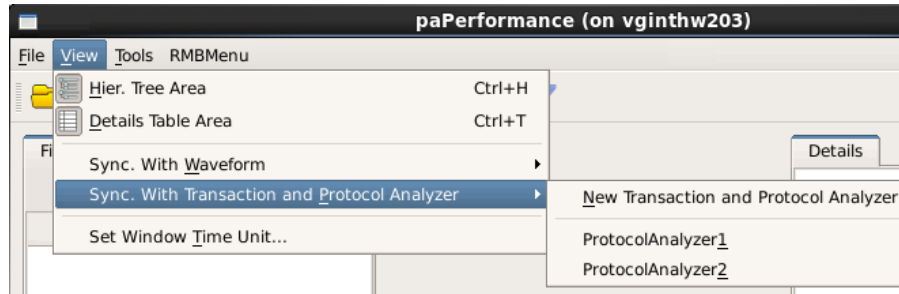
When the *time* unit is changed, all the columns having the *FSDB time* attribute are updated accordingly.

Synchronization Between Performance Analyzer and Transaction and Protocol Analyzer

You can use the **View --> Sync with Transaction and Protocol Analyzer** menu option to synchronize *Transaction and Protocol Analyzer* with *Performance Analyzer*. You can either sync with a new *Transaction and Protocol Analyzer* (TNPA) window or an already

opened TNPA window that is utilizing the same FSDB using the sub-menu options. This is illustrated in the following figure:

Figure 26 Synchronization With Transaction and Protocol Analyzer



To disable synchronization with a certain window, select the same window again in the sub-menu.

To synchronize with multiple windows, use the sub-menu by choosing another open window or by selecting the menu option to open a new window.

Selection Synchronization

You can use the  icon for selection synchronization across the windows; the **Sync with Transaction and Protocol Analyzer** menu option and the synchronization icon are always in synchronization.

Points to Note:

- If you use the **Sync with Transaction and Protocol Analyzer** menu option to synchronize with a particular *TNPA* window, the synchronization action automatically changes the state from  to  in both *Performance Analyzer* and *TNPA* windows that are selected for synchronization.
- If you click the  icon in the *TNPA* window, then synchronization gets disabled and the **Sync with Transaction and Protocol Analyzer** menu option state is updated accordingly inside *Performance Analyzer*.
- The  action allows you to synchronize one-to-many and many-to-one windows. You can still use this action to continue to synchronize with multiple *TNPA* windows. If you want to synchronize with other *TNPA* windows that are already opened (or newly opened), click the  icon that enables synchronization with the *Performance Analyzer* that is available.

- If you click the  icon from an already synchronized *Performance Analyzer* window, then all the *TNPA* windows that were synchronized earlier are disabled. If you click the  icon again, the synchronization for all the earlier synchronized windows is re-enabled.

Synchronization From Performance Analyzer to Transaction and Protocol Analyzer

After you enable synchronization (as explained above), if you select a row in the *Details* tab that is associated with a transaction, the following operations occur inside *TNPA*:

- The corresponding transaction stream is loaded inside *TNPA*, if it is already not loaded and the associated transaction is selected inside the *Transaction* pane.
- If the transaction is not in the *Transaction* pane, then the *Transaction* pane is adjusted to view the start time of the transaction inside *TNPA*. If the transaction was already in the *Transaction* pane, then there is no change.
- If the *Stream* pane is empty in *TNPA* before the transaction was selected, then the selected transaction is auto-zoomed. The zooming is the same as the *Zoom Selected* action inside *TNPA*.
- If you drag a row that is associated with a transaction from the *Performance Analyzer* and drop it to *TNPA*, the stream with which the transaction is associated is loaded inside the *TNPA Transaction* pane of Transaction and Protocol Analyzer if not already loaded and the transaction is selected inside *TNPA*. If the transaction was already loaded, then the associated transaction is selected inside *TNPA*. The viewable area is adjusted to show the transaction.
- If the drawing area of *TNPA* is empty when you drag a row that is associated with a transaction, then the loaded transaction is auto zoomed to view the associated transaction. The auto zooming is the same as the *Zoom Selected* action inside *TNPA*.

Synchronization From Transaction and Protocol Analyzer to Performance Analyzer

There is no menu option in *TNPA* to enable synchronization from *TNPA* to *Performance Analyzer*. You must either enable the synchronization through *Performance Analyzer* (as explained above) or use the  selection synchronization icon.

Once you enable synchronization from *Performance Analyzer* or through the selection synchronization icon, you can perform the following synchronization operations:

- If a transaction is selected in *TNPA*, the associated transaction row inside the *Performance Analyzer Details* tab is highlighted.
- If a row is not visible in the Performance Analyzer, the *Details* tab is scrolled to show the row in *Performance Analyzer*.
- If there are no matching associated rows for the transaction, then nothing is selected inside *Performance Analyzer*.
- Drag and drop from *TNPA* to *Performance Analyzer* is not supported.

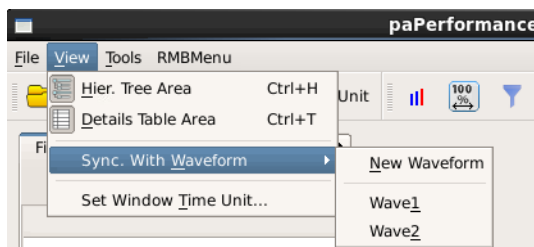
Note:

Multiple row selection synchronization is not supported from *Performance Analyzer* to *TNPA* and from *TNPA* to *Performance Analyzer*. There are certain metrics that allow multiple row selection inside the *Details* tab when the chart is selected; in that case, *TNPA* highlights all the matching transactions. If you select a particular transaction in *TNPA* or a particular row in *Performance Analyzer*, then all the highlighted transactions/rows are cleared from the respective window apart from the one selected.

Synchronization Between Performance Analyzer and nWave Windows

You can use the **View --> Sync with Waveform** menu option to synchronize *Performance Analyzer* with the *nWave* window. You can either synchronization with a new *nWave* window or an already opened *nWave* window. This is illustrated in the following figure:

Figure 27 Synchronization With Waveform



You can select multiple windows for synchronization. If you want to disable the synchronization, select the same window again that disables the synchronization for that window.

Once you enable synchronization (as explained above), markers are added inside the *nWave* window at the location that corresponds to the value of the time that is associated with the item selected inside the *Details* tab of *Performance Analyzer*.

Synchronization can only be performed for metrics whose results include time columns. Note that some metrics include only a single time in their results, while other metrics might contain two or more time columns.

- If there is only one time column, then only one marker is drawn inside the *nWave* window. For example, if the time of the selected row is 1000, then a marker is drawn at time 1000 inside the waveform viewer using a marker whose name is the same as the time column name.
- If the metrics contains two time columns, then two markers are drawn inside the waveform viewer with the two values matching those of the two time columns. For example, if the metric has two time columns (called `request_time` and `response_time`), and the value of `request_time` is 1000 and the value of `response_time` is 3500, then the `request_time` marker is drawn at time 1000 and the `response_time` marker is drawn at time 3500 inside the waveform viewer.

The *Details* tab does not allow multiple selections manually. Certain metrics that use the `rate` function contain values that are calculated using multiple events. For these values, all the events are highlighted when you select the value in the chart. In this case, the tool shows two markers inside the waveform viewer, which indicates the time of the first event as the *start prefix* column name and the *end prefix* column name as the last event in the list. The tool auto-detects the time column(s) to facilitate synchronization. If a metric does not contain any time columns, then there is no synchronization between *Performance Analyzer* and *nWave* windows for those metrics. The selection synchronization icon  does not have any impact on *nWave* and *Performance Analyzer* when enabled or disabled.

Note:

If there are more than two time columns, the values of the first two columns whose name ends with *time* are used to add markers in the *nWave* window.

Limitations

The following limitations apply to the synchronization functionality in *Performance Analyzer*:

- You cannot enable *Performance Analyzer* and *nWave* windows synchronization and *TNPA* and *nWave* window synchronization at the same time. If you have enabled any one of these synchronizations and attempt to synchronize the other window, then a dialog box appears asking if you want to switch between the two synchronization types.

- If you select the **Yes** option, the synchronization is switched from one to another.
- If you select the **No** option, then the current synchronization remains.

For example, if you have already enabled synchronization between *Performance Analyzer* and *nWave*, and then attempts to synchronize from *TNPA* to the same *nWave* window, the *Question* dialog box appears that shows the message - *Do you want switch synchronization from Performance Analyzer and Waveform to TNPA and Waveform?*. Based on your choice, the synchronization is enabled accordingly.

Cross-Instance Relationship

Some performance analysis needs to be measured from transactions across multiple VIP instances; for example, the analysis across Initiator (master) - Target (slave) pairs. This requires a way to associate the master transaction(s) with the corresponding slave transaction(s). As the parent-child relationship can be used to associate transactions within one VIP instance, the predecessor-successor relationship is used to associate transactions across different VIP instances. In addition, the master/slave cross-instance performance analysis metrics is built based on predecessor-successor relations. The predecessor-successor relation is added by the VIP.

It is assumed that for the same master-slave pair, there are three different scenarios of the predecessor-successor relations:

- One predecessor to one successor
- One predecessor to many successors
- Many predecessors to one successor

When the performance analysis metrics are written, the SQL query statement only references the predecessor or successor when it is single and unique.

In other words, for the following three scenarios,

- One Predecessor to one successor - The SQL statement can query either predecessor or successor.
- One predecessor to many successors - The SQL statement can only query the predecessor.
- Many predecessors to one successor - The SQL statement can only query the successor.

Note:

Many predecessors to many successors scenario is not supported.

When translating the FSDB file streams into SQL tables, Verdi *Performance Analyzer* provides the following two extra columns:

- The **pred_event** column - Records the single unique predecessor.
- The **succ_event** column - Records the single unique successor.

When a transaction has 0 or more than 1 predecessors, its **pred_event** entry is null. Whereas, when a transaction has 0 or more than 1 successors, its **succ_event** entry is null. This is illustrated in the following figure:

#	event	begin_time	end_time	parent_event	PRED_EVENT	SUCC_EVENT	label	ty
1	7	190	210	null	null	null	qsb_axi_transaction-Read	Tr
2	31	210	510	null	142	null	qsb_axi_transaction-Read	Tr
3	142	290	550	null	352	null	qsb_axi_transaction-Read	Tr
4	177	310	530	null	null	null	qsb_axi_transaction-Read	Tr
5	352	410	570	null	null	711	qsb_axi_transaction-Read	Tr
6	437	450	590	null	null	763	qsb_axi_transaction-Read	Tr
7	525	490	770	null	852	852	qsb_axi_transaction-Read	Tr

In a SQL query statement, **pred_event** and **succ_event** can be used to link two transactions.

If transaction **A** is the single unique predecessor of transaction **B**, **B.pred_event = A.event** are used.

If transaction **B** is the single unique successor of transaction **A**, **A.succ_event = B.event** is used.

Following is an example of a metric:

```
perfCreateMetric -protocol "AMBA_SVT AXI_SVT"
                 -name "cinst_concurrent_write_response_count"
                 -type "multi_instance"
                 -definition {SELECT COUNT(*) AS
concurrent_write_response_count
FROM $inst__Bus_Activity AS activity_resp,
     $inst__Transaction AS axi_transaction
WHERE axi_transaction.type_path =
      'Transaction.Write'
      AND activity_resp.type_path =
      'WRESP.WRESP'
      AND activity_resp.pred_event =
      axi_transaction.event }
                 -description "Multiple slaves are trying to send BRESP
to the same master in B channel."
```

Support for Instance-Based Constraints

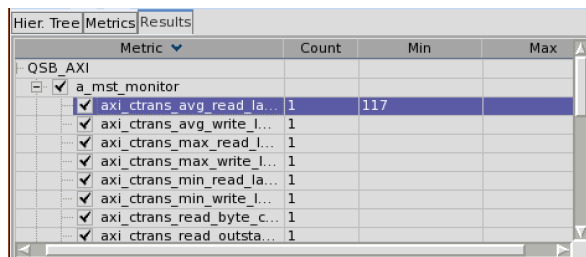
The Performance Analyzer allows you to set constraints for each performance metric on an instance-by-instance basis. You can define minimum and maximum constraints for the

instances of the selected metrics in **Min** and **Max** columns of *Results* tab. This allows you to set the constraints before the results are shown in the *Results* tab.

Note that,

- If you set the Min/Max constraint in the *Metrics* tab, it is applied to all the selected instances.
- If you set the Min/Max constraint in the *Results* tab for a single instance (or multiple instances) for a metric (or multiple metrics), the constraint settings in the *Metrics* tab are overridden by the value(s) specified in the *Results* tab.

Figure 28 The Results Tab



The screenshot shows the 'Results' tab in the Verdi Performance Analyzer. It features a tree view on the left and a table on the right. The tree view shows a hierarchy starting with 'QSB_AXI', followed by 'a_mst_monitor', and then a list of metrics. The table on the right has columns for 'Metric', 'Count', 'Min', and 'Max'. The 'axi_ctrans_avg_read_la...' metric is selected, and its value '117' is displayed in the 'Count' column.

Metric	Count	Min	Max
QSB_AXI			
a_mst_monitor			
axi_ctrans_avg_read_la...	1	117	
axi_ctrans_avg_write_l...	1		
axi_ctrans_max_read_l...	1		
axi_ctrans_max_write_l...	1		
axi_ctrans_min_read_la...	1		
axi_ctrans_min_write_l...	1		
axi_ctrans_read_byte_c...	1		
axi_ctrans_read_outsta...	1		

Support for Multiple Protocol Report Generation in Single Window

Performance Analyzer allows you to view the reports/results of multiple protocols in the same window to avoid opening separate windows for different protocols and view the reports separately. The instance definition shows the complete path of the instance from *env* to the instance name.

Use Model

Following are the sub-sections which explain the modifications performed for the report viewing:

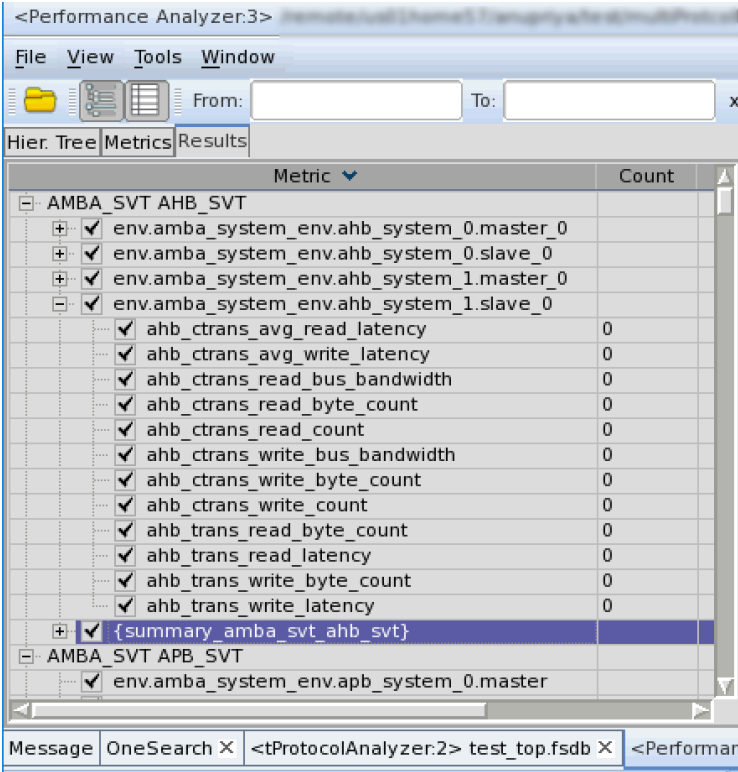
- [Result Tab Modification](#)
- [Report Pane Modification](#)
- [Support for Merge/Split of Metrics Data in a Single Graph](#)

Result Tab Modification

The *Results* tab is a tree view that displays performance analysis results.

The *Results* tab is updated to show the complete hierarchy path. **Summary** node is created for each complete path until the instance as shown in the following figure:

Figure 29 Results Tab



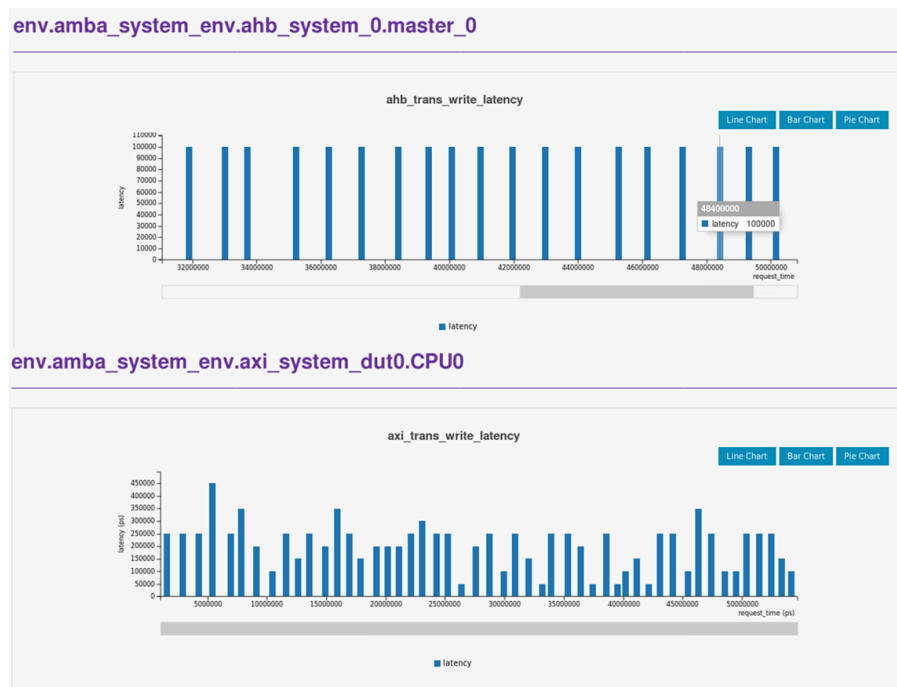
Report Pane Modification

The *Performance Report* pane displays the performance report of all protocols.

The *Report* pane is updated to show the complete hierarchy path.

In Report view, instance name is shown as a complete hierarchical path. Summary view name is updated for each hierarchy path as shown in the following figure.

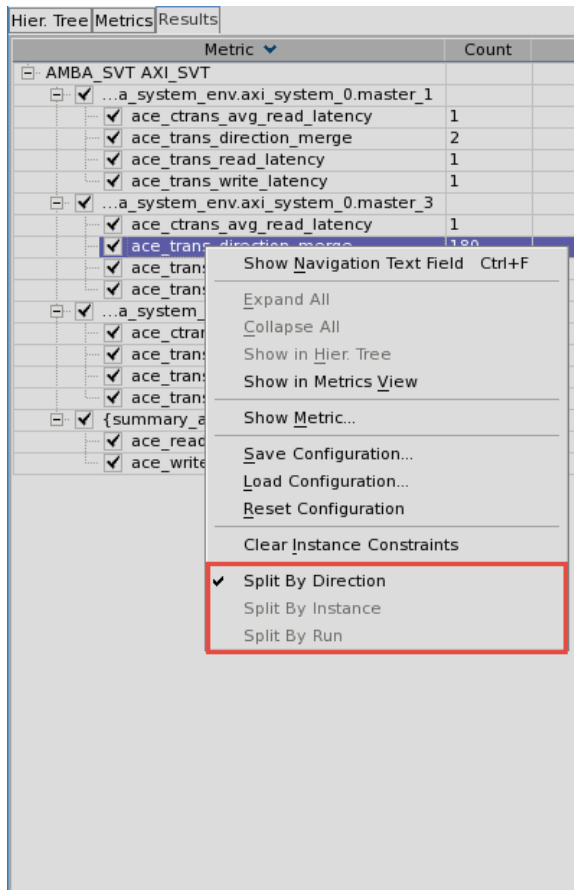
Figure 30 Report View



Support for Merge/Split of Metrics Data in a Single Graph

For metrics, you can include data from multiple FSDB files, multiple instances, or include both **READ** and **WRITE** data from single instance. You can split the result into multiple line/bars in different color in the same chart.

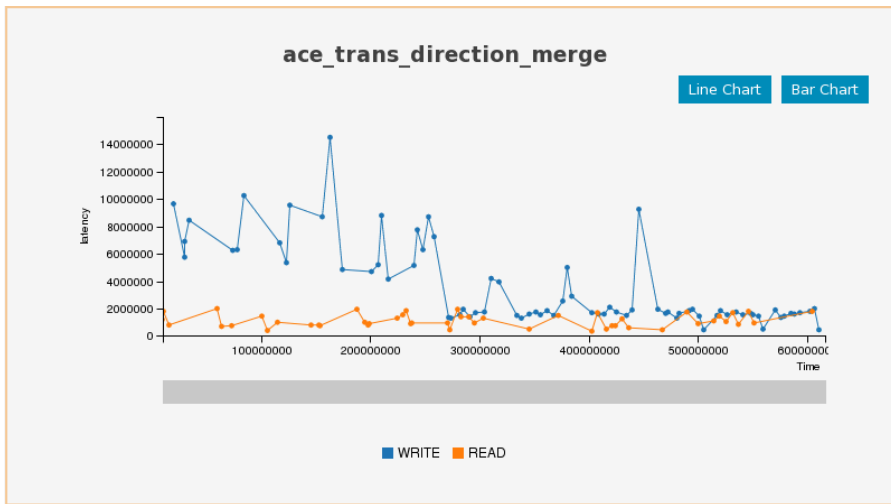
Figure 31 Split Options



Following are the different set of split reports, which are supported.

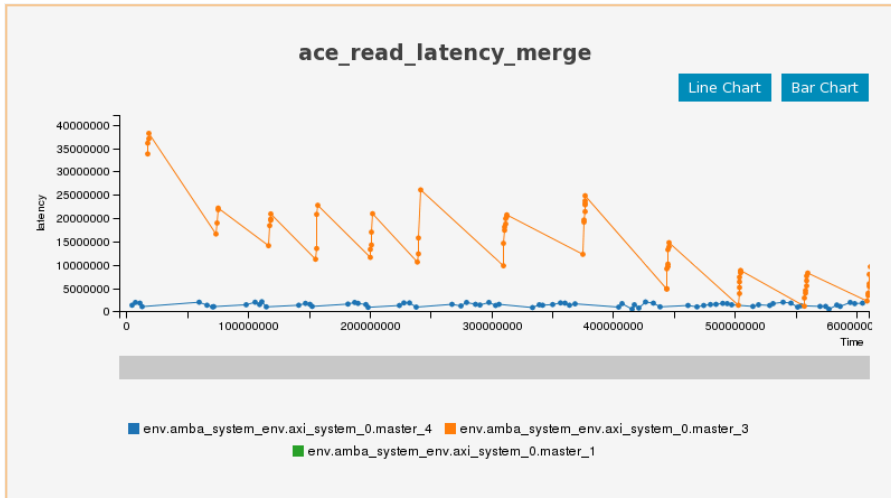
- **Split by Direction:** This option allows you to split **READ** and **WRITE** data in same chart by line or bar. This metric has the data for both **READ** and **WRITE** and it is available for all the instances. Right-click options are enabled for these metrics as shown in the above figure.

Figure 32 Split by Direction



- **Split by Instance:** This option allow you to split the metrics data by instance name. Currently cross instance metrics are added to support this feature. This is a customizable feature that allows you to add merged data for all the instance based metrics. These metrics are available under summary node.

Figure 33 Split by Instance

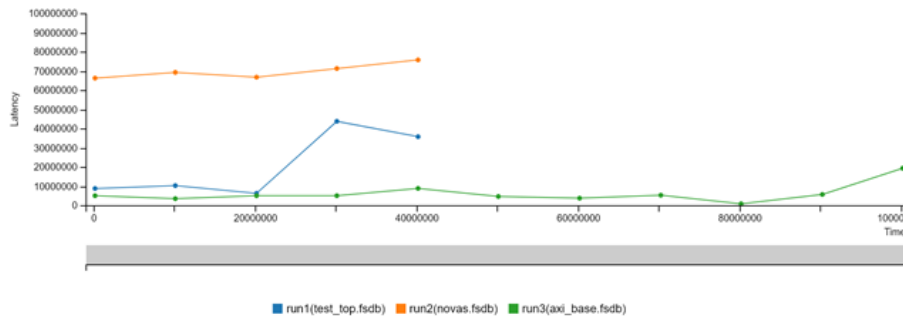


Note:

These metrics are split by default in the report and you can merge them back by using right-click option.

- **Split by Run:** This option allows you to split the metrics data by **FSDb** file name/path. You can configure the metrics that need to be displayed as split by run in the report. By default, the chart is in merged state. You can split the chart with the run action using the right-click options of result metric.

Figure 34 Split by Run



Note:

Split By Run option is applicable only for aggregate database.

Support for Summary Report

Verdi selects the summary related metrics and shows the reports/results in separate windows.

Use Model

In the *hierarchy tree*, select any one of the right-click options to view the report.

- **Show report in Stacked Window:** Displays the performance report in new window and moves the window down if there is availability of space in the window to arrange it down. If there is no space, then Verdi tabs to the current window.
- **Show Summary Report:** Displays the summary related metrics in the main window. You can also execute this action by clicking the icon provided in the toolbar. These summary related metrics are controlled by the Tcl command in the metric definition file.

Note:

Only the metrics which are defined for preload using the Tcl command is used for loading the results window. If there are no metrics with the

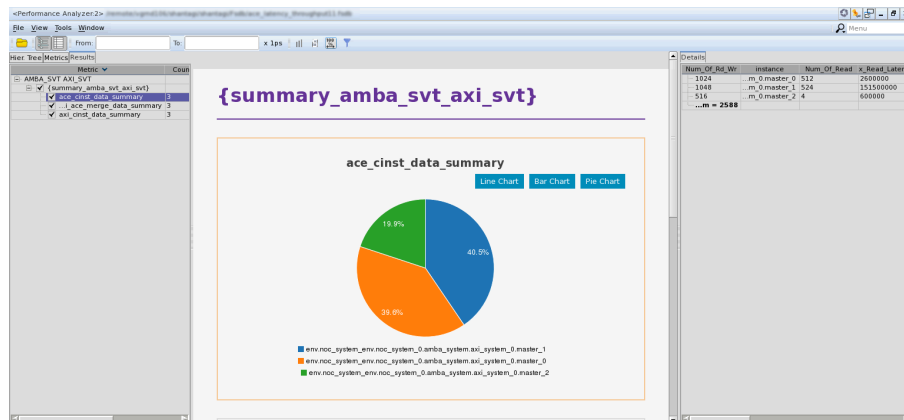
preloaded Tcl command, then Verdi issues a warning that there are no reports to load for summary results.

- **Show Summary Report in New Window:** Displays the performance report in new window. This action is similar to **Show Summary Report** except that it opens the report in the new window.

Show Summary Report in Stacked Window action is similar to **Show Summary Report in New Window** except that the window arranges in the bottom of the screen if there is a space.

Click the **Show Summary Report** icon on the tool bar to open the **Show Summary Report** in the main window. The **Show Summary Report** opens in the current window. If the full window is already opened then this window is overwritten with the new report. All the instances are selected from the root. The node which does not contain any transaction is not be allowed for selection. The *Performance Analyzer* only shows the VIP transaction streams/scopes for the selection.

Figure 35 Summary Report



New Tcl Commands (Summary Report)

Description

Shows the reports/results in separate windows.

Syntax

```
perfSetMetric -protocol {protocol_name} \
              -name "metric_name" \
              -preload
```

Arguments

`-protocol`

Specify the protocol name.

`-name`

Specify the metric name.

`-preload`

Switch which indicates the metric to be preload metric.

Value Returned

No value returned.

Example

As **Show Summary Report** is customized feature, you need to follow the following steps to enable this feature:

1. Add `perfSetMetric` Tcl command into the customized metric which is added in step2 custom path.

Example:

```
perfSetMetric -protocol {AMBA_SVT AXI_SVT}\  
-name axi_cinst_data_summary \  
-preload
```

2. Use the following command:

```
%> setenv PA_CUSTOM_VIP_PATH <customized_vip_path>
```

Support for Show/Hide Column and Column Ordering in Detail Tree

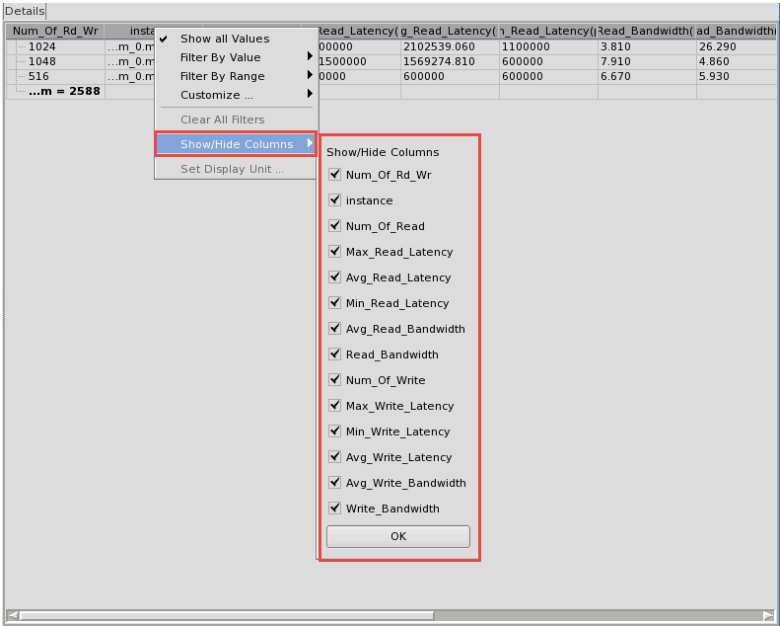
Detail tree displays all the column and their values for selected metric in tabular format. Some of the metric displays many columns.

Use Model

To show or hide the columns displayed, right-click the header and select **Show/Hide Columns** option. It allows you to toggle the corresponding check boxes and also selects multiple types at the same time which can be selected as per your preference. By default, all the check boxes are selected. Uncheck the check boxes which you want to

hide and select **OK** as shown in the following figure. These changes are saved and the configuration is restored for detail table header configuration of metric. Exporting the report in csv format retains the hidden column for each metric.

Figure 36 Show/Hide Columns



Swapping the Column is also supported in detail tree. Click the header of the column and hold the left mouse click and drag the column to the required index. Exporting the report in csv format retains the column swapped order for each metric. The following figure shows that **Num_Of_Rd_Wr** column is swapped with **Read_Bandwidth(MB/s)** column.

Figure 37 Swapping the Columns

Details					
Num_Of_Rd_Wr	instance	Num_Of_Read	Read_Bandwidth(MB/s)	Avg_Read_Bandwidth(MB/s)	
2	...xi_system_0.master_1	1	5.82	2	5.82
180	...xi_system_0.master_3	180	19.06	180	6.92
450	...xi_system_0.master_4	180	19.02	450	73.92
Sum = 632					

Details					
Read_Bandwidth(MB/s)	Num_Of_Read	instance	Num_Of_Rd_Wr	Avg_Read_Bandwidth(MB/s)	
5.82	1	...xi_system_0.master_1	2	2	5.82
19.06	180	...xi_system_0.master_3	180	180	6.92
19.02	180	...xi_system_0.master_4	450	450	73.92
			Sum = 632		

New Tcl Commands (column ordering)

Description

Swaps or changes the default order of the detail tree.

Syntax

```
perfColumnFiltering -protocol "protocol_name" -name \  
"metric_name" -instance \  
"instance_name" -order "column1" "column2" ...
```

Arguments

-protocol

Specify the protocol name.

-name

Specify the metric name.

-instance

Specify the instance name.

-order

Specify the new order in the detail tree.

Value Returned

No value returned.

Example

```
perfConfigDetails -protocol "AMBA_SVT AXI_SVT" -name  
axi_trans_read_latency -instance "env.axi_system_env.master_0" \  
-order "request_time" "latency"
```

Note:

Columns added after `-order` switch is displayed in the report as the same order given in Tcl command.

New Tcl Commands (show and hide columns)

Description

Hides and shows the columns in detail tree.

Syntax

For hiding columns, the syntax is as follows:

```
perfColumnFiltering -protocol "protocol_name" -name \  
"metric_name" -instance \  
"instance_name" -hide "columns_names"
```

For showing the hidden columns, the syntax is as follows:

```
perfColumnFiltering -protocol "protocol_name" -name \  
"metric_name" -instance \  
"instance_name" -showAll
```

Arguments

-protocol

Specify the protocol name.

-name

Specify the metric name.

-instance

Specify the instance name.

-hide

Specify the column that needs to be hidden in the detail tree.

-showAll

Specify to get back the hidden column.

Value Returned

No value returned.

Example

For hiding columns:

```
perfConfigDetails -protocol "AMBA_SVT AXI_SVT" -name  
axi_trans_read_latency -instance "env.axi_system_env.master_0" \  
-hide "request_time" "latency"
```

For showing back the hidden columns:

```
perfConfigDetails -protocol "AMBA_SVT AXI_SVT" -name  
axi_trans_read_latency -instance "env.axi_system_env.master_0" \  
-showAll
```

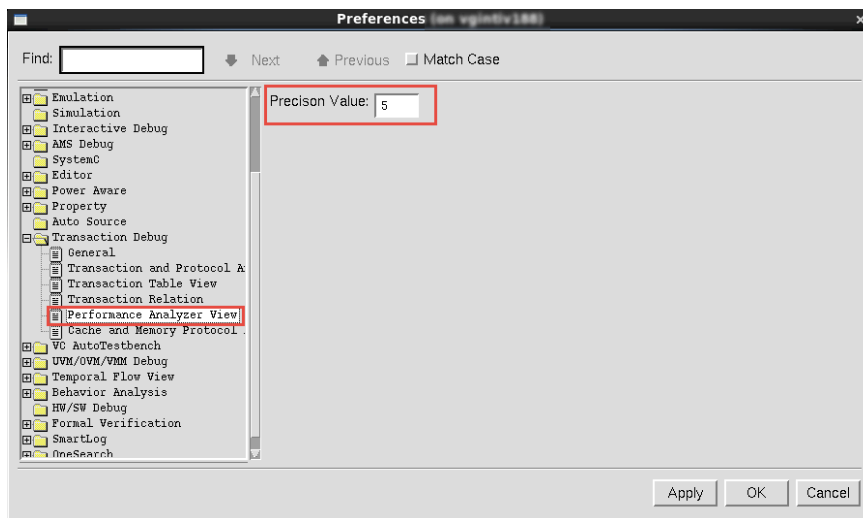
Note:

Columns added after `-hide` switch is hidden from the original list in **Report -> Detail tree** option.

Support for Precision for Double Values

You can select the precision value from *Preferences* form. Default precision is two for double values. To choose the precision, select **Tools -> Preferences** folder. The **Precision Value** option is added in the *Preferences* form as shown in the following figure:

Figure 38 Precision Value



Preference report is shown in the following figure.

Figure 39 Preference Report

A screenshot of a preference report snippet. It shows a label 'ace_ctrans_avg_read_bandwidth:' followed by a value '3.81000 MB/s' which is highlighted in orange. The entire snippet is enclosed in an orange rectangular border.

New Tcl Commands

Description

Tcl command for preference support.

Syntax

```
perPreferences -doublePrecision "value"
```

Value range is 1 to 6.

Arguments

`-doublePrecision`

Specifies the double precision value.

Value Returned

No value returned.

Example

```
perPreferences -doublePrecision "5"
```

Support for Column Value Sum in the Details Tab

Details tab supports sum of integer values for each column. The sum of integer values are displayed in the last row after using the Tcl command for one column at a time as shown in the following figure. It can be accessed only through the loaded metric file. The sum value is displayed in bold.

Figure 40 Column Value Sum

Details					
Num_Of_Rd_Wr	instance	Num_Of_Read	Read_Bandwidth(MB/s)	Avg_Read_Bandwidth(MB/s)	
2	...xi_system_0.master_1	1	5.82	5.82	
180	...xi_system_0.master_3	180	19.06	6.92	
450	...xi_system_0.master_4	180	19.02	73.92	
Sum = 632					

New Tcl Commands

Description

New Tcl command for column value sum.

Syntax

```
perfSetColumnFuncType -protocol {protocol_name} \  
-name "metric_name" \  
-column "column_name" \  
-func "function_type"
```

Arguments

`-protocol`

Specify the protocol name.

`-name`

Specify the metric name.

`-column`

Specify the column name.

`-func`

Specify the function type to sum.

Value Returned

No value returned.

Example

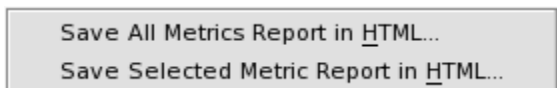
```
perfSetColumnFuncType -protocol {AMBA_SVT AXI_SVT} \
-name ace_cinst_data_summary \
-column Num_Of_Rd_Wr \
-func sum
```

Support for Exporting of csv Report and HTML Report Generation for Selected Metric

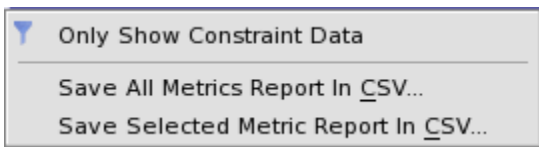
Verdi supports exporting the performance metrics data to `csv` and `html` format for all the evaluated metrics. You can choose the metric to export the data and also the format in which the data is to be exported.

You can either right-click the chart or details table to export the data for the current selected metric.

On the chart, select the option to export either all the metrics or the selected metric to HTML format as shown in the following figure.



On details table, select the option to export all the metrics or current metric to csv format as shown in the following figure.



New Tcl Commands

Description

New Tcl command for exporting the performance metric report.

Syntax

```
perReportView -instance "instance_name" \  
              -name "metric_name" \  
              -saveSingleHTML \  
              -file file_name
```

Arguments

-instance

Specify the instance name.

-name

Specify the metric name.

-file

Specify the file name to be saved.

-saveSingleHTML

Switch which indicates to save single metric report.

Value Returned

No value returned.

Example

```
perReportView -win $_perWnd4 -instance \  
              "env.noc_system_env.noc_system_0.amba_system.axi_system_0.master_0" \  
              -name "ace_ctrans_avg_read_bandwidth" -saveSingleHTML -file singleHtml
```

Improving Debug Messages for SQL Flow

It is difficult to debug the SQL flow in the tool. To solve this problem, tool debug messaging has been enhanced for all the SQL/NO SQL errors written out in a separate log file and this log file is stored with other Verdi logs.

The errors are captured and categorized with the unique code.

The following is the log path:

```
./verdiLog/verdi_perf_err.log
```

The following are the error examples:

- [SQL_ATTR_NOT_FOUND_ERROR] -> This error is flagged in the log file under this category if any attribute is missing in SQL schema. This error can be resolved by hitting "Resolved Attribute action".

For example, [SQL_ATTR_NOT_FOUND_ERROR] has no such column:

```
ace_transaction.total_byte_count in SQL
```

- [SQL_GEN_ERROR] -> Any generic SQL error is flagged in the log file under this category. This error can be resolved as per the description given for the error.

For example, [SQL_GEN_ERROR] has no such table:

```
env.axi_passive_system_env.master_0_ace_trans_read_latency in SQL
```

This log contains all the errors occurred during the analysis.

Resolving Missing Attributes

Following is the use model for this feature:

Use Model

The red error icon, as shown in the following figure, is added to the metric node if the error is of type No Attribute Found (NAF). You can right click on metric to get the RMB menu and click on the action **Resolve missing Attribute Errors**. After regenerating the report errors are resolved.

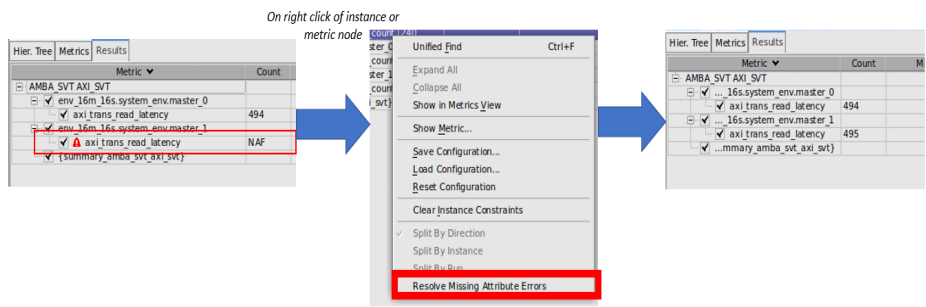
If the error is not resolvable then those attributes are the dumped part of `./verdiLog/verdi_perf_err.log` under macro (PERF_FSDB_MISSING_ATTR_LIST) file for user reference.

Note:

Invalid attributes are not resolvable.

You need to create a `json` file for all the resolved attributes. `Json` file can be copied to the path `$PA_CUSTOM_VIP_PATH/performance/metrics/`. This path is used for future reference.

Chapter 3: Using Verdi Performance Analyzer
Resolving Missing Attributes



4

Batch Mode Operation

This chapter discusses about the batch mode support available for *Verdi Performance Analyzer* in the following sections:

- [Batch Mode Use Model](#)
- [Batch Mode Tool](#)
- [Batch Mode Commands](#)

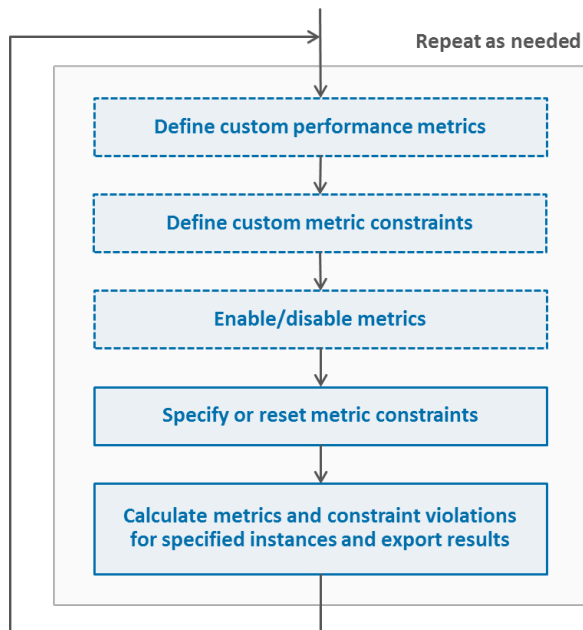
Batch Mode Use Model

The *Performance Analyzer* application in Verdi offers the functionality to calculate performance metrics as a batch operation. This allows you to include performance measurement and validation in a regression (or a similar automated validation) environment, calculate performance metrics, and check those metrics against desired minimum or maximum constraint values.

Considering the highest level, the batch mode use model comprises of the following three steps:

1. Write the command files [optional].
2. Execute the *verdi_perf* command to process the command files and generate one or more performance reports.
3. Review the reports in a web browser or in Microsoft Excel.

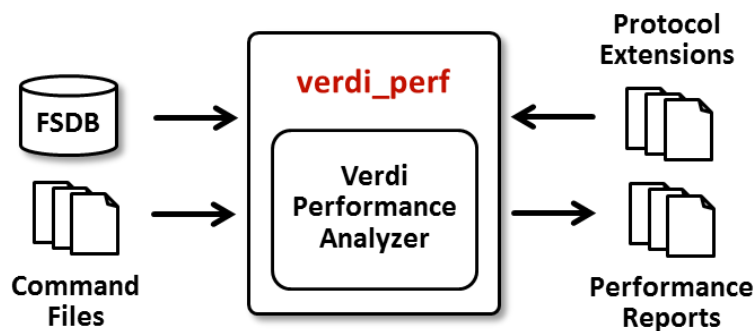
The structure of the command file enables the evaluation of multiple performance metrics for different sets of instances with only a single invocation of the Performance Analyzer. This can increase the overall efficiency of the flow, as illustrated in the following figure:



Batch Mode Tool

Performance Analyzer in Verdi comprises of an in-built wrapper that encapsulates the required capabilities into an easy-to-use batch mode tool. The basic environment in which this tool can be used is illustrated in the following figure:

Figure 41 Batch Mode Tool Environment



The wrapper has three types of input data and one type of output data.

It can be invoked using the following command:

```

verdi_perf -fsdb/-f
            -instance/-i
            [ -start/-s <start_time> ]
            [ -end/-s <end_time> ]
    
```

```
[ -cmd/-c <list_of_command_files> ]  
[ -output/-o <output_file_name> ]  
[ -format/-m <output_file_format> ]  
[ -session/-n <session_saving_option> ]
```

Parameters

`-fsdb/-f`

The `-fsdb` or the `-f` option specifies the FSDB file to be processed. The FSDB file is the output of a simulation that contains all the protocol-related simulation data that is required to perform the desired performance metric calculations. This means that FSDB dumping must have been enabled for all VIP instances of interest. The FSDB file needs to be specified either through the `-fsdb` option or in the command file specified by the `-cmd` option.

`-instance/-i`

The `-instance` or the `-i` option specifies the instances on which the performance evaluation is done. The instance needs to be specified either through the `-instance` option or in the command file specified by the `-cmd` option.

`-start/-s`

The `-start` or the `-s` option specifies the start time for performance evaluation.

`-end/-e`

The `-end` or the `-e` option specifies the end time for performance evaluation.

`-cmd/-c`

The `-cmd` or the `-c` option specifies a list of one or more command files to be processed. These command files can contain Tcl commands to define custom performance metrics, enable or disable performance metrics, set or clear constraint values, perform metric calculations, and specify the location and type of results to produce.

`-output/-o`

The `-output` or the `-o` option specifies the name of the output file.

`-format/-m`

The `-format` or the `-m` option specifies the format of the output file: *html* for HTML format, and *csv* for CSV format.

`-session/-n`

The `-session` or the `-n` option specifies the condition/option to save the current session. The saved session can later be opened in the *Performance Analyzer* GUI.

The various options to save a session are as follows:

- `on` - Save the session always.
- `off` - Never save the session.
- `fail` - Save the session if there is any constraint violation.

The default option to save a session is `fail`.

Protocol Extensions

The Protocol Extensions defined for the Verdi *Transaction and Protocol Analyzer* also apply to the Verdi *Performance Analyzer* and are present at the same location as pre-defined performance metrics. In addition to the Synopsys VIPs that are located in the `DESIGNWARE_HOME` directory, any directories that are specified using the `PA_CUSTOM_VIP_PATH` environment variable are also analyzed for protocol definitions and any associated pre-defined performance metric definitions.

Batch Mode Processing Flow

The following two flows exist for batch mode processing:

1. Write the operations in a TCL command file, and specify the TCL command through the `-cmd/-c` option.
2. Use other command line options to specify the FSDB file, instances, and so on.

Note:

The `-cmd/-c` command line option is mutually exclusive with other command line options. You need to choose to use either the command file or the command line options, but not both.

If no command files are specified, the Verdi *Performance Analyzer* applies the pre-defined performance metrics to the VIP instances found in the design and produces a report using the default output format, which is CSV.

If the command file is specified, the command file can contain one or more of the TCL commands mentioned in [Batch Mode Commands](#) section.

Batch Mode Commands

The following commands can be used in a *Performance Analyzer* batch-mode command file:

perfCreateMetric

This command is used to define a custom performance metric.

Syntax

```
perfCreateMetric
  -protocol <protocol_name>
  -name <metric_name>
  [-description <metric_description> ]
  -type <metric_type>
  -definition <metric_sql_expression>
  [-dependent { <dependent_metric_names> } ]
```

perfEnableMetric

This command is used to enable or disable performance metrics.

Syntax

```
perfEnableMetric
  -protocol <protocol_name>
  -name <metric_name>
  -disable <add to disable a metric, otherwise to enable a metric>
```

perfDeleteMetric

This command is used to delete a custom performance metric.

Syntax

```
perfDeleteMetric
  -protocol <protocol_name>
  -name <metric_name>
```

perfSetConstraint

This command is used to define a custom metric constraint.

Syntax

```
perfSetConstraint
  -protocol <protocol_name>
  -name <metric_name>
  -instance <instance_name>
  [ -min <minimal_constraint_value> ]
  [ -max <maximal_constraint_value> ]
  [ -minRemove ] <to remove the minimal constraint>
  [ -maxRemove ] <to remove the maximal constraint>
```

Note:

If the `-instance` option is not specified, then the constraint is applied on all the instances.

perfSetChart

This command is used to set the chart option for metric.

Syntax

```
perfSetChart
  -protocol <protocol_name>
  -name <metric_name>
  -type <specify the chart type, bar, line, or pie>
  -frequency <add to do frequency statistics >
```

perfSaveReport

This command is used to save performance report into files.

Syntax

```
perfSaveReport
  -file <name of the output file>
  -format <format of the output file, csv or html>
```

perfSetTimeRange

This command is used to set the time range for performance evaluation.

Syntax

```
perfSetTimeRange
  -begin <start of the time range>
  -end <end of the time range>
```

perfSetTimeUnit

This command is used to set the time unit for the analysis.

Syntax

```
perfSetTimeUnit  
-timeunit <the unit for time range, e.g. ns, ms, us etc...>
```

Syntax With Multiplier

```
perfSetTimeUnit  
-timeunit <the unit for time range, e.g. ns, ms, us etc...> -Multiplier  
<"1", "10", "100">
```

perfSetFsdb

This command is used to set the FSDB file.

Syntax

```
perfSetFsdb  
-file <The fsd file. >
```

perfSaveConfig

This command is used to save the metric configuration into the Tcl file.

Syntax

```
perfSaveConfig  
-file <The config tcl file. >
```

perfLoadConfig

This command is used to load the metric configuration from the Tcl file.

Syntax

```
perfLoadConfig  
-file <The config tcl file.>
```

perfResetConfig

This command is used to reset all custom configurations.

Syntax

```
perfResetConfig
```

perfResetConstraint

This command is used to reset all the constraints.

Syntax

```
perfResetConstraint
```

perfEvaluate

This command is used to perform evaluation of the enabled metrics on the specified instances.

Syntax

```
perfEvaluate  
-instance { <list_of_instances> }
```

Note:

The performance result is generated when the `perfEvaluate` Tcl command is executed. You need to specify the `perfSaveReport` command after the `perfEvaluate` command; all other Tcl commands are used for evaluation configuration and need to be specified before the `perfEvaluate` command.

perfGroup

This command is used to add or define groups for metrics.

Syntax

```
perfGroup -protocol <protocol_name>  
-inst/-cinst  
-create <group_name>  
-delete <group_name>  
-group <group_name>  
-add <metric_name>  
-enable  
-disable
```

Note:

For details on this command, see [Defining Groups for Metrics](#).

perfConfigDetails

This command is used to show and hide the columns in detail tree.

Syntax

```
perfConfigDetails  -protocol "protocol_name"
                   -name "metric_name"
                   -instance "instance_name"
                   -hide "col_name_1" "col_name_2" "col_name_3"
```

Example

```
perfConfigDetails  -protocol "AMBA_SVT AXI_SVT"
                   -name ace_cinst_data_summary
                   -instance "{summary_amba_svt_axi_svt}"
                   -hide "Avg_Read_Bandwidth" "Avg_Write_Bandwidth"
```

perfConfigDetails

This command is used to swap columns in detail tree.

Syntax

```
perfConfigDetails  -protocol "protocol_name"
                   -name "metric_name"
                   -instance "instance_name"
                   -order  "col_name_1" "col_name_4" "col_name_3" "col_name_2"
```

Example

```
perfConfigDetails  -protocol "AMBA_SVT AXI_SVT"
                   -name ace_cinst_data_summary
                   -instance "{summary_amba_svt_axi_svt}"
                   -order "instance" "Read_Bandwidth" "Write_Bandwidth"
                        "Avg_Read_Latency" "Avg_Write_Latency"
                        "Max_Read_Latency" "Max_Write_Latency"
                        "Number_of_Read" "Number_of_Write" "Number_of_Rd_Wr"
```

5

Aggregate Performance Analysis

Verdi *Performance Analyzer* can also generate an aggregate performance analysis report for all the FSDB files generated from multiple simulation runs for the same design. For multiple FSDB files, the performance analysis report includes results for the merged data from all the FSDB files and results for data from every single FSDB file. Therefore, you can get a summary of performance results for all runs and also the statistical result distribution/difference among every FSDB file.

This chapter explains about how to generate and view an aggregate performance analyzer report in the following sections:

- [Prerequisites](#)
- [Generating Aggregate Performance Analysis Report](#)
- [verdi_perf Utility](#)
- [Viewing Aggregate Performance Analysis Report in GUI](#)
- [Examples to Demonstrate Merged Results for Multiple FSDB Files](#)

Prerequisites

The following are the prerequisites for aggregate performance analysis:

- Each FSDB file generated by a regression or by multiple runs must have a unique identifier. This can be used by Verdi to organize, indicate, add or remove the test results associated with a specific test. The FSDB full path (generated from the tests) is used as the unique identifier.
- The FSDB files generated from different tests must have similar VIP instantiation hierarchies which merges the performance analysis results. Some FSDB files may contain more instance hierarchies, while some may have fewer instance hierarchies. However, only the results from the same VIP instantiation hierarchy path are merged.

Generating Aggregate Performance Analysis Report

To generate an aggregate performance analysis report, perform the following steps:

1. Generate the result database (in Batch Mode)

Execute the `verdi_perf` command to initialize a result database (that is, a directory) to save the performance results, and configure the metrics and constraints for the performance analysis on multiple results.

Note:

Performance analysis on multiple test results share the same metrics and constraints configuration.

2. Add or remove FSDB files to/from the database (in Batch Mode)

Run simulations for multiple tests, with Protocol Analyzer dumping enabled to dump the FSDB file with protocol data. After each test, execute the `verdi_perf` command to run performance analysis on the generated FSDB file.

The result is saved in the result database directory.

If there are constraints violations, the `verdi_perf` command issues error messages.

The FSDB file can be deleted after the file has been added to the result database; aggregate results will not be affected.

In many regression systems, multiple tests are run at the same time. The `verdi_perf` command can also run in parallel to analyze the dynamically generated FSDB files and write the results to the result database.

You can also execute the `verdi_perf` command to override or remove an existing analysis result from the result database.

3. Generate and view the aggregate performance analysis report (in Batch Mode)

- a. Execute the `verdi_perf` command with `-db/-d`, `-format/m`, and `-output/-o` options to generate HTML or CSV format analysis results.
- b. View the report in a web browser (HTML format) or in an excel sheet (CSV format).

Note:

You can also view the aggregate performance report in Verdi GUI mode.
For details, see [Viewing Aggregate Performance Analysis Report in GUI](#).

Example

Consider the following `axi.cfg` configuration file:

```
perfSetTimeRange -begin 1024 -end 2048

perfCreateMetric -protocol "amba_svt axi_svt"
                 -name     "inst_avg_trans_write_latency"
                 -desc     "Computes the average latency
                           for all write transactions"
                 -type     "instance"
                 -definition "SELECT avg( write_latency ) AS
                           avg_write_latency,
                           request_time,
                           response_time,
                           request_id,
                           response_id,
                           trans_id
                           FROM inst_trans_write_latency"
perfSetConstraint -dependent "inst_trans_write_latency"
                 -protocol  "amba_svt axi_svt"
                 -name      "inst_trans_write_latency"
                 -max       2048
perfEvaluate      -instance  \${trans_root}/VIP/env/
                           axi_system_env/master_0 /
                           \${trans_root}/VIP/env/
                           axi_system_env/master_1
```

The following is an example to generate an aggregate performance analysis report using a configuration file:

1. In the first phase, an empty database directory needs to be created with the specified protocol, instances, and configurations. This is an example to generate multiple FSDB file result database for the `svt_axi` protocol with the preceding configuration file as follows:

```
verdi_perf -protocol "amba_svt axi_svt" -cmd axi.cfg -db axi.dbdir
```

2. In the second phase, with the created empty result database directory (`axi.dbdir`), execute the `verdi_perf` command multiple times (in parallel, if desired) to analyze the performance of the FSDB files (generated from each test) into the result database.

```
verdi_perf -add axi01.fsdb -db axi.dbdir
verdi_perf -add axi02.fsdb -db axi.dbdir
...
```

You can remove an FSDB file from the database as follows:

```
verdi_perf -remove axi02.fsdb -db axi.dbdir
```

3. In the third phase, with the created result database directory, and FSDB files added into the database, use the `verdi_perf` command to generate an HTML or a CSV format report.

```
verdi_perf -db axi.dbdir -format html -output axi_html
verdi_perf -db axi.dbdir -format csv -output axi.csv
```

verdi_perf Utility

Verdi Performance Analyzer comprises of an in-built wrapper which encapsulates the required capabilities into an easy-to-use batch mode tool. It can be invoked using the `verdi_perf` utility to generate an aggregate performance analysis report.

Following is the list of valid options for generating an aggregate performance report:

```
verdi_perf -instance/-i
[ -protocol/-p <protocol> ]
[ -cmd/-c <config_file_name> ]
[ -db/-d <database_name> ]
[ -add/-a <fsdb_name> ]
[ -remove/-r <fsdb_name> ]
[ -force/-w <fsdb_name> ]
[ -output/-o <output_file_name> ]
[ -format/-m <output_file_format> ]
```

`-instance/-i`

The `-instance` or the `-i` option specifies the instances and scopes on which the performance evaluation is done. The instance needs to be specified either through the `-instance` option or in the command file specified by the `-cmd` option. The value of an instance is a list of nodes where a node could be an instance or a scope.

`-protocol/-p <protocol>`

The `-protocol` or the `-p` option specifies the protocol to be processed for performance evaluation. All the instances for the specified protocol are evaluated. This option is required for database initialization.

`-cmd/-c`

The `-cmd` or the `-c` option specifies the metrics/constraints configuration file. The configuration file syntax and format is the same as the one used for single FSDB file processing. For example, the Tcl command for adding or changing metrics or setting constraints could be added into the configuration file.

`-db/-d <database_name>`

The `-db` or the `-d` option specifies the name of the result database directory. This option is mandatory for generating an aggregate performance report. If this option is not added, Performance Analyzer enters the single FSDB mode.

`-info`

The `-info` option shows information about the specified aggregate database. The information includes a global environment, which contains the FSDB file list, protocol, and instance names. If the number of FSDB files is more than sixteen, only the first sixteen file paths are displayed in the output. The output format is as follows:

```
General information for dbdir: <dbdir>
Protocol: "<protocol>"
Instance: <instance>
Number of instances: n
<instance1>
<instance2>
Number of fsdb files: n
<file1>
<file2>
```

`-add/-a <fsdb_name>`

The `-add` or the `-a` option adds the specified FSDB file into the database.

`-remove/-r <fsdb_name>`

The `-remove` or the `-r` option removes the specified FSDB file from the database.

Note:

If neither `-add` nor `-remove` option is added, and either `-instance` or `-cmd` option is specified, the `verdi_perf` command performs database initialization. The `-add` and the `-remove` options can not be added at the same time in the command.

`-force/-w <fsdb_name>`

The `-force` or the `-w` option overwrites the FSDB file with the same name in the database.

`-output/-o`

The `-output` or the `-o` option specifies the name of the output file. This is an optional parameter.

`-format/-m`

The `-format` or the `-m` option specifies the format of the output file: *html* for HTML format, and *csv* for CSV format. This is an optional parameter.

Note:

If neither `-add` nor `-remove` is added, and neither `-instance` nor `-cmd` option is specified, the `verdi_perf` command generates the report for the specified database. If the `-add` option is added, the `verdi_perf` command adds the FSDB file into the database.

You can use the following Tcl command to view information about the aggregate database in batch mode:

```
verdi_perf -db <dbdir>
           -info
```

The output format is as follows:

```
General information for dbdir: <dbdir>
Protocol: "<protocol>"
Instance: <instance>
Number of instances: n
<instance1>
<instance2>
Number of fsdb files: n
<file1>
<file2>
```

Viewing Aggregate Performance Analysis Report in GUI

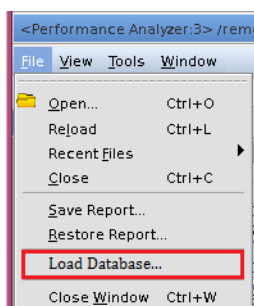
The results from multiple FSDB files are merged and appended in the aggregate performance analysis report with an extra column to indicate the FSDB file where the result originated.

To view the aggregate performance report for multiple FSDB files, you need to use the *Performance Analyzer* view in GUI mode as follows:

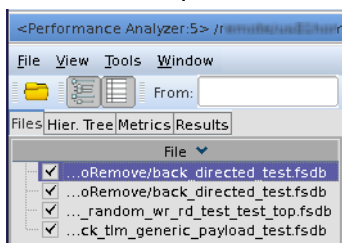
1. Invoke the Verdi GUI with the **Transaction Debug** work mode.
2. Click the **Menu -> Tools -> Transaction Debug -> Performance Analyzer** menu command to invoke *Performance Analyzer*.
3. In the *Performance Analyzer* pane, click the **File -> Load Database** command and choose the performance analysis aggregate result database from the *Load Database* form.

Chapter 5: Aggregate Performance Analysis

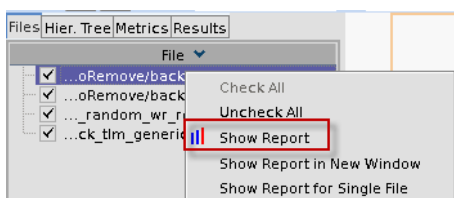
Viewing Aggregate Performance Analysis Report in GUI



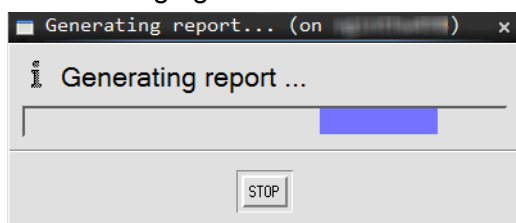
4. Select the required files in the **Files** tab of the *Hierarchy Tree* pane.



5. Click the **Show Report** icon in the toolbar or click the **Show Report** right-click command for the selected files.



Click the **Show Report** icon in *Performance Analyzer* which also displays a progress bar to notify users that the report generation is in progress. The progress bar is illustrated in the following figure:



Note:

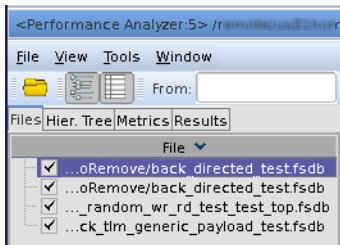
For aggregate performance databases, the **Reload** menu command is always disabled. If any FSDb files are added to or removed from the aggregate database, you must close the current aggregate database and open the same database again.

If any FSDB files inside the aggregate database are changed, the aggregate performance report is not updated automatically to include the change; you must add the same FSDB file to overwrite the previous FSDB file.

Files Tab for Multiple FSDB Files Database

If multiple FSDB files are opened in *Performance Analyzer*, the *Files* tab is available in the *Hierarchy Tree* pane, which displays the list of all the FSDB files present in the database. You can select any of the FSDB files (by checking the check-box) to view its performance report and perform other operations.

The *Files* tab is illustrated in the following figure:



The following right-click commands are available in the *Files* tab:

- **Check All:** Selects/checks all the files.
- **Uncheck All:** Deselects/unchecks all the files.
- **Show Report:** Displays the performance report for the selected files.
- **Show Report in New Window:** Displays the performance report for the selected files in a new window.
- **Show Report for Single File:** Displays the performance report for the selected file (single file) in a new window and also opens the FSDB file in a new window.

Note:

This command is enabled only if the FSDB file exists.

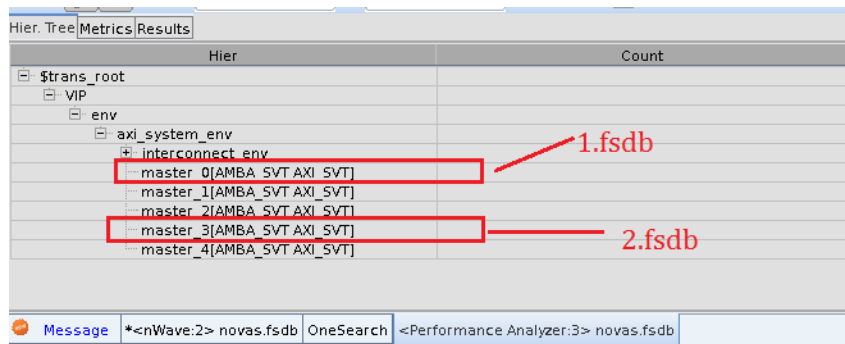
Hierarchy Tree Tab for Multiple FSDB Files

- The **Hierarchy Tree** displays only the instances. The stream nodes are not displayed since there are multiple FSDB files for the database and they may have different streams in the same named instance.
- The **Hierarchy Tree** displays a union of instances. For example, if *1.fsdb* has *VIP.env.axi_system_env.master_0* as an instance but *2.fsdb* does not have this instance and *2.fsdb* has *VIP.env.axi_system_env.master_3* as an instance but

1.fldb does not have it, then both of the instances appear in the hierarchy view of the merged database.

- The transaction count is not displayed since the transaction counts may be different for the same instance in different Fldb files.
- If the option to check an instance is disabled, that is, if the aggregate report is displayed for all the available instances. To view the report for different instances as the current aggregate database, re-generate the database with the new instance list.

The following figure illustrates the *Hierarchy Tree* tab for multiple Fldb files



Metrics Tab for Multiple Fldb Files

All the metrics are read-only because the metrics configuration is decided based on the results for the Fldb files that were added to the results database. You can not change/add or disable/enable the metrics,

- However, you can still use the *Metrics* view to change other settings which trigger recalculation, such as constraints, chart types, and so on.
- You can add/change constraints for metrics.
- You can set/change the chart type for metrics.

Results Tab for Multiple Fldb Files

Multiple instance metrics are evaluated on the merged instances from all Fldb files.

Performance Report Pane for Multiple Fldb Files

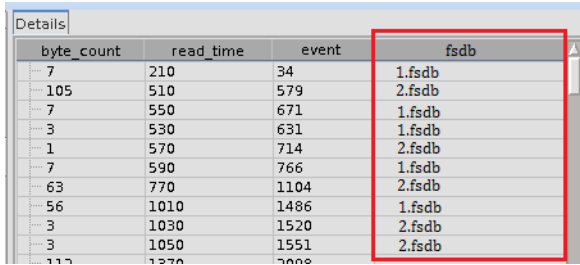
For some of the metrics (mostly instance based and multiple instances based metrics), the result for a single Fldb file is present in a single row and is displayed as a label.

However, for a result database with multiple Fldb files, the result is presented in a tabular form and is displayed as a line or a bar chart (with the x axis as the name of the Fldb and

the y axis as the result of that FSDB). Therefore, you can easily view the summary of the result distribution in different FSDB files.

Details Tab for Multiple FSDB Files

The *Details* tab for a result database with multiple FSDB files is similar to the single FSDB details pane. The difference between them is that, there exists an extra column to indicate the FSDB file to which the result is related. This is illustrated in the following figure:



byte_count	read_time	event	fsdb
7	210	34	1.fsdb
105	510	579	2.fsdb
7	550	671	1.fsdb
3	530	631	1.fsdb
1	570	714	2.fsdb
7	590	766	1.fsdb
63	770	1104	2.fsdb
56	1010	1486	1.fsdb
3	1030	1520	2.fsdb
3	1050	1551	2.fsdb

Note:

For aggregate databases, synchronization between *Details* pane and *Performance Report* pane is supported.

For aggregate databases, synchronization between *Performance Analyzer* view and other views (like protocol views) is not supported, since there are multiple FSDB files involved. To synchronize with other views, you must open a single FSDB file in a new *Performance Analyzer* window.

Examples to Demonstrate Merged Results for Multiple FSDB Files

This section provides various examples to illustrate the merged results for multiple FSDB files.

Example 1

Consider the `trans_read_latency` metric, which is used to calculate the read latency for every single read operation. Assume that you have two FSDB files: *1.fsdb* and *2.fsdb*.

The result for *1.fsdb* is shown in the following table:

Latency	Request_Time	Response_Time
1000	1000	2000
1500	2000	3500

Latency	Request_Time	Response_Time
2000	3000	5000

The result for *2.fsdb* is shown in the following table:

Latency	Request_Time	Response_Time
1500	1000	2500
2000	2000	4000
2500	3000	5500

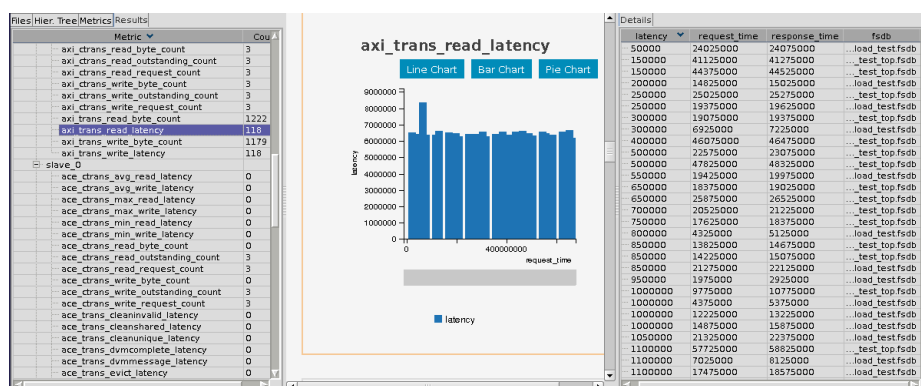
The merged metric result is shown in the following table:

Latency	Request_Time	Response_Time	FSDB
1000	1000	2000	1.fsdb
1500	2000	3500	1.fsdb
1500	1000	2500	2.fsdb
2000	3000	5000	1.fsdb
2000	2000	4000	2.fsdb
2500	3000	5500	2.fsdb

As an example, for `axi_trans_read_latency` metric, the merged results for two FSDB files is illustrated in the following figure:

Chapter 5: Aggregate Performance Analysis

Examples to Demonstrate Merged Results for Multiple FSDB Files



Example 2

Consider the `ctrans_min_read_latency` metric, which is used to compute the minimal read latency for all read operations associated with an instance.

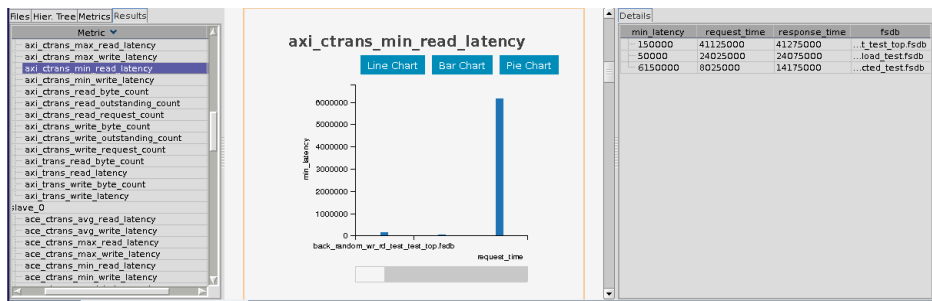
Assume that you have three FSDB files: *1.fsdb*, *2.fsdb*, and *3.fsdb*. The results are as follows:

- The result for *1.fsdb* is 1000
- The result for *2.fsdb* is 2000
- The result for *3.fsdb* is 2500

The merged metric result for multiple FSDB files is shown in the following table:

Mimimal Latency	FSDB
1000	1.fsdb
2000	2.fsdb
2500	3.fsdb

As an example, for `axi_ctrans_min_read_latency` metric, the merged results for three FSDB files is illustrated in the following figure:



Example 3

Consider the `ctrans_avg_read_latency` metric, which is used to compute the average read latency for all read operations associated with an instance.

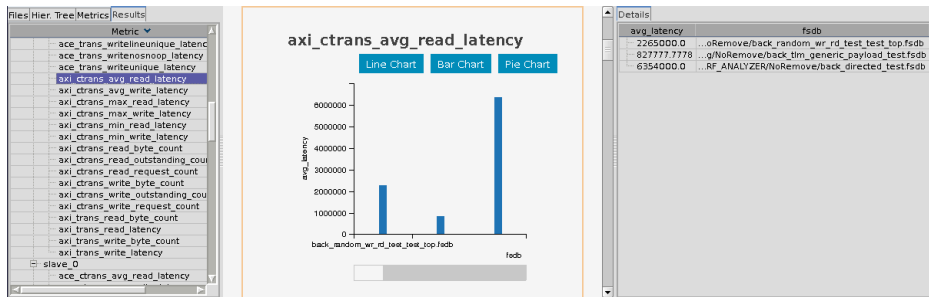
Assume that you have three FSDB files: `1.fsdb`, `2.fsdb`, and `3.fsdb`. The results are as follows:

- The result for `1.fsdb` is 1500 (count 3)
- The result for `2.fsdb` is 2500 (count 3)
- The result for `3.fsdb` is 3000 (count 3)

The merged metric result for multiple FSDB files is as follows:

Average Latency	FSDB
1500	1.fsdb
2500	2.fsdb
3000	3.fsdb

As an example, for `axi_ctrans_avg_read_latency` metric, the merged results for three FSDB files is illustrated in the following figure:



Example 4

Consider the `cinst_read_request_percentage` metric, which is used to calculate the percentage of read request counts in different instances.

Assume that you have two FSDB files: *1.fldb* and *2.fldb*.

The result for *1.fldb* is shown in the following table:

Request_count	Instance
20	A
30	B
40	C

The result for *2.fldb* is shown in the following table:

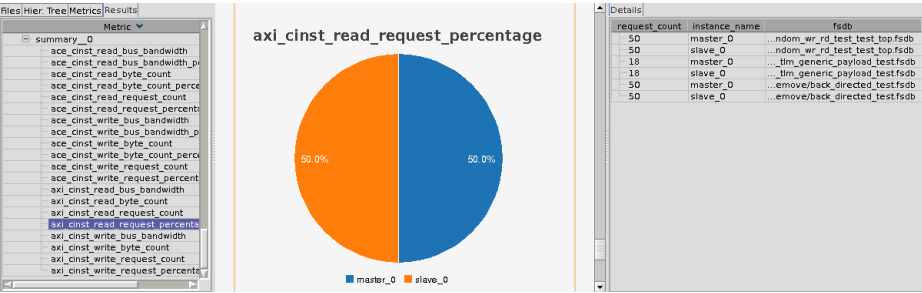
Request_count	Instance
40	A
60	B
70	C

The merged metric result for multiple FSDB files is shown in the following table:

Request_count	Instance	FSDB
20	A	1.fldb
30	B	1.fldb
40	C	1.fldb

Request_count	Instance	FSDB
40	A	2.fsdb
60	B	2.fsdb
70	C	2.fsdb

As an example, for `axi_cinst_read_request_percentage` metric, the merged results for three FSDB files is illustrated in the following figure:



6

Custom Performance Analyzer Metrics

This chapter provides you an insight into computation and visualization elements that comprise a performance metric, which helps you to understand the design considerations and the operation of previously-written performance metrics. This information is also a prerequisite to designing and implementing custom performance metrics (either by modifying a previously written performance metric or creating a new performance metric entirely from scratch).

This chapter helps you to understand the following concepts in detail:

- The elements that comprise a performance metric and how they are utilized
- The computation and visualization elements that comprise a performance metric

This chapter also explains how to perform the following tasks:

- Modify the computation element of a predefined metric provided with a Synopsys VIP to create a new custom performance metric.
- Modify the visualization element of a pre-defined metric provided with a Synopsys VIP to create a new custom performance metric.

This chapter primarily focuses on the portion of the metric creation process that is utilized to extract information from transactions in the FSDB file and assumes a basic operational knowledge of *Protocol Analyzer* and *Performance Analyzer*.

This chapter consists of the following sections:

- [Elements of a Performance Metric](#)
- [Performance Metric Definitions](#)
- [Performance Metric Column Data Types](#)
- [Performance Metric Visualization](#)
- [Viewing, Modifying, and Creating Performance Metrics](#)
- [Summary](#)

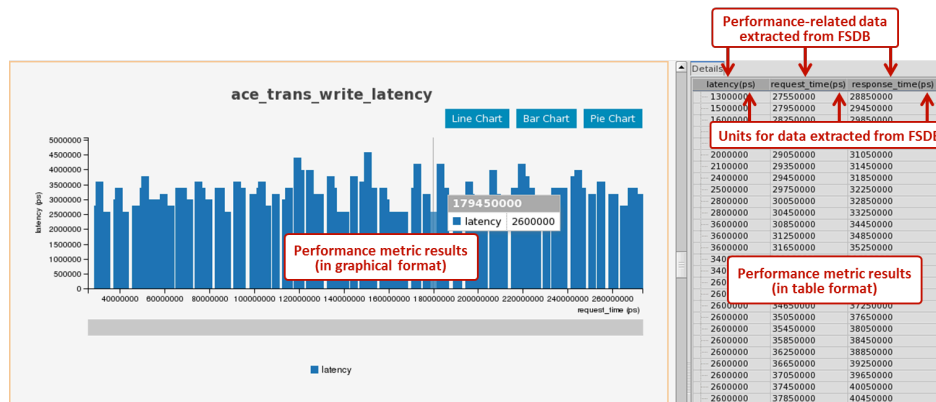
Elements of a Performance Metric

The following three elements comprise a performance metric:

1. Performance metric definition: Defines how to extract the desired performance data from the VIP transaction data in the FSDB file.
2. Data or column type specifications: Indicates the units associated with each piece of data which has been extracted from the FSDB file for each performance metric.
3. Visualization attributes associated with each performance metric: Defines how the resulting data must be presented. Note that both elements of a performance metric are utilized in the interactive mode and the batch mode.

The following figure illustrates the meaning of these three elements:

Figure 42 Elements of a Performance Metric



Performance Metric Definitions

A performance metric definition is characterized by the following attributes:

- Name of the protocol for which the performance metric is defined
- Name of the performance metric
- Description of the performance metric
- Expression to use to calculate the metric
- Type of metric being defined (instance or multi-instance)
- Metrics on which the metric depends (if any)

The first three items in the above list are relatively self-explanatory. However, the last three items are explained in the following sections. The visualization of performance metrics is discussed in a separate section.

Performance Analyzer SQL Database

Internally, *Performance Analyzer* constructs an SQLite database that contains all the protocol transaction information that is captured during simulation. This database enables the creation of SQL expressions to extract the data necessary to calculate each performance metric. While this chapter is not intended to be an SQL tutorial, basic SQL expressions are discussed using examples.

An SQL database contains a collection of tables. Each table can be thought of as an Excel spreadsheet, with rows and columns. One or more tables are created for each VIP instance in the design. For each table, each row contains information about an individual transaction and the each column represents the individual attributes for each transaction.

Since the metric expressions are written in SQL, knowledge of the SQL database structure is required to understand how the metric expressions need to be written and how those expressions are evaluated.

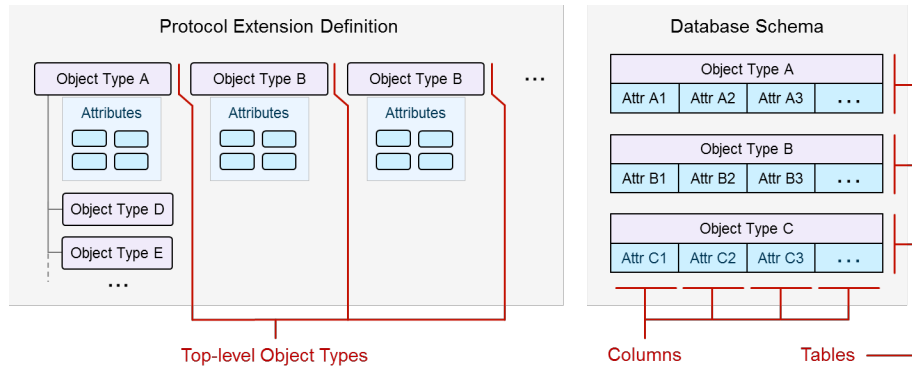
The general structure of the SQL database that is associated with a protocol instance is derived from the object types in the protocol extension definition that is associated with that instance. If performance metrics are created for a custom protocol, the details of the object types protocol extension definition are defined during the VIP instrumentation process. For Synopsys VIPs, this information is understood by examining the results of a simulation in Protocol Analyzer (which is shown in detail below).

The protocol transaction information is used to construct the SQL database in the following manner:

- Each top-level object type in the protocol extension definition is mapped to a separate table in the SQL database for each instance. The name of the table is constructed from the name of the instance and the name of the top-level object type display name (as it appears in the *Protocol Analyzer* hierarchy tree).
- Each attribute that is defined for an object type in the protocol extension definition is mapped to a column in the table in the SQL database that is associated with the object type. The name of the column is the same as the attribute display name (as it appears in the *Protocol Analyzer* hierarchy tree).
- For object types that define an object sub-type in the protocol extension definition and the object sub-type defines additional attributes, those attributes are also mapped to columns in the database table associated with the top-level object type. The name of the column is the same as the attribute name (as specified in the protocol extension).

The following figure is a simple visual representation of the relationship between a protocol extension definition and the corresponding performance database schema:

Figure 43 Relation Between Protocol Extension Definition and Corresponding Performance Database Schema

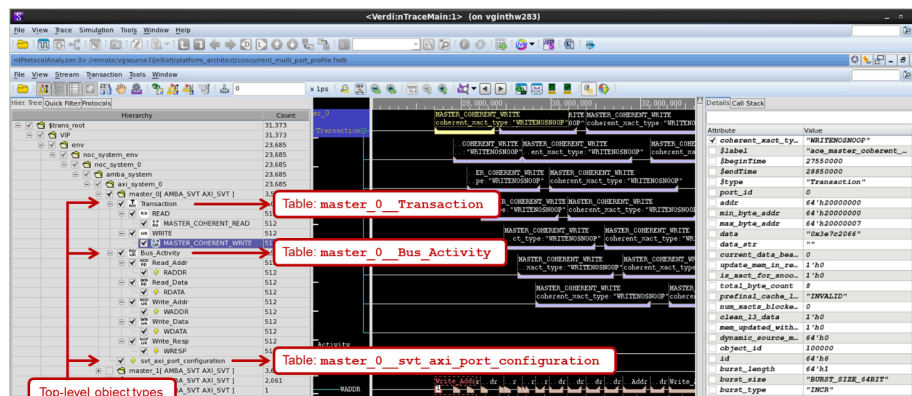


As previously mentioned, the name of the tables that are created in the SQL database are constructed from the name of the VIP instance and the name of each top-level object type. The format of the table name is as follows:

`<instance_name>__<top_level_object_type_name>`

For Synopsys VIPs, the protocol extension definition is provided as part of the VIP. However, the object type hierarchy can be easily understood by examining the protocol transactions that are created during simulation in *Protocol Analyzer*, as illustrated in the following figure:

Figure 44 Protocol Transactions Created During Simulation



The above figure shows the transactions recorded during the simulation of an AMBA AXI system. Although not all the instances are visible, the design contains multiple AXI master and AXI slave VIP instances.

The figure focuses on a VIP instance named `master_0`. On expanding the hierarchy tree for this instance, the top-level object types are easily visible. For this protocol, there are three top-level object types: `Transaction`, `Bus_Activity`, and `svt_axi_port_configuration`.

Note that there are object sub-types for two of the top-level object types. For example, the `Transaction` object type has two sub-types called `READ` and `WRITE`. Each of those object sub-types have additional object sub-types and taken together, these define an object type hierarchy. When looking at the object type hierarchy in *Protocol Analyzer*, it is important to note that only the object types for the transactions that were created in the specific simulation are displayed. The protocol extension definition may define other object types, but if transactions of those types are not created during the simulation, they do not appear in the *Hierarchy Tree*.

In addition to the columns that are associated with object type attributes (the information that is displayed in the *Details* tab in the previous figure), each table in the database that is associated with an object type has the following additional columns:

Column Name	Description
<code>begin_time</code>	This column stores the begin time of the transaction.
<code>end_time</code>	This column stores the end time of the transaction.
<code>event</code>	This column stores the unique identifier associated with a transaction. While the value of this identifier is not particularly useful to an end user, it is used to enable selection synchronization between <i>Performance Analyzer</i> results and <i>Protocol Analyzer</i> .
<code>parent_event</code>	This column stores the unique identifier for the parent transaction (if the transaction has a parent transaction). The value for this column is NULL for transactions that do not specify a parent transaction.
<code>pred_event</code>	This column stores the unique identifier for the predecessor transaction (if the transaction has a predecessor transaction). The value for this column is NULL for transactions that do not specify a processor transaction.
<code>succ_event</code>	This column stores the unique identifier for the successor transaction (if the transaction has a successor transaction). The value for this column is NULL for transactions that do not specify a successor transaction.

Column Name	Description
type_path	This column stores a string that represents the complete object type <i>path</i> . If the transaction object type is a top-level object type, then the value for this column is same as the <i>object_type</i> column. For object sub-types, the format of the path is: <i><top_level_object_type>.<object_sub_type>.<object_sub_type> ...</i>
channel	This column stores the name of the channel if any protocol-related transactions specify a channel (such as <i>Rx</i> or <i>Tx</i>).
label	This column stores the label associated with a transaction. This value corresponds to the value of the attribute <i>\$label</i> , which is shown in the <i>Details</i> tab of <i>Protocol Analyzer</i> .

All protocol related transactions are converted into an SQL database (based on the database schema described above). During this conversion process, each transaction is added as a record (that is, a row) in one of the database tables based on the object type transaction's. This database can then be used to evaluate the metric expressions that are defined for each performance metric (and written using SQL syntax) to extract the required performance information.

Examples of a Performance Metric Definition

This sub-section analyzes a basic performance metric definition using the following two examples:

- [Example 1](#)
- [Example 2](#)

Example 1

In this example, the performance metric definition calculates the latency of specific transaction types and is one of the pre-defined performance metrics that is provided with the *Synopsys AMBA ACE VIP*. Even though this metric definition is specific to the *Synopsys AMBA ACE VIP*, the calculation of transaction latency is common to almost every protocol, which makes this a particularly useful metric to examine in detail.

Each performance metric definition is described using the `perfCreateMetric` Tcl command, which has the following syntax:

Figure 45 Syntax - `perfCreateMetric`

```
perfCreateMetric -protocol <protocol_suite_name> \
                 -name <metric_name> \
                 -description <metric_description> \
                 -definition <metric_sql_expression> \
                 -type <metric_type> \
                 -dependent <list_of_dependent_metrics>
```

Note that the six arguments to the `perfCreateMetric` Tcl command correspond to the six elements that comprise a performance metric definition (that are introduced at the beginning of this section). The methods by which predefined or custom metrics are loaded into *Performance Analyzer* is discussed in a later section of this document.

The following figure illustrates the definition of the predefined metric called `ace_trans_read_latency`:

Figure 46 Predefined Metric Definition

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
                 -name ace_trans_read_latency \
                 -type instance \
                 -definition { SELECT ace_transaction.end_time - ace_transaction.begin_time AS latency,
                                   ace_transaction.begin_time AS request_time,
                                   ace_transaction.end_time AS response_time,
                                   ace_transaction.event AS event
                               FROM $inst__Transaction AS ace_transaction,
                                   $inst__svt_axi_port_configuration AS ace_configuration
                               WHERE ace_transaction.type_path IN ('Transaction.READ.MASTER_COHERENT_READ',
                                                                    'Transaction.READ.SLAVE_COHERENT_READ')
                                   AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
                                       ace_configuration.axi_interface_type = 'ACE_LITE'); } \
                 -description {... metric computes time taken by an ACE read type transaction to complete} \
                 -dependent
```

The suite name (and protocol name, if appropriate)

The name of the metric

The metric type *

The metric definition (SQL expression)

A description of the metric

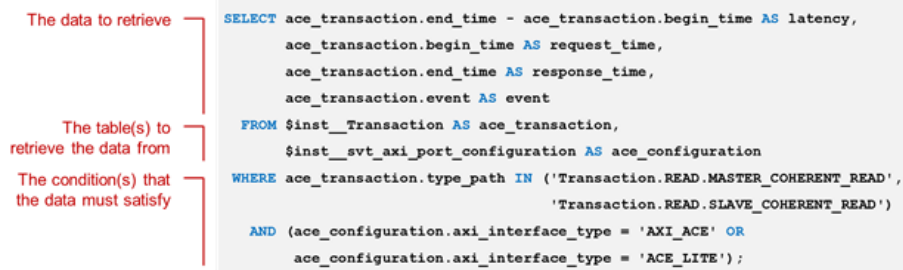
Dependent metrics (if any) *

* Topic will be discussed later

This performance metric computes the time taken by an *ACE Read* transaction to complete (that is, the latency of each *ACE Read* transaction).

This definition illustrates that most the work required to define a performance metric is constructing the appropriate SQL expression to extract the desired data from the SQL database. The SQL expression is shown separately in the following figure:

Figure 47 SQL Expression



While SQL expressions can be quite complex, they all have the general structure that is shown by the labels in the previous figure. Specifically, the SQL expression defines what data is to be retrieved, where that data is to be retrieved from (that is, which database tables) and the conditions that the table data must satisfy to be included in the results.

To understand this performance metric,

- It is easiest to first consider the *FROM* portion of the SQL expression.

The table(s) to retrieve the data from

```

FROM $inst__Transaction AS ace_transaction,
     $inst__svt_axi_port_configuration AS ace_configuration
  
```

For this performance metric, it is necessary to analyze data from two tables: *\$inst__Transaction* and *\$inst__svt_axi_port_configuration*.

As discussed earlier, *Transaction* and *svt_axi_port_configuration* are the names of two of the top-level object types that are defined for this protocol. Also, database tables are created based on the names of the instances in the design.

Since performance metrics are intended to be used for any design, the names of the specific instances in the design cannot be required when defining metrics. To satisfy this requirement, the *\$inst* string is used when defining performance metrics. When the performance metrics are evaluated, *Performance Analyzer* automatically replaces *\$inst* with the name of the instance for which the metric is being evaluated. For example, if this performance metric is being evaluated for the instance named *master_0*, the *\$inst* string is replaced with *master_0* and the tables called *master_0__Transaction* and *master0__svt_axi_port_configuration* are queried by the tool.

Note that this portion of the SQL expression includes the SQL keyword: *AS*, which allows a table to be locally named to make the rest of the expression easier to understand. For example, the table called *\$inst__Transaction* can also be referred to as *ace_transaction* in the rest of the SQL expression. In other words, this creates an alias for the table.

- Next, consider the *WHERE* portion of the SQL expression.

The condition(s) that the data must satisfy

```
WHERE ace_transaction.type_path IN ('Transaction.READ.MASTER_COHERENT_READ',
                                     'Transaction.READ.SLAVE_COHERENT_READ')
AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
     ace_configuration.axi_interface_type = 'ACE_LITE');
```

For this performance metric, the transactions in the database must satisfy two conditions to be included in the result.

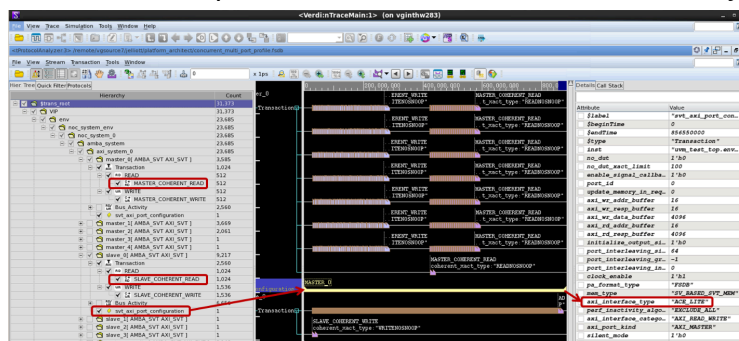
- The first condition that must be satisfied is that the value of *type_path* must be equal one of the two specified strings. As mentioned earlier, the *type_path* database column is automatically populated by *Performance Analyzer* and represents the complete object type *path* for each transaction. Note that, the table alias is used to reference the *type_path* database column (that is, *ace_transaction*), which means that this condition only applies to the database tables whose top-level object type is *Transaction*.

For the *Synopsys AMBA ACE VIP*, there are two READ transaction object types (one type is used if the VIP is configured to operate as a master; the other type is used if the VIP is configured to operate as a slave). Since this performance metric is intended to be applicable for both master and slave ACE VIPs, both values are included in the SQL expression.

- The second condition that must be satisfied is that the value of *axi_interface_type* must also equal one of the two specified values. This portion of the condition also uses a table alias, so this condition only applies to the database tables whose top-level object type is *svt_axi_port_configuration*. This additional condition is included in this metric, because the *Synopsys AMBA VIP* is very flexible and can be configured to represent one of the several different *AMBA* protocols, including *ACE* or *AXI*. This additional condition ensures that this metric only applies to instances of the *Synopsys AMBA VIP* that are configured to operate as *ACE* masters or slaves.

The following figure shows how *FROM* and *WHERE* portions of the SQL expression correlate to *Protocol Analyzer*.

Figure 48 SQL Expression Correlation With Protocol Analyzer



- Finally, consider the *SELECT* portion of the SQL expression.

The data to retrieve

```
SELECT ace_transaction.end_time - ace_transaction.begin_time AS latency,  
       ace_transaction.begin_time AS request_time,  
       ace_transaction.end_time AS response_time,  
       ace_transaction.event AS event
```

This portion of the SQL expression defines the information to be extracted from the database and placed in the Results table for this performance metric.

As mentioned earlier, this performance metric is intended to calculate the latency values for *ACE Read* transactions. The latency of a transaction is calculated by subtracting the end time of the transaction from the begin time of the transaction. This is the first value that is calculated for the performance metric. The *AS* portion of the expression allows you to specify a name for the calculated result. If a name is specified, that name is used for the column in the results table when the performance metric is evaluated.

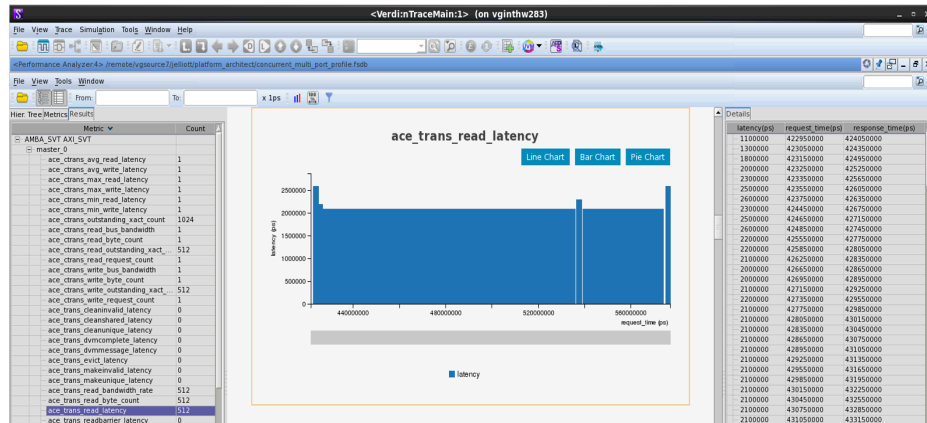
The next two values that are extracted from the transaction database are the begin time and end time for the transaction. As mentioned earlier, *begin_time* and *end_time* columns are automatically added to the transaction database tables by *Performance Analyzer*. These two values are included in the results table for two reasons. The first reason is for graphing. When this performance metric is graphed, the goal is to use the *latency* value for the x-axis and the *request_time* (that is, *begin_time* of the transaction) for the y-axis. Visualization definitions for performance metrics is described in detail in a later section of this document.

The final value that is extracted from the transaction database (and placed in the Results table) is the transaction *event*. This is the unique identifier that is associated with a transaction. While this value is not particularly useful to an end user, it enables selection synchronization between *Performance Analyzer* and *Protocol Analyzer*. For example, assume that the latency of a transaction violates a maximum constraint value specified by the user. If the *event* value is included in the metric result, the offending transaction can be selected in *Performance Analyzer* and the same transaction gets selected in *Protocol Analyzer* (note that the windows must have synchronization enabled to utilize this capability).

The names that are specified in the *SELECT* portion of the SQL expression (using the *AS* SQL keyword) define the column names in the result table that is generated when the performance metric is evaluated. For this example, there are four columns in the Results table with the names *latency*, *request_time*, *response_time* and *event*.

The following figure shows the result data for this performance metric in the *Performance Analyzer* GUI:

Figure 49 Result Data for Performance Metric



Since the *event* data is used internally by Verdi, the *event* values are not displayed in the data that is presented to the user.

Example 2

In this example, the performance metric definition calculates the number of bytes associated with a specific transaction type and is also one of the predefined performance metrics that is provided with the *Synopsys AMBA ACE VIP*.

This performance metric is slightly more complex than the previous example and introduces a few new concepts related to writing performance metrics. First, this metric demonstrates how to utilize the relationships (that is, parent-child) between transactions; and second, it showed that SQL *SELECT* statements can incorporate more complex manipulation of data.

The definition of the predefined metric called *ace_trans_read_byte_count* is illustrated in the following figure:

Figure 50 Predefined Metric Definition

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
-name ace_trans_read_byte_count \
-type instance \
-definition { SELECT ( cast( substr( ace_transaction.burst_size, 12 ) AS int ) / 8 ) AS byte_count,
                    activity_data.begin_time AS read_time,
                    ace_transaction.event AS event
FROM $inst__Bus_Activity AS activity_data,
    $inst__Transaction AS ace_transaction,
    $inst__svt_axi_port_configuration AS ace_configuration
WHERE activity_data.type_path = 'Bus_Activity.Read_Data.RDATA'
AND activity_data.parent_event = ace_transaction.event
AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
      ace_configuration.axi_interface_type = 'ACE_LITE'); } \
-description {... metric computes computes total byte count of the READ type transaction} \
-dependent
```

This performance metric computes the number of bytes associated with each ACE read. The SQL expression is shown in the following figure:

Figure 51 SQL Expression

Data to retrieve Table(s) to retrieve the data from Condition(s) that the data must satisfy	<pre>SELECT (cast(substr(ace_transaction.burst_size, 12) AS int) / 8) AS byte_count, activity_data.begin_time AS read_time, ace_transaction.event AS event FROM \$inst__Bus_Activity AS activity_data, \$inst__Transaction AS ace_transaction, \$inst__svt_axi_port_configuration AS ace_configuration WHERE activity_data.type_path = 'Bus_Activity.Read_Data.RDATA' AND activity_data.parent_event = ace_transaction.event AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR ace_configuration.axi_interface_type = 'ACE_LITE');</pre>
--	---

Similar to the previous performance metrics example, to understand this performance metric,

- First, consider the *FROM* portion of the SQL expression.

Table(s) to retrieve the data from	<pre>FROM \$inst__Bus_Activity AS activity_data, \$inst__Transaction AS ace_transaction, \$inst__svt_axi_port_configuration AS ace_configuration</pre>
---	--

For this performance metric, it is necessary to analyze data from three tables. The names of these tables are *\$inst__Bus_Activity*, *\$inst__Transaction*, and *\$inst__svt_axi_port_configuration*, which is similar to the previous example, with the addition of the *\$inst__Bus_Activity* table. Note that, this table includes an *AS* statement that allows the table to be referred to as *activity_data* in the rest of the SQL statement.

- Next, consider the *WHERE* portion of the SQL expression

Condition(s) that the data must satisfy	<pre>WHERE activity_data.type_path = 'Bus_Activity.Read_Data.RDATA' AND activity_data.parent_event = ace_transaction.event AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR ace_configuration.axi_interface_type = 'ACE_LITE');</pre>
--	---

The portion of the *WHERE* condition which selects only *ACE* instances is the same the previous example. However, the way the desired transactions are selected is somewhat different than the previous example. In this example, the transactions are selected utilizing the parent-child relationships.

To provide some background, for each *ACE Read* transaction, there are read data transactions that are recorded as child transactions, whose type is *RDATA* and are recorded as *Bus_Activity*. Thus, the condition checks two things:

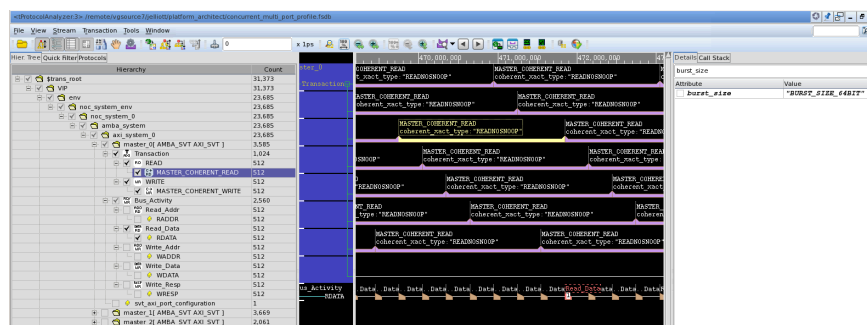
- Firstly, the condition checks the database for any transactions in the *Bus_Activity* table whose type is *RDATA*
- Secondly, the condition limits the result to transactions in the *Bus_Activity* table that have a parent-child relationship with a transaction in the *Transaction* table. This is accomplished using the *event* and *parent_event* attributes that are automatically generated by *Performance Analyzer* when the SQL tables are constructed.
- Finally, consider the *SELECT* portion of the SQL statement.

Data to retrieve {

```
SELECT ( cast( substr( ace_transaction.burst_size, 12 ) AS int ) / 8 ) AS byte_count,
        activity_data.begin_time AS read_time,
        ace_transaction.event AS event
```

As previously mentioned, the goal of the metric is to extract the number of bytes associated with each *ACE Read* transaction. Consider the following screenshot from *Protocol Analyzer*:

Figure 52 Protocol Analyzer Screenshot



In the above figure, an *ACE Read* transaction is selected. In addition, the filter capability of the *Details* tab is used to show the value of the *burst_size* attribute. The value of that attribute for the selected transaction is *BURST_SIZE_64BIT*, which is a string. However, the metric needs to extract an integer value. This can be accomplished by using some of the more advanced capabilities of the *SQLite* language. The SQL language provides many built-in functions that can be utilized in SQL expressions. This expression uses two of these functions: *substr* and *cast*.

The *substr* function is used to transform the string *BURST_SIZE_64BIT* to *64BIT*. This new string can then be transformed to an integer using the *cast* function. The result is *64*, but this is the number of bits associated with the transaction. To compute the number of bytes, this value (which is now an integer) can be divided by 8 to calculate the number of bytes associated with the transaction.

Performance Analyzer supports all the built-in functions available with *SQLite*. There are many on-line resources that list these functions and provide examples of their usage. For additional information about the built-in functions available with *SQLite*, see https://sqlite.org/lang_corefunc.html

In addition to the built-in functions that are available with *SQLite*, *Performance Analyzer* provides four extended functions, which are described in the following table:

Function	Definition	Input	Output
get_int	Translates a string to an integer	A decimal value string	A 64-bit integer
hex2int	Translates a hex value string to an integer	A hex string with the format <code><size>'h<hex_value>'</code> . (" <code><size></code> " is optional)	A 64-bit integer
power	Calculates a power	Two values, "a" and "b" of type integer or float	A double value: <code>pow(a,b)</code>
power2	Calculates a power of 2	An integer, "a"	An integer: <code>2^a</code>

For general information about *SQLite*, see <https://sqlite.org>

An SQL statement can also refer to *special* internal variables, that are unique to *Performance Analyzer*. These variables are described in the following table:

Variable	Meaning
\$inst	The instance on which the metric is evaluated (required by most metric definitions)
\$begin_time	The start time for performance evaluation. The unit for this value is the raw time unit of the FSDB file
\$end_time	The end time for performance evaluation. The unit for this value is the raw time unit of the FSDB file

\$sim_begin_time	The begin time of the FSDB file. The unit for this value is the raw time unit of the FSDB file
\$sim_end_time	The end time of the FSDB file. The unit for this value is the raw time unit of the FSDB file
\$timeunit	The time unit of the FSDB file (for example, "ns", "s", "ms", ...)
\$timeunit_factor	A double value used to normalize the FSDB file time unit to "ns"

Instance and Multi-Instance Performance Metrics

There are two types of performance metrics:

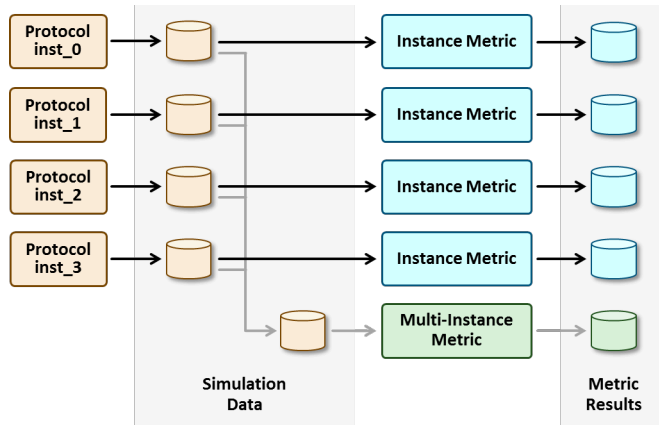
- Performance metrics based on data from a single instance, referred to as *Instance* performance metrics.
- Performance metrics based on data from multiple instances, referred to as *Multi-Instance* performance metrics.

The difference between these two metric types is the data that is provided as input to the metric (and thus what data can be referenced in the metric expression).

As the name implies, the input to an *Instance* performance metric is simulation data that is associated with a single instance (of a protocol). However, for a *Multi-Instance* performance metric, the simulation data from multiple instances (of a protocol) is first aggregated and then provided as an input to the metric.

The following figure illustrates the difference between an *Instance* performance metric and a *Multi-Instance* performance metric (in this example, when four instances of a protocol are applied to each):

Figure 53 Instance and Multi-Instance Performance Metrics



The two examples that are described in the previous sections are both instance performance metric. This was indicated in the `perfCreateMetric` Tcl command by the `-type instance` argument. This means that performance metric results are generated for each of the instances selected in *Performance Analyzer* (provided they match the criteria of the specified metric).

For example, the sample FSDB file that is used for the previous examples has five *ACE Master* instances (called *master_0* through *master_4*) and four *ACE Slave* instances (called *slave_0* through *slave_3*). Therefore, if all instances are selected and the previously described two performance metrics are to be evaluated, *Performance Analyzer* performs 18 evaluations (9 instances x 2 metrics) and produces 18 results tables.

Each of the results tables are named based on the name of the instance and the name of the metric. The name of each results table is constructed using the following pattern:

```
<instance_name>__<metric_name>
```

For example, if just the ACE master instances are selected, the following results tables is created:

```

master_0__ace_trans_read_latency
master_1__ace_trans_read_latency
master_2__ace_trans_read_latency
master_3__ace_trans_read_latency
master_4__ace_trans_read_latency
master_0__ace_trans_read_byte_count
master_1__ace_trans_read_byte_count
master_2__ace_trans_read_byte_count
master_3__ace_trans_read_byte_count
master_4__ace_trans_read_byte_count

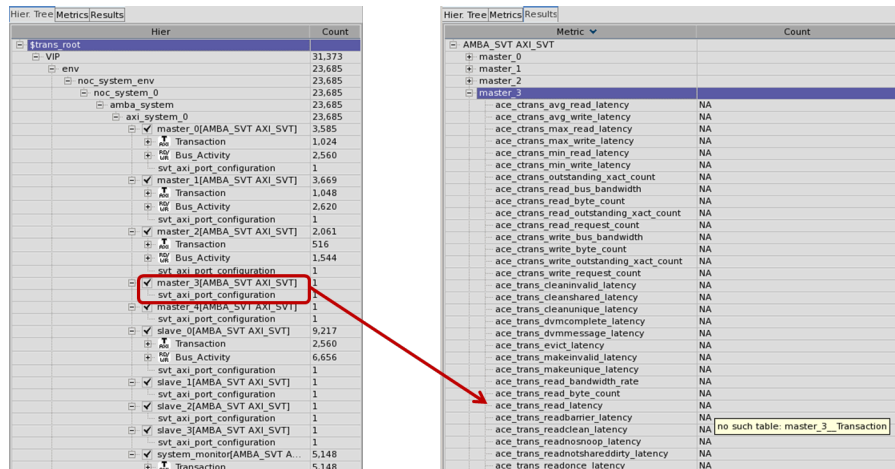
```

The behavior of the two metric types (*Instance* and *Multi-Instance*) is summarized in the following table:

Metric Type	Input	Metric Result
Instance	The protocol-related simulation data associated with one of the selected instances ¹	The metric is evaluated for each of the selected instances; multiple metric results are generated (one for each selected instance)
Multi-Instance	The protocol-related simulation data associated with all selected instances ¹	The metric is evaluated only once for all selected instances; only one metric result is generated

Multi-instance metrics are explained in detail in examples later in this chapter.

Note that under certain conditions, all results tables are not generated. Specifically, for specific simulation results, it may not be possible to evaluate every performance metric on every selected instance. This can happen when certain input tables are not created because the simulation did not cause certain types of transactions to be recorded. Consider the following figure:



The image on the left shows the *Hierarchy tree* in *Performance Analyzer* that shows the instances to be evaluated. The image on the right shows the metric results.

Note that the *Count* column on the right shows the value "N/A" for the metrics shown.

The value "N/A" indicates that the metric could not be evaluated and the tooltip describes

1. This is a slight over-simplification, as the input can either be protocol-related simulation data or the results that were calculated by one or more dependent metrics. The concept of primary and dependent metrics is discussed in the next subsection.

the reason for the metric not being evaluated. In the case of the *ace_trans_read_latency* metric, it could not be evaluated because for this simulation data, there are no transactions associated with the instance *master_3*, that is, the table *master_3__Transaction* cannot be created. This does not indicate that there is anything wrong with the metric definition; it simply indicates that the metric cannot be calculated for this instance based on the input data provided to *Performance Analyzer*. A different FSDB file could yield completely different results.

Primary and Dependent Performance Metrics

Not only does a performance metric have a type, a performance metric can be further classified as being either a primary performance metric or a dependent performance metric.

If the expression that is used to calculate the performance metric is described directly in terms of protocol-related simulation data, then the metric is a primary performance metric. However, the metric model that is used by *Performance Analyzer* is very flexible; the model permits the specification of performance metrics that takes the results that are calculated by one or more other metrics as their input. If the expression that is used to calculate the performance metric is described in terms of one or more other performance metric results, then the metric is a dependent performance metric.

Note that the figure from the previous sub-section illustrates only primary performance metrics (even though the figure shows both instance and multi-instance performance metrics), they are all primary performance metrics. However, it is also possible to define dependent, instance performance metrics and dependent, multi-instance performance metrics.

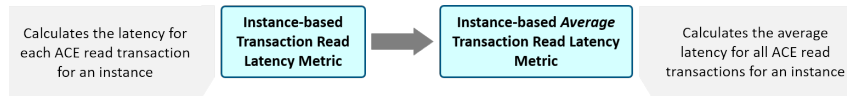
As an example, consider some performance metrics that could be defined for the *AMBA ACE* protocol. This document has already examined an instance performance metric that calculates the latency of each read transaction that is processed by a single instance of the *Synopsys ACE SVT VIP*. As demonstrated in the discussion of this example, the latency of an *ACE* transaction is calculated by subtracting the begin time of transaction from the end time of the transaction. Thus, if this performance metric is applied to an instance of the *ACE VIP* that processes 250 transactions over the course of the simulation, the calculation of the metric produces a result with 250 latency values^{2, 3}.

Now, consider some other performance metrics that could be defined for the *AMBA ACE* protocol. One of these might be the average latency for transactions processed by an instance of the *ACE VIP*. The average value can easily be calculated by taking

2. The result could contain additional information, but the main point is that the result can at least contain a collection of latency values.
3. It is important to note that even though the result of calculation produces 250 latency values, the output of the metric calculation is still considered a single performance metric result. If you had selected a second instance of the *ACE VIP* that had calculated 100 transactions during the simulation, that can cause a second performance metric result to be created (containing 100 latency values).

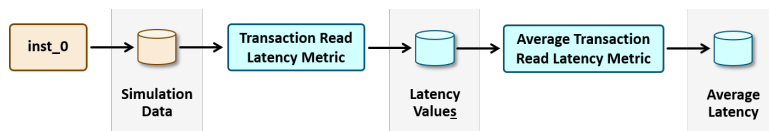
the average of the individual transaction latency values that were calculated by the first example metric. This is an example of a dependent, instance performance metric.

The relationship between these performance metrics is illustrated in the following figure:

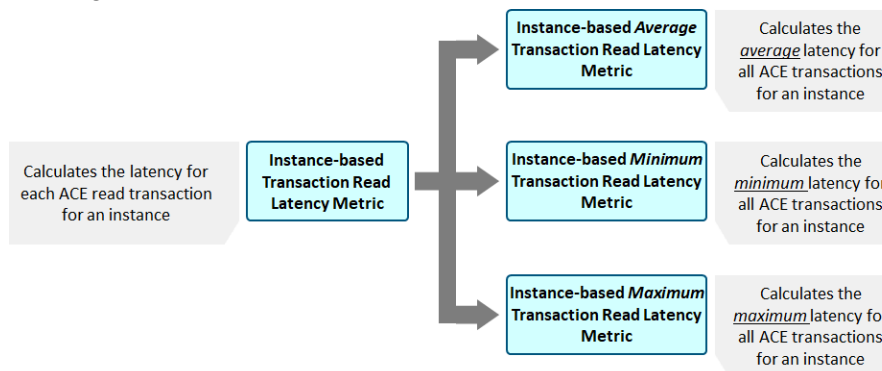


The arrow indicates that there is a dependency between these two metrics, which means that the read latency metric must be evaluated before the average read latency metric can be evaluated.

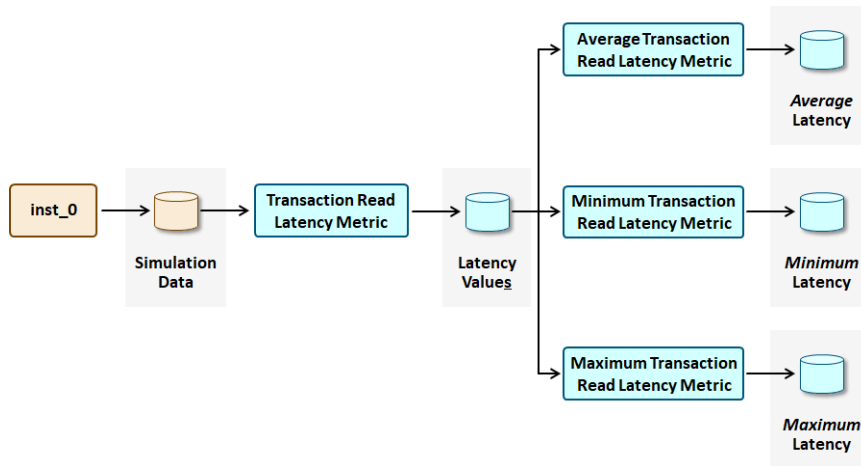
The following figure illustrates the processing steps (and data flow) associated with this performance metric and clearly shows that the input to the primary metric is protocol-related simulation data while the input to the dependent metric is the result of a metric evaluation.



The use of dependent performance metrics can greatly simplify the process of defining performance metrics. The following figure shows how the primary metrics can be leveraged to define similar metrics:



The following figure shows the combined processing steps for these three metrics:



An additional advantage to utilizing this approach is illustrated in the previous figure; note that it is only necessary for *Performance Analyzer* to evaluate the primary metric once, which also improves performance when these dependent metrics need to be evaluated for an instance.

Examples of Dependent Performance Metric Definition

This sub-section analyzes a dependent performance metric definition using the following two examples:

- [Example 1](#)
- [Example 2](#)

Example 1

Following is an example showing a dependent performance metric definition that is used to define one or more performance metrics:

```

perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
  -name ace_ctrans_avg_read_latency \
  -type instance \
  -definition { SELECT round( avg( latency ), 4 ) AS avg_latency
                FROM $inst__ace_trans_read_latency; } \
  -description {... computes the average latency of the READ type transactions} \
  -dependent ace_trans_read_latency
  
```

This performance metric computes the average latency for all the read latencies for each selected instance. The type of this metric is still *instance* (as it is for the previous examples), as the metric calculates a value for each selected instance. However, note that this metric definition now lists a dependent metric (specifically *ace_trans_read_latency*),

which is previously examined. In this example, there is only one dependent metric, although multiple dependent metrics can be specified, if required.

The dependent metric is also utilized in the *FROM* portion of the SQL expression. It is prefaced with the string *\$inst__* in a similar manner as the transaction tables are referred to. When *Performance Analyzer* evaluates a dependent metric, the string *\$inst* is automatically replaced with the name of the metric for which the dependent metric is being evaluated. For example, if the average latency value is being computed for the instance called *master_0*, *Performance Analyzer* automatically uses the data from the results table called *master_0__ace_trans_read_latency* as the input for this metric.

The *SELECT* statement requires some additional explanation. First, the statement utilizes some built-in SQL functions that are previously described: *round* and *avg*. The syntax and use of these functions is self-explanatory.

However, the use of *latency* may not be so obvious. To understand where that column is defined, consider again the SQL statement for the dependent metric:

```
SELECT ace_transaction.end_time - ace_transaction.begin_time AS latency,
       ace_transaction.begin_time AS request_time,
       ace_transaction.end_time AS response_time,
       ace_transaction.event AS event
FROM $inst__Transaction AS ace_transaction,
     $inst__svt_axi_port_configuration AS ace_configuration
WHERE ace_transaction.type_path IN ('Transaction.READ.MASTER_COHERENT_READ',
                                   'Transaction.READ.SLAVE_COHERENT_READ')
AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
     ace_configuration.axi_interface_type = 'ACE_LITE');
```

In the dependent metric, the latency value is given the name *latency*. This name is used when creating the results tables for this metric. Thus, when a dependent metric needs to reference column values in a *Results* table, it must use the name that is assigned to the data in the original performance metric definition. In other words, the *Results* tables for the *ace_trans_read_latency* performance metric contains the tables - *latency*, *request_time*, *response_time*, and *event*.

Similar dependent metrics can be defined to calculate minimum and maximum latency for read transactions for each instance. The definition for the minimum latency for *ACE* read transactions is shown in the following figure:

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
  -name ace_ctrans_min_read_latency \
  -type instance \
  -definition { SELECT latency AS min_latency,
                request_time AS request_time,
                response_time AS response_time,
                event AS event
                FROM $inst__ace_trans_read_latency
                ORDER BY latency ASC LIMIT 1; } \
  -description {... computes the minimum latency of the READ type transactions} \
  -dependent ace_trans_read_latency
```

The general structure of this metric is similar to the metric that calculates the average latency for *ACE* read transactions, but the *SELECT* statement is different. There is no specific transaction that can be identified when calculating the average latency value. However, when calculating minimum or maximum latency values, these values can be associated with specific transactions. Since this information might help you to track down a problem, it is best to provide as much information as possible.

To facilitate this, the *SELECT* statement takes an entirely different approach to determine the minimum latency. It does so in a manner that the other transaction-related information can be easily extracted and placed in the *Results* table. One key feature of SQL is the ability to order rows based on column values. This is accomplished using *ORDER BY* and *ASC* (for ascending) or *DESC* (for descending) keywords. That is, this *SELECT* statement orders all of the items in the instance-specific *ace_trans_read_latency* results tables according to the value of the *latency* column in ascending order, and then selects all of the data from the first item (this is accomplished by *LIMIT 1*, which limits the results to the specified number of items). The effect of *LIMIT 1* is to select information from the transaction with the minimum latency value.

This metric can also be written in a manner similar to the metric that calculated the average latency value (such as, by using the *min* SQL function), but cross-selection is not possible with such a definition, as no *event* is present in the *Results* table.

Dependent metrics can simplify the creation of performance metrics and dramatically improve evaluation performance, hence they should be carefully considered when designing performance metrics.

Example 2

Following is an example showing a dependent performance metric definition that counts the number of bytes associated with each *ACE* read transaction; a similar dependent performance metric can be created for this example.

Consider a dependent performance metric (called *ace_ctrans_read_byte_count*) that calculates the total number of bytes associated with all of the *ACE* read transactions for an instance.

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
  -name ace_ctrans_read_byte_count \
  -type instance \
  -definition { SELECT sum( byte_count ) AS read_byte_count
                FROM $inst__ace_trans_read_byte_count; } \
  -description {... computes the byte count for all READ transactions} \
  -dependent ace_trans_read_byte_count
```

This performance metric computes the total byte count for all the *ACE* read latencies for each selected instance. The type of this metric is still *instance* (as it is for the previous examples), as the metric calculates a value for each selected instance. However, note that this metric definition now lists a dependent metric (specifically *ace_trans_read_byte_count*), which was previously examined.

As with the prior example, the dependent metric is also utilized in the *FROM* portion of the SQL expression. As is previously explained, it is prefaced with the *\$inst__* string in a similar manner as the transaction tables are referred to. Also, the use of the *byte_count* column comes from the definition of the dependent metric, whose definition is shown below for reference.

```
SELECT ( cast( substr( ace_transaction.burst_size, 12 ) AS int ) / 8 ) AS byte_count,
        activity_data.begin_time AS read_time,
        ace_transaction.event AS event
FROM $inst__Bus_Activity AS activity_data,
     $inst__Transaction AS ace_transaction,
     $inst__svt_axi_port_configuration AS ace_configuration
WHERE activity_data.type_path = 'Bus_Activity.Read_Data.RDATA'
      AND activity_data.parent_event = ace_transaction.event
      AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
           ace_configuration.axi_interface_type = 'ACE_LITE');
```

Multi-Instance Performance Metrics

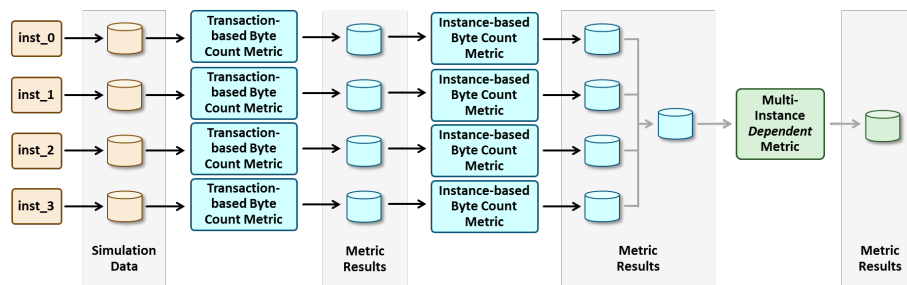
This chapter so far has focused on instance performance metric definitions. Examples are shown that calculate the latency for each *ACE* read transaction associated with an instance and both average and minimum latency for all *ACE* read transactions associated with an instance. Other examples are also shown that calculate the bytes counts for each *ACE* read transaction associated with an instance and the total byte count for all *ACE* read transactions associated with an instance.

It is useful to know the total byte count for all read transactions for all selected instances of the *ACE* VIP. The total can easily be calculated by adding together the total byte counts for the read transactions for each instance of the *ACE* VIP. The instance-based transaction counts can be calculated using the data that is calculated by the previously-described metric (*ace_ctrans_read_byte_count*). This is an example of a dependent, multi-instance performance metric.

The relationship between these two performance metrics is illustrated in the following figure:



The following figure illustrates the processing steps (and data flow) associated with this performance metric. The figure shows how a "chain" of instance-based dependent metrics can be used to calculate a result that considers the results of all instance-based metric calculations and combines that data into a single result using a *multi-instance* metric.



The performance metric definition that calculates the total number of bytes associated with all transactions for all instances is shown as follows:

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
  -name ace_cinst_read_byte_count \
  -type multi_instance \
  -definition { SELECT sum(read_byte_count) AS byte_count
                FROM $inst_ace_ctrans_read_byte_count; } \
  -description {... computes the byte count for all READ transactions for all instances} \
  -dependent ace_ctrans_read_byte_count
```

Note that this performance metric definition (called *ace_cinst_read_byte_count*) is similar to the previously described dependent performance metric definition (called *ace_ctrans_read_byte_count*). Note that the metric type is defined as *multi_instance* instead of *instance*. Setting the type to *multi_instance* indicates to *Performance Analyzer* that *\$inst* no longer refers to the *Results* table for a single instance, but instead the results tables from all calculations of the dependent metrics should be merged prior to performing the evaluation of the metric.

Summary of Performance Metric Definition Types

As a quick review, there are four different types of performance metrics which differ by (a) whether the input data is associated with a single instance or multiple instances; and (b) whether the input data is protocol-related simulation data or is the result of evaluating a different performance metric.

The four types of performance metrics are as follows:

- Instance, primary performance metrics
- Multi-instance, primary performance metrics
- Instance, dependent performance metrics
- Multi-instance, dependent performance metrics

The four types of performance metrics are shown in the following figure:



As illustrated in the last example, different kinds of metrics can be “chained” to create complex relationships between performance metric definitions. *Performance Analyzer* automatically keeps track of these relationships so that the metrics are evaluated in the appropriate order to calculate the correct performance results.

Performance Metric Column Data Types

A unit must be assigned for each column that is extracted from the SQL database for a performance metric. Consider the SQL expression for the first performance metric that is introduced in this chapter. The user-visible data items are highlighted in red in the following figure:

```
SELECT ace_transaction.end_time - ace_transaction.begin_time AS latency,
       ace_transaction.begin_time AS request_time,
       ace_transaction.end_time AS response_time,
       ace_transaction.event AS event
FROM $inst__Transaction AS ace_transaction,
     $inst__svt_axi_port_configuration AS ace_configuration
WHERE ace_transaction.type_path IN ('Transaction.READ.MASTER_COHERENT_READ',
                                    'Transaction.READ.SLAVE_COHERENT_READ')
AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
     ace_configuration.axi_interface_type = 'ACE_LITE');
```

Each of these items must be assigned a data type. This is accomplished using the `perfSetColumnType` Tcl command.

```
perfSetColumnType -protocol <protocol_suite_name> \  
                  -name <metric_name> \  
                  -column <column_name> \  
                  -type <type_string> \  
                  -unit <unit>
```

The protocol suite name and the metric name must match the names that are specified in the `perfCreateMetric` Tcl command.

There are four possible values for the `type_string`, as described in the following table:

Value	Meaning
None	The result does not require a time unit. For example, if the performance metric produces a “count” (such as the number of “read” transactions), no unit is required.
Time	The column represents time. When this type is specified, the value of “unit” is almost always “fsdb time unit”, which is a special string that represents the time unit associated with the FSDB file. In the sample SQL above, “latency”, “request_time” and “response_time” units should all be set to “fsdb time unit”. In certain cases (which are typically rare), the column data is extracted from an attribute that already has a particular time unit associated with it. In such cases, the time unit can be provided (such as, “fs”, “ps”, “ns” and so on). If there is any doubt about the time unit, then usually “fsdb time unit” should be specified.
Data	The column represents a data value. Examples of units that can be specified with this type are: “Byte”, “KB”, “MB”, “GB”, and “TB”.
Bandwidth	The column represents the bandwidth data. Examples of units that can be specified with this type are: “MB/s”, “Byte/ns”, and so on.
Customize	The column represents a data type that does not match any of the data types already described in this table. For this type, the unit can be any string.

The SQL expression shown above is for the performance metric called *ace_trans_read_latency* for the protocol *AMBA_SVT AXI_SVT*. Thus, the following `perfSetColumnType` commands should be created for this performance metric:

```
perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name ace_trans_read_latency \
                  -column latency \
                  -type Time \
                  -unit fsdb time unit

perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name ace_trans_read_latency \
                  -column request_time \
                  -type Time \
                  -unit fsdb time unit

perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name ace_trans_read_latency \
                  -column response_time \
                  -type Time \
                  -unit fsdb time unit
```

Consider again the multi-instance performance metric that calculates the total number of bytes associated with all transactions for all instances. The column data is highlighted in red in the following figure:

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
                -name ace_cinst_read_byte_count \
                -type multi_instance \
                -definition { SELECT sum( read_byte_count ) AS byte_count
                             FROM $inst__ace_ctrans_read_byte_count; } \
                -description {This metric computes the byte count for all READ transactions for all instances} \
                -dependent ace_ctrans_read_byte_count
```

For this metric, the data type for the column is *Data* and the unit is *Byte*. Therefore, the corresponding `perfSetColumnType` Tcl command for this metric should be as follows:

```
perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name ace_cinst_read_byte_count \
                  -column byte_count \
                  -type Data \
                  -unit Byte
```

Performance Metric Visualization

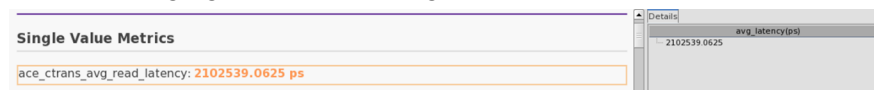
This chapter has described how to extract information from the SQLite database that represents the FSDB file transaction data and has also described how to assign data

types to each column of data that is extracted. The third element of a performance metric is the visualization of the performance data. Note that this document also contains detailed explanations about the various types of visualization that are available in the tool and specifically how performance metric visualization can be controlled through the GUI.

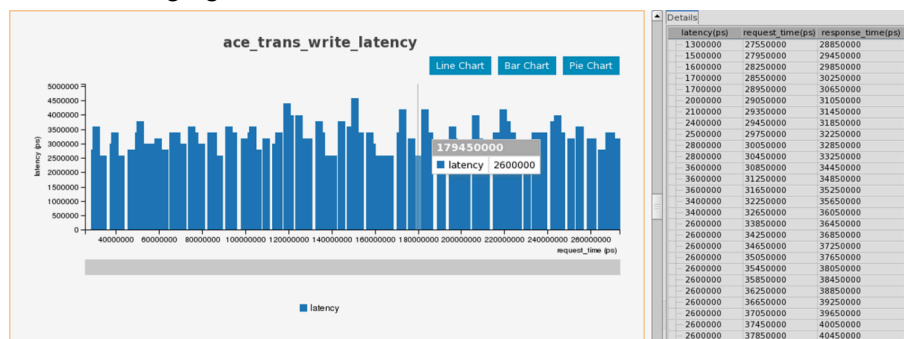
Types of Performance Metric Visualization

There are five ways in which performance metrics can be graphically displayed within *Performance Analyzer*: single-value performance results, bar charts, line charts, pie charts, and step charts.

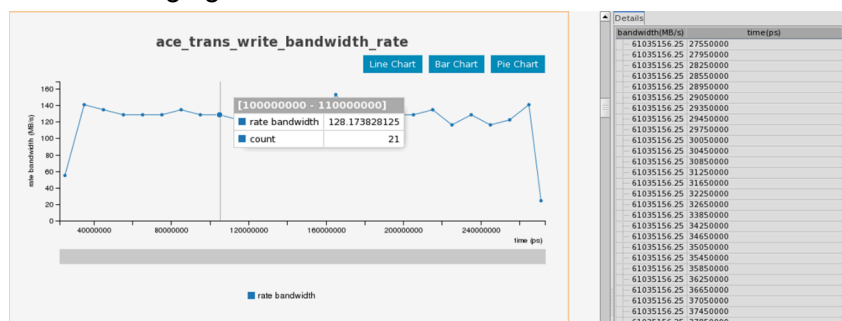
The following figure shows a single-value metric:



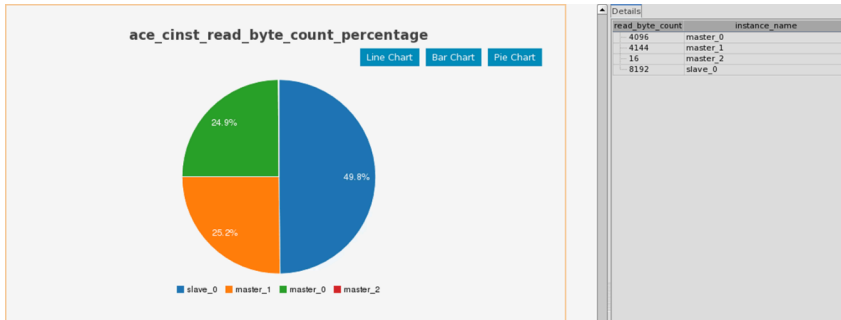
The following figure shows a bar chart:



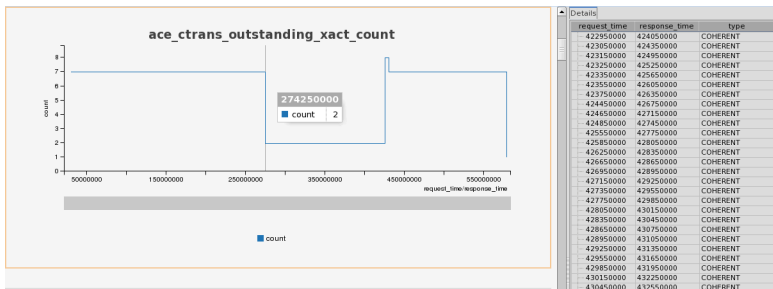
The following figure shows a line chart:



The following figure shows a pie chart:



The following figure shows a step chart:



Controlling Metric Visualization

Performance metric visualization is primarily controlled using the `perfSetChart` Tcl command. The following figure illustrates the Tcl command (including all possible command options):

```
perfSetChart -protocol <protocol_suite_name> \
  -name <metric_name> \
  -x <column_for_x_axis> \
  -y <column_for_y_axis> \
  -start <start_of_range> \
  -end <end_of_range> \
  -step <width_of_interval> \
  -aggr <aggregate_function> \
  -other <include_or_ignore_out_of_range_data> \
  -type <chart_type>
```

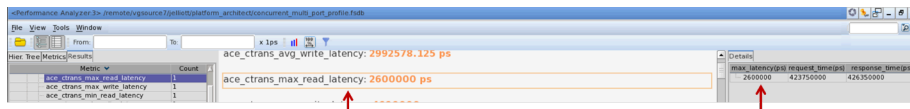
The `perfSetChart` Tcl command has many options, but typically only a few of these options are required. Note that this is the same Tcl command that can be used to modify the chart type of predefined metrics when running *Performance Analyzer* in batch mode (that is, using the `verdi_perf` command). Thus, there are a few options that are available to modify the visualization of a specific set of performance results (such as `-start` and `-end`), which means that these options are usually not utilized when defining a custom (or predefined) performance metric.

The values for `-protocol` and `-name` options are the same as described for previous Tcl commands (`perfCreateMetric` and `perfSetColumnType`). The other options are described in detail in the later sections of the document.

Default Metric Visualization

If there is no `perfSetChart` command associated with a performance metric, *Performance Analyzer* utilizes default chart types for displaying performance metric results.

If a performance metric result contains a single row of data, the first column of that row is displayed as a single-value metric. Following is an example:



Note that even though this is a single-value metric, the single result row includes multiple columns.

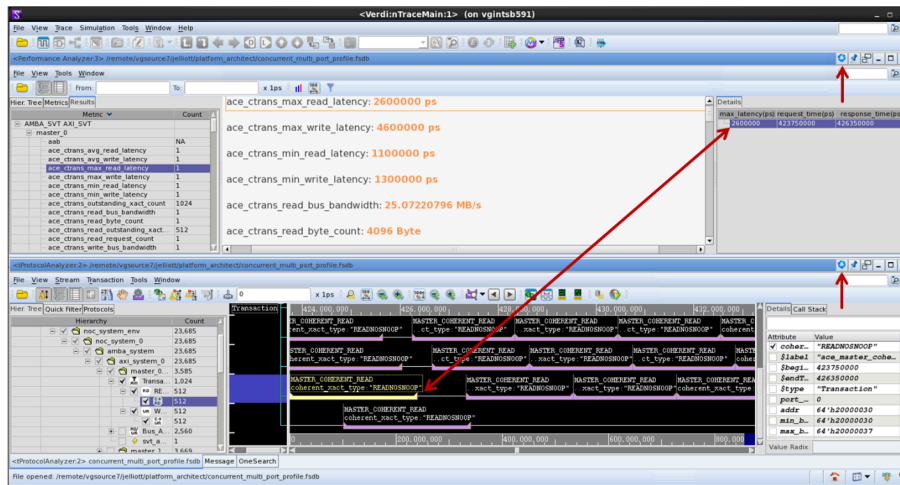
The columns that are created in the results table for this performance metric can be understood by examining the *SQL SELECT* statement that is associated with this metric:

```
SELECT latency AS max_latency,  
       request_time AS request_time,  
       response_time AS response_time,  
       event AS event  
FROM $inst__ace_trans_read_latency  
ORDER BY latency DESC LIMIT 1;
```

One of the columns (that is, “event”) is highlighted in red. Even though this is a single-value metric, by including this column in the result row, selection synchronization is possible between *Performance Analyzer* and *Transaction and Protocol Analyzer* windows, as shown in the following figure:

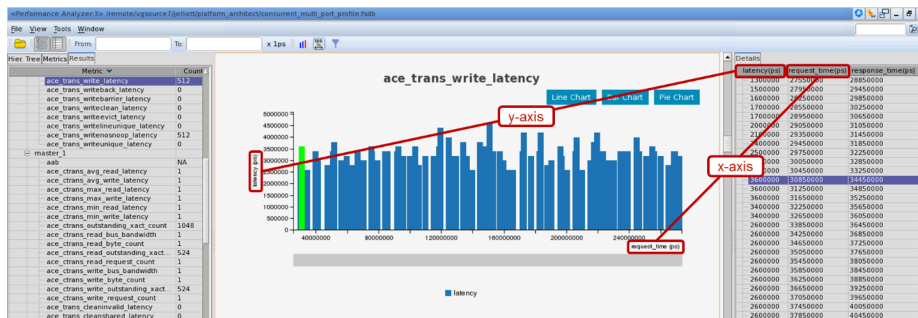
Chapter 6: Custom Performance Analyzer Metrics

Performance Metric Visualization



When creating custom performance metrics, consider adding any additional data in the *SELECT* statement that can aid the debug process. For this metric, not only is the *event* added to the results table, but also the *request time* and the result time.

If a performance metric result contains multiple rows of data, a bar chart is utilized, where the data found in the first column of the results table is used for the y-axis and data found in the second column of the results table is used for the x-axis. An example of this is as follows:



The results table for this performance metric also includes additional data, specifically the *response time* (which is shown in the table), and the *event* (which is not shown in the table, but enables selection synchronization between *Performance Analyzer* and *Transaction and Protocol Analyzer* windows).

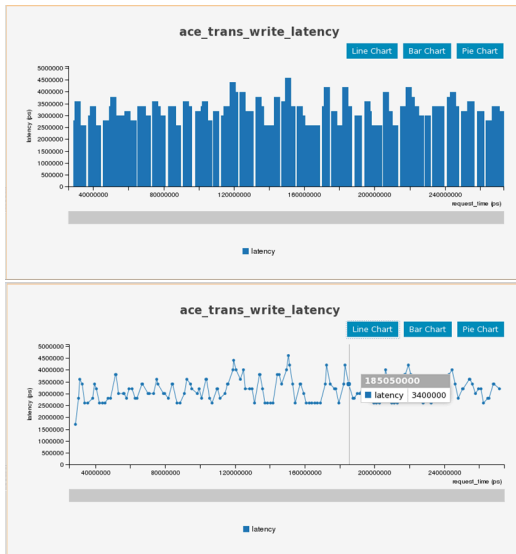
Simple Chart Types

For many performance metrics that produce a results table with multiple rows, the bar chart is often a reasonable visualization choice. However, for some performance metrics, a line chart may be more appropriate for representing the results. This may also be a personable preference.

To utilize a line chart instead of a bar chart, only a few of the options are used for the `perfSetChart` Tcl command⁴. The following `perfSetChart` command is used to display the `ace_trans_write_latency` performance metric as a line chart instead of a bar chart:

```
perfSetChart -protocol {AMBA_SVT AXI_SVT} \
             -name      ace_trans_write_latency \
             -type      line
```

Both the bar chart and the line chart visualizations of this performance metric are illustrated as follows:



The `-x` and the `-y` options for the `setPerfChart` Tcl command apply to bar charts and line charts and should be self explanatory. These options allow you to override the default behavior of which data to utilize for x-axis and y-axis. You can recall that by default, the data from the first column of the results table is used for the y-axis and the data from the second column of the results table is used for the x-axis. There is one exception to this rule: if the same column is specified for both the x-axis and the y-axis, the frequency (that is, the count) of every distinct x value is the y value of that x value.

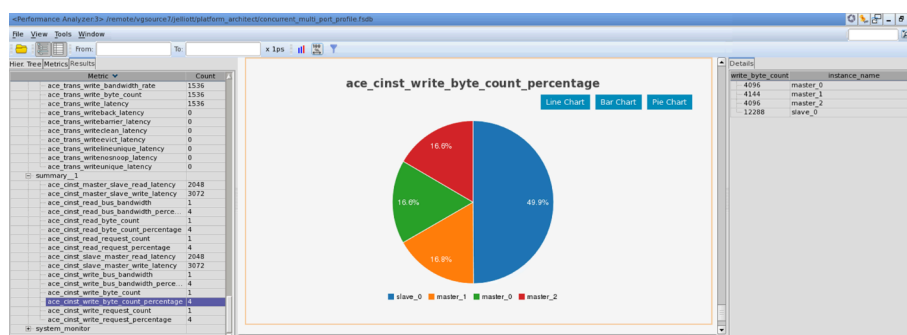
As previously mentioned, `-start` and `-end` options for the `setPerfChart` Tcl command provide you the options to visualize a portion of the performance metric results and is specific to a particular set of input data. Thus, these options are not applicable to general performance metric definitions and are shown in this document for completeness.

There are two other chart types that are yet described: pie charts and step charts. Pie charts are particularly useful for showing relative percentages. For example, in an *AMBA* system, a pie chart is a good way to visualize which master instances and slave instances are contributing to the overall transmission of bytes.

The following figure illustrates the pie chart:

4. More complex uses of both bar and line charts is described in the following section.

Chapter 6: Custom Performance Analyzer Metrics Performance Metric Visualization



The pie chart is constructed from the data in the first and second columns of the results table. The number of “slices” in the pie is determined by the number of unique values in the second column of the results table. Based on these “slices”, the values from the first column of the results table are added together (based on the “slices” with which they are associated) and then translated into percentages. From this chart, it is easy to see that the instance called *slave_0* is responsible for almost half of the byte count associated with this particular simulation result.

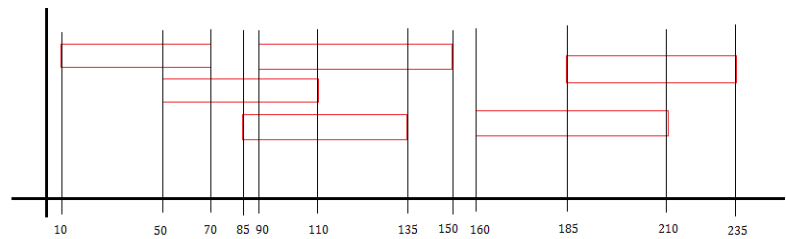
The step chart is used to show count changes on the x-axis. This chart uses the data from x-axis and y-axis differently than the other kinds of charts. The data for x-axis and y-axis are used to form a range; the chart does not directly use the x-axis and y-axis data for each item; instead, the tool performs a translation of the data to determine the values for the x-axis and y-axis.

This chart type is easiest described using an example.

Consider the following data:

delay	request_time	response_time
60	10	70
60	50	110
50	85	135
60	90	150
50	160	210
50	185	235

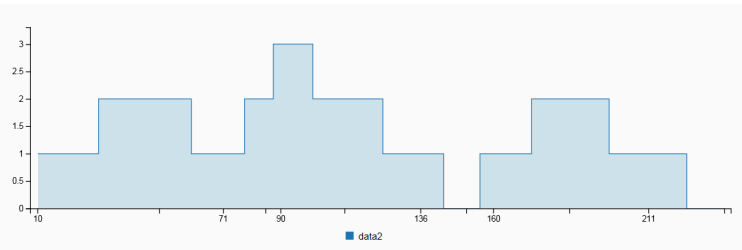
If *request_time* and *response_time* are chosen as x-axis and y-axis data respectively, then the range formed by x and y axis data looks like the following figure:



Once the counts of the overlapping data are calculated, the final data for x-axis and y-axis is as follows:

x (time for count change)	10	50	71	85	90	111	136	151	160	185	211	236
y (count of transactions)	1	2	1	2	3	2	1	0	1	2	1	0

Based on this calculated data, the step chart is as follows:



For the *AMBA ACE* protocol, one important performance metric is the number of “outstanding transactions”. This term refers to the number of transactions that are “active” at any time during the simulation. To calculate this performance metric, it is necessary to select the *begin time* and the *end time* of each transaction of interest. Once this data is selected, the number of outstanding transactions can be easily displayed using a step chart.

The `perfCreateMetric` and `perfSetChart` commands for the `ace_ctrans_read_outstanding_xact_count` performance metric are as follows:

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
  -name ace_ctrans_read_outstanding_xact_count \
  -type instance \
  -definition { SELECT ace_trans.begin_time AS request_time,
                     ace_trans.end_time as response_time,
                     ace_trans.event as event
                FROM $inst__Transaction AS ace_trans,
                     $inst__svt_axi_port_configuration AS ace_configuration
                WHERE ace_trans.type_path IN ( 'Transaction.READ.MASTER_COHERENT_READ',
                                              'Transaction.READ.SLAVE_COHERENT_READ')
                AND (ace_configuration.axi_interface_type = 'AXI_ACE' OR
                     ace_configuration.axi_interface_type = 'ACE_LITE'); } \
  -description {Indicates the number of outstanding read transactions} \
  -dependent ace_ctrans_read_byte_count

perfSetChart -protocol {AMBA_SVT AXI_SVT} \
  -name ace_ctrans_read_outstanding_xact_count \
  -x request_time \
  -y response_time \
  -type step
```

Aggregating Chart Data into Ranges

Currently, each row of the performance metric results data is shown as a line or a bar in the corresponding chart. However, sometimes it is useful to split the data into ranges on the x-axis and show only a statistical result value for every range. For example, you may want to view how many read or write requests occur every 100 ns.

Performance Analyzer supports this capability by allowing the x-axis to be split into ranges. For the data in each range, the y-values are calculated based on an aggregate function that is specified in the `perfSetChart` Tcl command. The `-x` and `-y` options for the Tcl command (that are previously described) can be used to specify the data to utilize for x-axis and y-axis.

If the x-axis is split into intervals, then for every interval there may be multiple data items. However, only one value is displayed for each interval. This value is a statistical result of all the data items in the interval, which are specified using the `-aggr` option of the `perfSetChart` Tcl command. The following aggregate functions are supported:

Function	Definition
cnt	The number of data items in an interval.
max	The maximum value of the data items in an interval.
min	The minimum value of the data items in an interval.
sum	The sum of all the data items in an interval.
avg	The average value of all the data items in an interval.

Function	Definition
rate	The sum of all the data items in an interval divided by the interval.

The default interval size is specified using the `-step` option of the `perfSetChart Tcl` command. Note that this is the default value and that the end user can change this value from within the GUI or in a batch command file, depending on their requirements as they apply to a specific set of simulation results.

Consider the *AMBA* metric that computes the bandwidth for *ACE* write transactions in MB/s.

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \
-name ace_trans_write_bandwidth_rate \
-type instance \
-definition { SELECT round( cast( substr( ace_transaction.burst_size, 12 ) AS real ) /
      ( 1024 * 1024 * $timeunit_factor * 1e-9 ), 8 ) AS bandwidth,
      ace_transaction.begin_time AS time,
      ace_transaction.event AS event
FROM $inst__Transaction AS ace_transaction,
      $inst__svt_axi_port_configuration AS ace_configuration
WHERE ace_transaction.type_path IN ( 'Transaction.WRITE.MASTER_COHERENT_WRITE',
      'Transaction.WRITE.SLAVE_COHERENT_WRITE' )
      AND ((ace_configuration.axi_interface_type = 'AXI_ACE') OR
      (ace_configuration.axi_interface_type = 'ACE_LITE')); } \
-description {This metric computes the bandwidth for ACE write transactions.} \
-dependent
```

When examining a bandwidth value, it makes sense to look at the bandwidth value for specific intervals of time, rather than for each transaction. This can be accomplished by utilizing the `perfSetChart` command to display a line chart for visualizing the metric, including a rate aggregate function (with a reasonable default step value). Since *Performance Analyzer* does not know the rate units, the units for the columns in the results table must be specified using `perfSetColumnType`, as described in an earlier

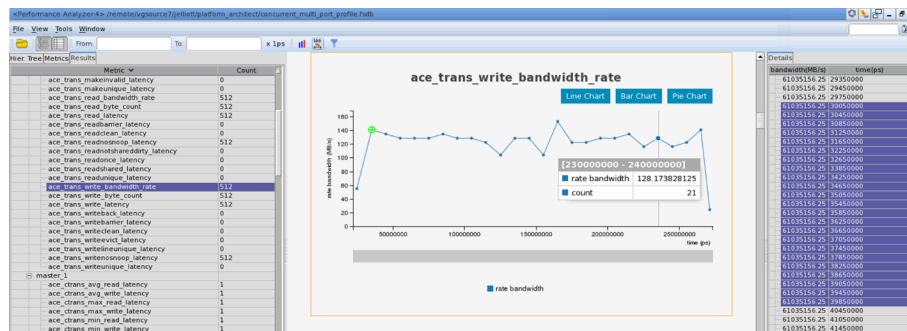
section of this chapter. These commands are shown in the following figure (first the `perfSetColumnType` commands and then the `perfSetChart` command):

```
perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name      ace_trans_write_bandwidth_rate \
                  -column    bandwidth \
                  -type      Bandwidth \
                  -unit      MB/s

perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \
                  -name      ace_trans_write_bandwidth_rate \
                  -column    time \
                  -type      Time \
                  -unit      fsdb time unit

perfSetChart -protocol {AMBA_SVT AXI_SVT} \
             -name      ace_trans_write_bandwidth_rate \
             -step      100000000 \
             -aggr      rate \
             -type      line
```

The resulting graph is shown in the figure below. Note that when an individual data point is selected (as represented by the light green dot), multiple rows in the results table are selected. All the selected items are part of the interval that are selected.



Viewing, Modifying, and Creating Performance Metrics

Performance metrics can be viewed and modified in the *Performance Analyzer* GUI. This topic is described in the earlier chapters.

VIP PreDefined Performance Metrics

Some Synopsys Verification IPs (VIPs) are shipped with predefined performance metrics. The type and the number of metrics is protocol-specific and can vary widely from one protocol to the other protocol.

The *Synopsys AMBA VIP* has one of the most extensive list of predefined performance metric. For example, the figure below shows the predefined metrics for the *AMBA ACE* protocol. There is a similar list of predefined metrics for the *AMBA AXI* protocol.

\$DESIGNWARE_HOME/vip/svt/amba_svt/latest/pa/axi_svt/performance/metrics/ace*	
ace_trans_bandwidth_rate.tcl	ace_ctrans_avg_read_latency.tcl
ace_trans_cleaninvalid_latency.tcl	ace_ctrans_avg_write_latency.tcl
ace_trans_cleanshared_latency.tcl	ace_ctrans_max_read_latency.tcl
ace_trans_cleanunique_latency.tcl	ace_ctrans_max_write_latency.tcl
ace_trans_dvmcomplete_latency.tcl	ace_ctrans_min_read_latency.tcl
ace_trans_dvmmessage_latency.tcl	ace_ctrans_min_write_latency.tcl
ace_trans_evict_latency.tcl	ace_ctrans_outstanding_txn_range.tcl
ace_trans_makeinvalid_latency.tcl	ace_ctrans_read_bus_bandwidth.tcl
ace_trans_makeunique_latency.tcl	ace_ctrans_read_byte_count.tcl
ace_trans_readbarrier_latency.tcl	ace_ctrans_read_request_count.tcl
ace_trans_read_byte_count.tcl	ace_ctrans_write_bus_bandwidth.tcl
ace_trans_readclean_latency.tcl	ace_ctrans_write_byte_count.tcl
ace_trans_read_latency.tcl	ace_ctrans_write_request_count.tcl
ace_trans_readnosnoop_latency.tcl	ace_cinst_master_slave_latency.tcl
ace_trans_readnotshareddirty_latency.tcl	ace_cinst_read_bus_bandwidth_percentage.tcl
ace_trans_readonce_latency.tcl	ace_cinst_read_bus_bandwidth.tcl
ace_trans_readshared_latency.tcl	ace_cinst_read_byte_count_percentage.tcl
ace_trans_readunique_latency.tcl	ace_cinst_read_byte_count.tcl
ace_trans_writeback_latency.tcl	ace_cinst_read_request_count.tcl
ace_trans_writebarrier_latency.tcl	ace_cinst_read_request_percentage.tcl
ace_trans_write_byte_count.tcl	ace_cinst_slave_master_latency.tcl
ace_trans_writeclean_latency.tcl	ace_cinst_write_bus_bandwidth_percentage.tcl
ace_trans_writteevict_latency.tcl	ace_cinst_write_bus_bandwidth.tcl
ace_trans_write_latency.tcl	ace_cinst_write_byte_count_percentage.tcl
ace_trans_writelineunique_latency.tcl	ace_cinst_write_byte_count.tcl
ace_trans_writenosnoop_latency.tcl	ace_cinst_write_request_count.tcl
ace_trans_writeunique_latency.tcl	ace_cinst_write_request_percentage.tcl

Directory Structure and Files

Performance metric information is stored in the same directory structure that is used to define protocol-specific information for *Protocol Analyzer*; the directory structure used by *Protocol Analyzer* is reviewed as follows.

For each protocol that is integrated with *Protocol Analyzer*, there exists information about the protocol that needs to be captured only once. This information is stored in a collection of files, referred as the protocol extension definition. The overall directory structure and files that comprise a protocol extension definition are described in this section.

The purpose of a protocol extension definition is to provide *Protocol Analyzer* with protocol-specific information that enables the tool to organize and display the protocol-related data that was generated during a simulation in a manner that is most conducive to protocol-oriented debugging. The same concept applies to predefined protocol metrics. These metrics need to be defined only once and can then be applied to multiple designs and simulation results associated with the designs.

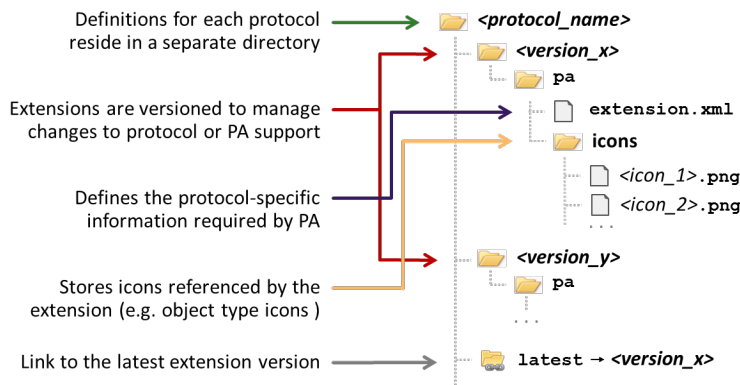
The extension definition for each protocol is contained in a directory structure. The name of the top-most directory is the internal name of the protocol. This name cannot contain any spaces or non-alphanumeric characters, with the exception of underscore ('_') characters. For simplicity, the use of all lower-case letters is recommended. To avoid

naming collisions, consider adding a prefix or suffix that clearly differentiates the protocol name (such as, *snps_xyzbus* or *xyzbus_snps*).

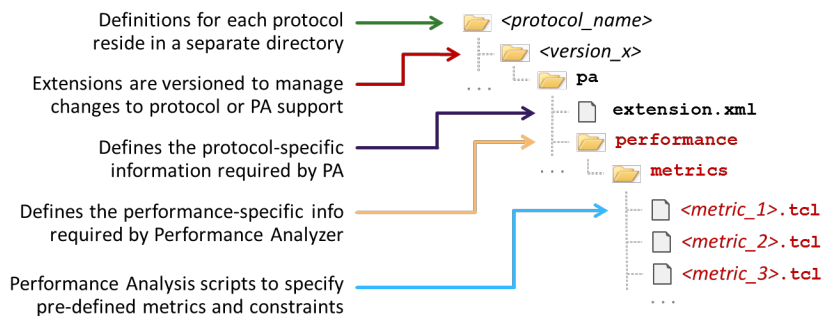
Performance Analyzer locates protocol extension definitions using the same mechanism as *Protocol Analyzer*. Specifically, if the `DESIGNWARE_HOME` environment variable is defined, that directory structure is automatically scanned for protocol extension definitions.

Protocol extension definitions can reside in one or more custom directory locations. These directories are specified by setting the value of the `PA_CUSTOM_VIP_PATH` environment variable. Multiple directory paths can be specified using the colon character (:) as a delimiter⁵.

The following figure shows the overall directory structure and files that comprise a protocol extension definition as defined for *Protocol Analyzer*.



The specification of predefined metrics is simply an extension of this directory structure.



Each Tcl file in the *metrics* directory contains Tcl commands to define the performance metric definition and the visualization characteristics for that performance metric. The commands are the same as those previously discussed.

While it is not required to separate the Tcl commands for each performance metric into a separate file, it is highly recommended for maintenance purposes. This approach also makes it easier to locate the file that contains a performance metric of interest.

5. For additional details, refer to the *VC Apps Protocol Analyzer User Guide*.

For the predefined metrics that are provided with Synopsys VIPs, the names of the performance metrics also follow a specific naming convention. Consider the following performance metric definitions that are discussed earlier in this document: *ace_trans_read_byte_count*, *ace_ctrans_read_byte_count*, and *ace_cinst_read_byte_count*.

Note that,

- Each name begins with the string “ace” to indicate that these metrics are associated with the *AMBA ACE* protocol.
- The second string indicates the kind of metric that is being defined.
- The “trans” string (shorthand for “transaction”) indicates that the metric is evaluated on multiple transactions independently, which creates a results table that contains multiple rows (one for each transaction, as determined by the SQL statement associated with the performance metric definition).
- The “ctrans” string (shorthand for “cross-transaction”) indicates that the metric considers all the transactions together (again, in the manner determined by the SQL statement associated with the performance metric definition). Both “trans” and “ctrans” performance metric definitions are instance-based performance metrics.
- The “cinst” string (shorthand for “cross-instance”) indicates that the metric considers results data from multiple instances. In other words, these are “multi-instance” performance metric definitions.

This is a naming convention that is used by the Synopsys VIPs to make it easier to identify performance metric definitions and is not a requirement of the *Performance Analyzer* tool.

For example, to specify a predefined metric called *ace_cinst_read_byte_count* (that is, the metric that was discussed in the previous section), the commands to define the metric and the visualization characteristics are specified in a Tcl file called *ace_cinst_read_byte_count.tcl* in the *metrics* directory.

The content of the *ace_cinst_read_byte_count.tcl* file is as follows:

```
perfCreateMetric -protocol {AMBA_SVT AXI_SVT} \  
-name ace_cinst_read_byte_count \  
-type multi_instance \  
-definition { SELECT sum( read_byte_count ) AS byte_count  
              FROM $inst_ace_ctrans_read_byte_count; } \  
-description {This metric computes the byte count for all READ transactions for all instances} \  
-dependent ace_ctrans_read_byte_count  
  
perfSetColumnType -protocol {AMBA_SVT AXI_SVT} \  
-name ace_cinst_read_byte_count \  
-column byte_count \  
-type Data \  
-unit Byte
```

Merging Custom Metrics With PreDefined Metrics

The previous section described how to create a directory for a custom protocol that contains performance metric definitions. However, sometimes it is desirable to merge the predefined metrics that are shipped with a Synopsys VIP or transactor with additional custom metrics. At other times, it is desirable to slightly modify the behavior of such predefined metrics to satisfy specific performance measurement requirements.

To achieve this goal, it is necessary to merge the predefined metrics that are shipped with the Synopsys tool and the custom metrics that are independently developed into a single directory structure. Then, the `PA_CUSTOM_VIP_PATH` environment variable can be set to point to this directory. As previously mentioned, multiple directory paths can be specified using the colon character (:) as a delimiter.

This approach can be easily shown with an example. Consider the predefined performance metrics that are shipped with the *AMBA AXI VIP*. The Tcl files that define these metrics can be found in the following directory:

`$DESIGNWARE_HOME/vip/svt/amba_svt/latest/pa/axi_svt/performance/metrics`

To merge these predefined metrics with other metrics (or to modify these metrics), these Tcl files must first be copied to a directory that represents a “custom” protocol definition.

- The first step is to create the custom directory structure, as described in the previous section. For the *AMBA AXI VIP*, this task can be accomplished with the following commands:

```
$ mkdir <custom_protocol_dir>  
$ mkdir <custom_protocol_dir>/axi_svt  
$ mkdir <custom_protocol_dir>/axi_svt/latest  
$ cp -r $DESIGNWARE_HOME/vip/svt/amba_svt/latest/pa/axi_svt  
  <custom_protocol_dir>/axi_svt/latest/pa
```

- Then, new performance metric Tcl files can be added to the directory as follows:

```
<custom_protocol_dir>/axi_svt/latest/pa/performance/metrics
```

- Finally, prior to invoking *Performance Analyzer*, set the `PA_CUSTOM_VIP_PATH` environment variable to point to the custom protocol directory as follows:

```
setenv PA_CUSTOM_VIP_PATH <custom_protocol_dir>
```

To check that the custom performance metrics is loaded, open a *Performance Analyzer* window and examine the list of available metrics in the *Metrics* tab on the left side of the *Performance Analyzer* window.

Summary

This chapter describes how to create custom performance metrics. The primary focus of this chapter is to aid in understanding how the FSDB file transaction data is translated into the internal SQLite database and how to extract performance-related data from that database.

It also briefly describes setting the correct data types for the data that is extracted from the database and how that data can be graphically represented. These two topics are described in detail in the *Performance Analyzer User Guide*, so additional information can be found in that document.

The predefined performance metrics that are provided with select Synopsys Verification IP can also serve as useful examples.

For additional information about SQLite, refer to the SQLite websites or other Internet resources:

https://sqlite.org/lang_corefunc.html

<https://sqlite.org>

For any additional assistance, please contact your local Synopsys support team.

7

Performance Analysis Runtime APIs

The following sections describe the *Performance Analysis Runtime APIs* supported by *Verdi Performance Analyzer*:

- [Introduction](#)
- [API Descriptions](#)
- [Source code and Examples](#)

Introduction

As most of the designs incorporate more complex protocols, the verification engineer's task to evaluate the performance of those designs is a challenging task. It not only takes a significant effort but is also time-consuming.

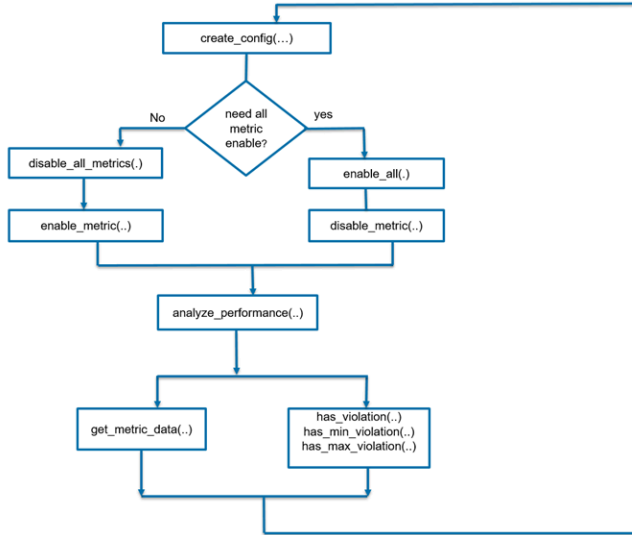
Verdi Performance Analyzer provides a solution to this problem by enabling users to easily analyze the performance of any design. However, many verification engineers need access to performance parameters during the simulation run in the testbench code. The SystemVerilog APIs described in this chapter can provide runtime access to the Performance Data of *Verdi Performance Analyzer*.

Starting with Verdi N-2017.12 release, the performance analysis runtime APIs are designed to address such challenges. Firstly, runtime performance analysis is easy to integrate for regressions. Secondly, with the use of these APIs, the same benefits can be extended to third party, internally developed, or other proprietary protocols.

This chapter highlights the overall runtime performance application integration process as well as the steps that are required to use these APIs in SystemVerilog code.

The following figure shows the API Integration Flow:

Figure 54 API Integration Flow



API Descriptions

new (constructor)

This method constructs a new instance of the `verdi_pa_runtime_analysis` class.

Parameters

Name	Description
<code>fsdb_name</code>	The name of the FSDB file that contains the transaction data
<code>protocol_name</code>	The name of the protocol to process
<code>instance_names</code>	An array of the names of the instances to process
<code>begin_time</code>	An optional begin time to use for performance calculations
<code>end_time</code>	An optional end time to use for performance calculations
<code>time_unit</code>	The time unit of begin time and end time

Prototype

```
function new( input_string fsdb_name,  
              input_string instance_names[],  
              input_time   begin_time = 0.0,  
              input_time   end_time   = 0.0,  
              input_string time_unit   = "" );
```

disable_all_metrics

This method disables all metrics.

Parameters

None

Prototype

```
function void disable_all_metrics();
```

enable_all_metrics

This method enables all metrics.

Parameters

None

Prototype

```
function void enable_all_metrics();
```

disable_metric

This method disables the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric to disable

Prototype

```
function void disable_metric( input_string metric_name );
```

enable_metric

This method enables the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric to enable

Prototype

```
function void enable_metric( input string metric_name );
```

set_metric_constraints

This method sets both minimum and maximum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose constraints is to be set
min_constraint	The value of the minimum constraint
max_constraint	The value of the maximum constraint

Prototype

```
function void set_metric_constraints( input string metric_name,  
                                     input real   min_constraint,  
                                     input real   max_constraint );
```

set_metric_min_constraint

This method sets the minimum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose minimum constraint is to be set
min_constraint	The value of the minimum constraint

Prototype

```
function void set_metric_min_constraint  
    ( input_string    metric_name,  
      input_real      min_constraint );
```

set_metric_max_constraint

This method sets the maximum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose maximum constraint is to be set
max_constraint	The value of the maximum constraint

Prototype

```
function void set_metric_max_constraint( input_string metric_name,  
                                         input_real   max_constraint );
```

clear_metric_constraints

This method clears both minimum and maximum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose constraints are to be cleared

Prototype

```
function void clear_metric_constraints( input string metric_name );
```

clear_metric_min_constraint

This method clears the minimum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose minimum constraint is to be cleared

Prototype

```
function void clear_metric_min_constraint( input string metric_name );
```

clear_metric_max_constraint

This method clears the maximum constraint for the metric with the specified name.

Parameters

Name	Description
metric_name	The name of the metric whose maximum constraint is to be cleared

Prototype

```
function void clear_metric_max_constraint( input string metric_name );
```

analyze_performance

This method analyzes and populates the performance results with the instance of the `verdi_pa_runtime_analysis` class.

Parameters

None

Return Value

The status of the operation (1 = success; 0 = failure).

Prototype

```
function int analyze_performance();
```

get_metric_data

This method returns the performance metric data for the metric with the specified name and the specified instance. For multi-instance metrics, no instance name is specified.

Parameters

Name	Description
metric_name	The name of the metric whose data is to be queried
instance_name	The name of the instance whose data is to be queried, or "" for multi-instance metrics
row_count	The number of rows of data that were produced
column_count	The number of columns of data that were produced
column_names	The names of the columns in the resulting table
data	The performance data table

Return Value

The status of the operation (1 = success; 0 = failure).

Prototype

```
function int get_metric_data( input  string metric_name,
                             input  string instance_name = "",
                             output int  row_count,
                             output int  column_count,
                             output string column_names[],
                             output string data[][] );
```

has_violation

This method indicates if the metric with the specified name violates either minimum or maximum constraint for the specified instance. For multi-instance metrics, no instance name is specified.

Parameters

Name	Description
metric_name	The name of the metric whose minimum constraint is to be checked
instance_name	The name of the instance to be checked, or "" for multi-instance metrics

Return Value

The status of the operation (1 = either a minimum or maximum constraint was violated; 0 = no constraints were violated).

Prototype

```
function int has_violation( input string metric_name = "",  
                           input string instance_name = "" );
```

has_min_violation

This method indicates if the metric with the specified name violates the minimum constraint for the specified instance. For multi-instance metrics, no instance name is specified.

Parameters

Name	Description
metric_name	The name of the metric whose minimum constraint is to be checked
instance_name	The name of the instance to be checked, or "" for multi-instance metrics

Return Value

The status of the operation (1 = the constraint was violated; 0 = the constraint was not violated).

Prototype

```
function int has_min_violation(input string metric_name = "",  
                             input string instance_name = "" );
```

has_max_violation

This method indicates if the metric with the specified name violates the maximum constraint for the specified instance. For multi-instance metrics, no instance name is specified.

Parameters

Name	Description
metric_name	The name of the metric whose maximum constraint is to be checked
instance_name	The name of the instance to be checked, or "" for multi-instance metrics

Return Value

The status of the operation (1 = the constraint was violated; 0 = the constraint was not violated).

Prototype

```
function int has_max_violation(input string metric_name = "",  
                              input string instance_name = "" );
```

Source code and Examples

Source code can be found in the Verdi release at:

```
$VERDI_HOME/share/verdi_perf/sv/src/
```

Examples on how to use the APIs can be found in the Verdi release at:

```
$VERDI_HOME/share/verdi_perf/sv/examples/... /uvm-1.2/integrated/codec/  
tb_env.svh
```